樂升半導體
LEVETOP

LT7689

High Performance Serial Uart TFT Graphics Controller

Data Sheet

V1.3

www.levetop.tw

Levetop Semiconductor Co., Ltd.

Version History

Version	Date	Description
V1.0	2020/5/18	LT7689 Preliminary Release
V1.1	2020/10/25	Update Table 8-1: Protocol List Table
V1.2	2022/02/16	Update Table 7-1 and Table 7-2
V1.3	2022/09/05	Update Table 1-1: QFN-96Pin Dimenstion

Content

Table List

Figure List

1. Introduction

LT7689 is an efficient serial Uart TFT panel controller. It combines a Cortex-M4 MCU and 2D TFT graphical display accelerators. Through Uart interface, **LT7689** can receive commands from the host MCU, and display predefined pictures and contents to the connected TFT panel. In addition to its high-performance M4 MCU, **LT7689** also provides graphics acceleration, PIP (Picture-in-Picture), geometry drawing and other functions. It can improve the efficiency while displaying pictures/contents to the TFT panel and reduce the time it takes for the host MCU to process graphical displays. **LT7689** supports 16/18/24bits TFT RGB panels with a display resolution ranging from 320 x 240 (QVGA) to 1280 x 1024 (SXGA).

LT7689 has an embedded M4 core MCU, with a maximum CPU speed of 150MHz, and contains 508Kbytes Flash, 256Kbytes SRAM, 2D graphics acceleration display, and 128Mb display memory. It provides more than 70 commands, including picture display, GIF animation display, loop chart display, power-on image display, progress bar display, text string display, QR code generation, audio playback, and combining touch screen to achieve touch function, etc. With the editing software and simulation tool provided by Levetop, users can easily get their project done with TFT display functions in no time. Along with the Uart interface for comunicating with the host MCU, **LT7689** also provides multiple groups of SCI (Uart) interfaces that can connect to peripherals such as blue tooth modules, and WiFi modules, etc. USB interfaces, SD cards, analog input AIN, PWM and INT interrupts are also available. These interfaces can also be set up as normal IO interfaces. **LT7689** also has an embedded RTC.

Benefiting from its high-capacity Flash and SRAM, **LT7689** can also be used as a host MCU with TFT controller. The hardware's graphics acceleration engine (BTE) and geometric drawing engine can even support display rotation, picture mirror shooting, picture-in-picture (PIP/child picture) and graphics mixed transparent display, as well as painting points, drawing lines, curves, ellipses, triangles, rectangles, rounded rectangles, cylinders, tables and other functions. **LT7689**'s powerful display capabilities are ideal for electronic products with TFT-LCD screens, such as smart home appliances, automotive dashboards, motorcycle panels, multi-function transaction machines, industrial control applications, electronic instruments, medical beauty equipment, testing equipment, charging equipment, hydropower meters, and smart audio player with TFT panel, etc.

2. System Application

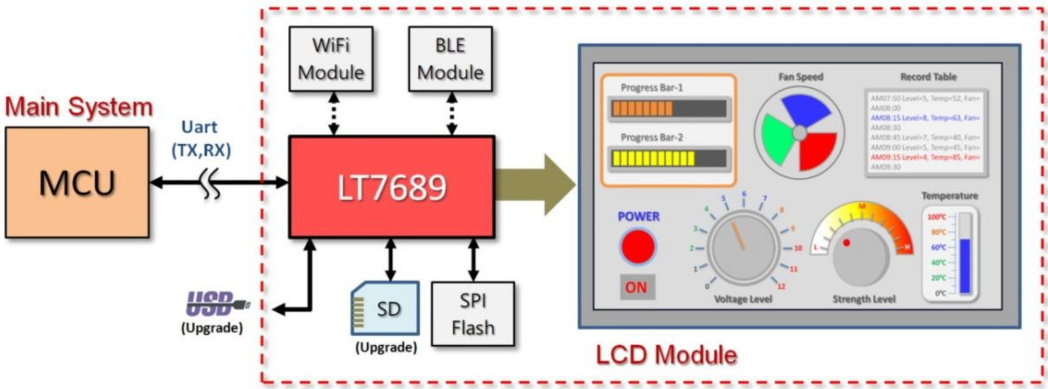


Figure 2-1: LT7689 Applicatio-1

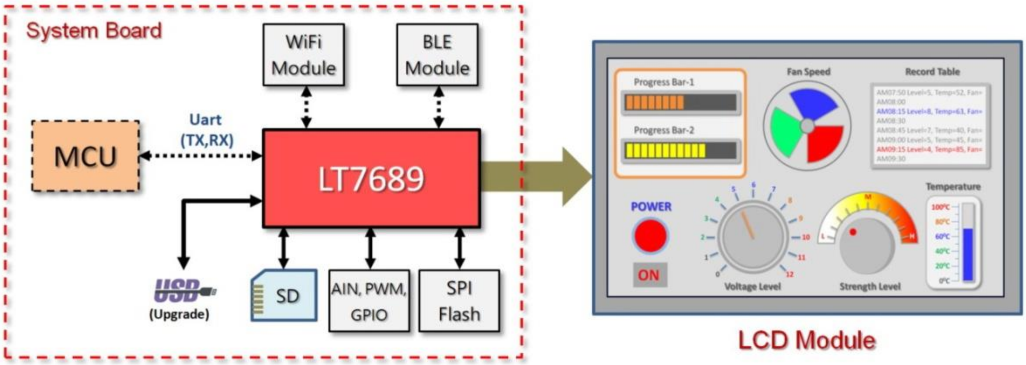


Figure 2-2: LT7689 Application-2

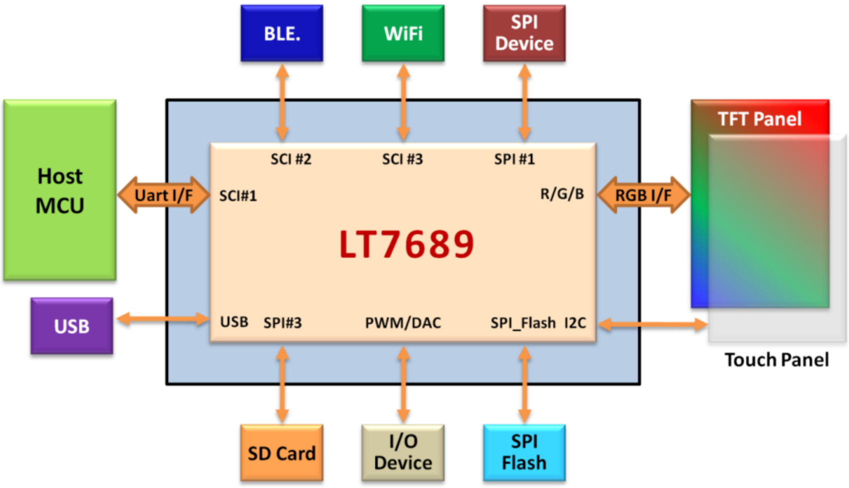


Figure 2-3: Applicaion Block Diagram

3. Internal Block

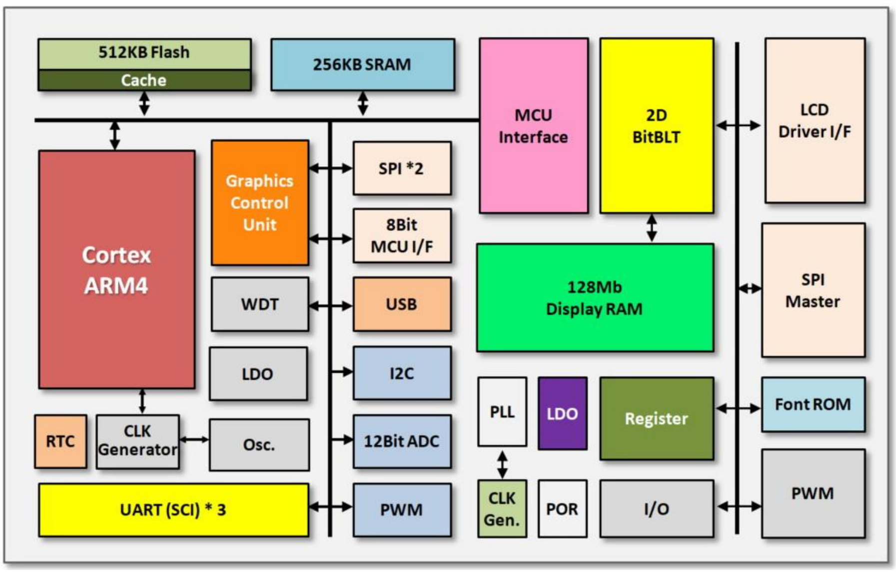


Figure 3-1: Internal Block Diagram

4. Features

System MCU Interface

- Support SCI/Uart and USB Interface.
- Embedded 32Bit Cortex- M4 with 150MHz Clock.

USB Interface

- Supports USB2.0 Full Speed.

SCI (Uart) Interface

- Support three SCI (Serial Communications Interface)

Memory

- MCU Embedded 508K bytes Flash.
- MCU Embedded 256K bytes SRAM.
- TFTController Embedded 128Mb Display RAM.

Display Data Formats

- 1bpp: Monochrome Data (1-bit/Pixel)
- 8bpp: RGB 3:3:2 (1-byte/Pixel)
- 16bpp: RGB 5:6:5 (2-byte/Pixel)
- 24bpp: RGB 8:8:8 (3bytes/Pixel).
  - Index 2:6 (64 Index Colors/Pixel with Opacity Attribute)
  - αRGB 4:4:4:4 (4096 Colors/Pixel with Opacity Attribute)

Support Panel and Resolution

- Support 16, 18, 24 bits RGB Panel.
- Supported Resolution:
  - QVGA : 320*240, LCD Panel
  - WQVGA: 480*272, LCD Panel
  - VGA : 640*480, LCD Panel
  - WVGA : 800*480, LCD Panel
  - SVGA : 800*600, LCD Panel
  - WSVGA : 1024*600, LCD Panel
  - XGA : 1024*768, LCD Panel
  - SXGA : 1280*1024, LCD Panel

Display Functions

- Multiple Display Buffer: Multi buffering allows the main display window to be switched among buffers. Multi buffering allows a simple animation display to be performed by switching the buffers
- Horizontal/Vertical Flip Display: Vertical Flip display functions are available for image data reads. PIP window will be disabled if flip display function enable
- Mirror and Rotation Functions are available for Image Data Writes
- Provide four User-defined 32*32 Pixels Graphic Cursor
- Virtual Display: Virtual display is available to show an image which is larger than LCD panel size. The image may scroll easily in any direction
- Picture-in-Picture (PIP) Display: Two PIP windows area are available: Enabled PIP windows are always displayed on top of the Main window. The PIP1 window is always on top of the PIP2 window.
- Wake-up Display: Wake-up Display is used to quickly show the display data stored in Display RAM. This feature is used when returning from Standby mode or Suspend mode.
- Color Bar: It can display color bar on panel directly. Default resolution is 640 dots by 480 dots.

Bit Block Transfer Engine (BitBLT)

- 2D BTE Engine
- Copy Image with Raster Operators
- Color Depth Conversion
- Solid Fill & Pattern Fill
- Provide User-defined Patterns with 8*8 Pixels or 16*16 Pixels
- Opacity (Alpha-Blend) Control: It blends two images and then generates a new image
  - Chroma-Keying Function: Mixes images with the specified RGB color based on the transparency rate.
  - Window Alpha-Blending Function: Mixes two images based on the transparency rate of the specified region (fade-in and fade-out functions are available).
  - Dot Alpha-Blending Function: Mixes images based on the transparency rate when the target is a graphic image in the RGB format.

Shape Drawing Engine

- Provide Smart Drawing Features: Line, Rectangle, Triangle, Polygon, Poly-Line, Circle, Ellipse, Arc, Rounded-Rectangle and Circle-Rectangle

Text Features

- Embedded 8*16, 12*24, 16*32 Character Sets of ISO/IEC 8859-1/2/4/5.
- User-defined Characters Support Half Size & Full Size for 8*16, 12*24 and 16*32.
- Programmable Text Cursor for Writing with Character.
- Character Enlargement Function *1, *2, *3, *4 for Horizontal/Vertical Direction.
- Support Character Rotates 90 Degree.

SPI Master Interface

- Provide DMA Function: Support Direct Data Transfer from External Serial Flash to Frame Buffer
- Compatible with Standard SPI Specifications
- Provides 16bytes Read FIFO and 16bytes Write FIFO
- Provide Interrupt when Tx FIFO was Empty and SPI Tx/Rx Engine Idle.
- Support Extra 2 STD SPI nterface.

I2C Interface

- MCU Provide I2C Interface.
- Support Standard Mode (100kbps) and Fast Mode (400kbps).

PWM Interface

- MCU Provides 4 PWM Output.
- TFT Controller Embedded Two 16bits Counter and 2 PWM Output.
- Programmable Duty Control of Output Waveform (PWM).

Interrupt Signals

- MCU Provides 6 Interrupt Inputs.
- TFT Controller Provides 1 Interrupt Output.

GPIO

- MCU Supports 10 GPIO.
- TFT Controller Provides 14 GPIO.

Analog Input

- MCU Provide 2 ADC's Analog Input.

Reset

- MCU Provides Power-On-Reset, External Reset, Software Reset, WatchDog Reset and Voltage Detect Reset
- TFT Controller Power-On-Reset, External Reset, Software Reset.

Power Saving Mode

- Provide Standby, Suspend and Sleep Mode.
- Support MCU Wakeup.

Clock Source

- MCU Embedded a Clock up to150MHz.
- Support 32.768Khz External Crystal Oscillator Circuit for RTC.
- Embedded Programmable PLL for Core Clock, LCD Panel's Pixel Clock and Frame Buffer Clock.

Power Supply

- VDD Power: 3.3V +/- 0.3V.
- Embedded 1.1V、1.2V、1.8V LDO

Package

- QFN-96Pin (10*10mm^2)

Temperature

- -40℃~85℃.

5. Pin Description

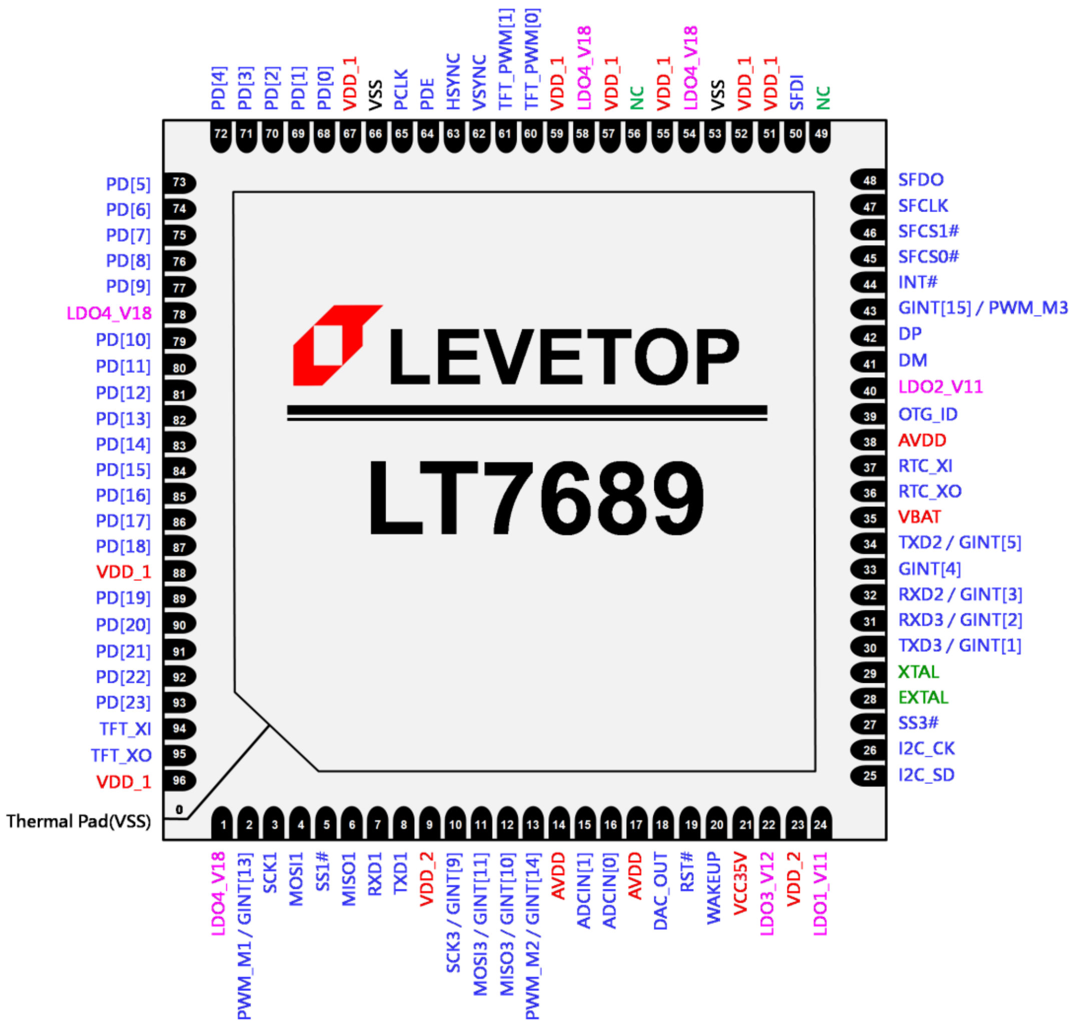


Figure 5-1: LT7689Pin Assignment (QFN-96Pin)

6. Pin Description

6.1 SCI (Uart) Signals

Table 6-1: SCI (Uart) Signals

Pin #	Pin Name	I/O	Pin Description
7	RXD1	I	**Received Data Input #1** This pin is for the Received Data Input of SCI module #1. It is used to connect the external SCI device.
8	TXD1	O	**Transmittion Data Output #1** This pin is for the Transmittion Data Output of SCI module #1. It is used to connect the external SCI device.
32	RXD2	I	**Received Data Input #2** This pin is for the Received Data Input of SCI module #2. It is used to connect the external SCI device. This pin also can be used as GPIO or GINT[3]
34	TXD2	O	**Transmittion Data Output #2** This pin is for the Transmittion Data Output of SCI module #2. It is used to connect the external SCI device. This pin also can be used as GPIO or GINT[5].
31	RXD3	I	**Received Data Input #3** This pin is for the Received Data Input of SCI module #3. It is used to connect the external SCI device. This pin also can be used as GPIO or GINT[2].
30	TXD3	O	**Transmittion Data Output #3** This pin is for the Transmittion Data Output of SCI module #3. It is used to connect the external SCI device. This pin also can be used as GPIO or GINT[1].

6.2 LCD Driver Signals

Table 6-2: LCD Driver Signals

<table>
<tr><th>Pin #</th><th>Pin Name</th><th>I/O</th><th>Pin Description</th></tr>
<tr><td>93~89,
87~79,
77~68</td><td>PD[23:19],
PD[18:10],
PD[9:0]</td><td>O</td><td>LCD Panel Data Bus
TFT LCD data bus output for source driver. User can connect to corresponding RGB bus for different settings.
<table>
<tr><th rowspan="2">Pin Name</th><th colspan="3">TFT-LCD Interface</th></tr>
<tr><th>16bits</th><th>18bits</th><th>24bits</th></tr>
<tr><td>PD[0]</td><td colspan="2">GPIOD[0]</td><td>B0</td></tr>
<tr><td>PD[1]</td><td colspan="2">GPIOD[1]</td><td>B1</td></tr>
<tr><td>PD[2]</td><td>GPIOD[6]</td><td>B0</td><td>B2</td></tr>
<tr><td>PD[3]</td><td>B0</td><td>B1</td><td>B3</td></tr>
<tr><td>PD[4]</td><td>B1</td><td>B2</td><td>B4</td></tr>
<tr><td>PD[5]</td><td>B2</td><td>B3</td><td>B5</td></tr>
<tr><td>PD[6]</td><td>B3</td><td>B4</td><td>B6</td></tr>
<tr><td>PD[7]</td><td>B4</td><td>B5</td><td>B7</td></tr>
<tr><td>PD[8]</td><td colspan="2">GPIOD[2]</td><td>G0</td></tr>
<tr><td>PD[9]</td><td colspan="2">GPIOD[3]</td><td>G1</td></tr>
<tr><td>PD[10]</td><td>G0</td><td>G0</td><td>G2</td></tr>
<tr><td>PD[11]</td><td>G1</td><td>G1</td><td>G3</td></tr>
<tr><td>PD[12]</td><td>G2</td><td>G2</td><td>G4</td></tr>
<tr><td>PD[13]</td><td>G3</td><td>G3</td><td>G5</td></tr>
<tr><td>PD[14]</td><td>G4</td><td>G4</td><td>G6</td></tr>
<tr><td>PD[15]</td><td>G5</td><td>G5</td><td>G7</td></tr>
<tr><td>PD[16]</td><td colspan="2">GPIOD[4]</td><td>R0</td></tr>
<tr><td>PD[17]</td><td colspan="2">GPIOD[5]</td><td>R1</td></tr>
<tr><td>PD[18]</td><td>GPIOD[7]</td><td>R0</td><td>R2</td></tr>
<tr><td>PD[19]</td><td>R0</td><td>R1</td><td>R3</td></tr>
<tr><td>PD[20]</td><td>R1</td><td>R2</td><td>R4</td></tr>
<tr><td>PD[21]</td><td>R2</td><td>R3</td><td>R5</td></tr>
<tr><td>PD[22]</td><td>R3</td><td>R4</td><td>R6</td></tr>
<tr><td>PD[23]</td><td>R4</td><td>R5</td><td>R7</td></tr>
</table>
When the LCD is set to 16/18bpp function mode, some PDs can be defined as GPIO pins.</td></tr>
<tr><td>65</td><td>PCLK</td><td>O</td><td>Pixel Clock
Generic TFT interface signal for panel scan clock. It derives from internal PLL.</td></tr>
</table>

Pin #	Pin Name	I/O	Pin Description
62	VSYNC	O	**VSYNC Pulse** Generic TFT interface signal for vertical synchronous pulse.
63	HSYNC	O	**HSYNC Pulse** Generic TFT interface signal for horizontal synchronous pulse.
64	PDE	O	**Data Enable** Generic TFT interface signal for data valid or data enable.

6.3 SPI Signals

Table 6-3: SPI Signals

Pin #	Pin Name	I/O	Pin Description
3	SCK1	O	**SPI #1 Clock Signal** This is the clock output pin for the 1st SPI. It is connected to the the external SPI device.
5	SS1#	O	**SPI #1 Chip Select** This is the chip select output pin for the 1st SPI.
4	MOSI1	O	**SPI #1 Data Output** This pin is to output data signal to external SPI device. (Master Out Slave In)
6	MISO1	I	**SPI #1 Data Input** This pin is to read data input from external SPI device. (Master Int Slave Out)
10	SCK3 GINT[9]	O	**SPI #3 Clock Signal** This is the clock output pin for the 3rd SPI. It is connected to the external SPI device. This pin also can be used as GINT[9].
27	SS3#	O	**SPI #3 Chip Select** This is the chip select output pin for the 3rd SPI.
11	MOSI3 GINT[11]	O	**SPI #3 Data Output** This pin is to output data signal to external SPI device. (Master Out Slave In) This pin also can be used as GINT[11].
12	MISO3 GINT[10]	I	**SPI #3 Data Input** This pin is to read data input from external SPI device. (Master Int Slave Out) This pin also can be used as GINT[10].

6.4 Serial SPI Flash Signals

Table 6-4: Serial SPI Flash Signals

Pin #	Pin Name	I/O	Pin Description
45	SFCS[0]# GPIOC[3]	IO	**Chip Select 0 for External Serial Flash or SPI device** SPI Chip select pin #0 for serial Flash or SPI device. If SPI master I/F is disabled then it can be programmed as GPIOC[3], and default is input function.
46	SFCS[1]# GPIOC[4]	IO	**Chip Select 1 for External Serial Flash or SPI device** SPI Chip select pin #1 for serial Flash or SPI device. If SPI master I/F is disabled then it can be programmed as GPIOC[4], and default is input function.
47	SFCLK GPIOC[0]	IO	**SPI Serial Clock** Serial clock output for serial Flash/ROM or SPI device. If SPI master I/F is disabled then it can be programmed as GPIO C[0], and default is input function.
48	SFDO GPIOC[1]	IO	**Master Output Slave Input** **Single Mode**: Data input of serial Flash or SPI device. For LT7689, it is output. **Dual Mode**: The signal is used as bi-direction data #0(SIO0). Only valid in serial flash DMA mode. If SPI master I/F is disabled then it can be programmed as GPIO C[1], and default is input function.
50	SFDI GPIOC[2]	IO	**Master Input Slave Output** **Single Mode**: Data output of serial Flash or SPI device. For LT7689, it is input. **Dual Mode**: The signal is used as bi-direction data #1(SIO1). Only valid in serial flash DMA mode. If SPI master I/F is disabled then it can be programmed as GPIOC[2], and default is input function.

6.5 PWM Signals

Table 6-5: PWM Signals

Pin #	Pin Name	I/O	Pin Description
60	TFT_PWM[0] GPIOC[7]	IO	**TFTC PWM #0 Output** This signal is controlled by the registers of the LT7689's internal TFT controller. It is a programmable PWM output signal that can be used to control the backlight of the TFT panel or other devices. The output mode of the TFT_PWM[0] can be set via the registers of the TFT controller. This pin also can be used as GPIOC[7].
61	TFT_PWM[1]	IO	**TFTC PWM #1 Output** This signal is controlled by the registers of the LT7689's internal TFT controller. It is a programmable PWM output signal that can be used to control the backlight of the TFT panel or other devices. The output mode of the TFT_PWM[1] can be set via the registers of the TFT controller.
2	PWM_M1 GINT[13]	IO	**MCU-controlled PWM Output Signal** It can be used as a PWM output or GPIO and is set by internal MCU registers. Normally, this pin outputs a fixed clock signal (8~12MHz) to the TFT clock signal input port – TFT_XI. This pin also can be used as GINT[13].
13	PWM_M2 GINT[14]	IO	**MCU-controlled PWM Output Signal** It can be used as a PWM output or GPIO and is set by internal MCU registers. This pin also can be used as GINT[14].
43	PWM_M3 GINT[15]	IO	**MCU-controlled PWM Output Signal** It can be used as a PWM output or GPIO and is set by internal MCU registers. This pin also can be used as GINT[15].

6.6 USB Signals

Table 6-6: USB Signals

Pin #	Pin Name	I/O	Pin Description
42	DP	IO	**USB Data Positive** These signals are used by the USB module.
41	DM	IO	**USB Data Negative** These signals are used by the USB module.
39	OTG_ID	IO	**USB Detect** This ID signal is used to indicate the ID status of the Mini USB plug, and to judge if it is a master device or a slave device.

6.7 GPIO and Interrupt Signals

Table 6-7: GPIO and Interrupt Signals

Pin #	Pin Name	I/O	Pin Description
30	GINT[1]	IO	**GPIO/INT #1 Signal** This pin is used as GPIO or interrupt input signal GINT[1]. It also can be used as TXD3.
31	GINT[2]	IO	**GPIO/INT #2 Signal** This pin is used as GPIO or interrupt input signal GINT[0]. It also can be used as TRD3.
32	GINT[3]	IO	**GPIO/INT #3 Signal** This pin is used as GPIO or interrupt input signal GINT[0]. It also can be used as RXD2.
33	GINT[4]	IO	**GPIO/INT #4 Signal** This pin is used as GPIO or interrupt input signal GINT[4].
35	GINT[5]	IO	**GPIO/INT #5 Signal** This pin is used as GPIO or interrupt input signal GINT[5]. It also can be used as TXD2..
10	GINT[9]	O	**GPIO/INT #9 Signal** This pin is used as GPIO or interrupt input signal GINT[9]. It also can be used as SCK3.
11	GINT[11]	O	**GPIO/INT #11 Signal** This pin is used as GPIO or interrupt input signal GINT[11]. It also can be used as MOSI3.
12	GINT[10]	I	**GPIO/INT #10 Signal** This pin is used as GPIO or interrupt input signal GINT[10]. It also can be used as MISO3.
2	GINT[13]	IO	**GPIO/INT #13 Signal** This pin is used as GPIO or interrupt input signal GINT[13]. It also can be used as PWM_M1.
13	GINT[14]	IO	**GPIO/INT #14 Signal** This pin is used as GPIO or interrupt input signal GINT[14]. It also can be used as PWM_M2.
43	GINT[15]	IO	**GPIO/INT #15 Signal** This pin is used as GPIO or interrupt input signal GINT[15]. It also can be used as PWM_M3.
44	INT#	O	**Interrupt Output Signal of TFTC** This pin is to indictate the interrupt status of the TFT controller. If an interrupt occurred, then this pin will be pulled low.
60, 46, 45, 50, 48, 47	GPIOC[7] GPIOC[4:0]	IO	**GPIO C Group** These are general purpose I/O. GPIOC are available when PWM and SPI Master functions are disabled. GPIOC[7] is the same pin with PWM[0]. GPIOC[4:0] are multiplex pins that share with {SFCS1#, SFCS0#, SFDI, SFDO, SFCLK}

Pin #	Pin Name	I/O	Pin Description
87, 70	GPIOD[7:6]	IO	**GPIO D Group** These are general purpose I/O. GPIOD[7] is a multiplex pin that shares with PD[18], and GPIOD[6] is a multiplex pins that shares with PD[2]. GPIOD[7,6] are available when LCD Panel interface is set to 16bits.

6.8 Miscellaneous Signals

Table 6-8: Miscellaneous

Pin #	Pin Name	I/O	Pin Description
26	I2C_CK	IO	**I2C Clock** This signal is the I2C clock signal, and normally used for capacitive touch panel.
25	I2C_SD	IO	**I2C Data** This signal is the I2C data signal, and normally used for capacitive touch panel.
18	DAC_OUT	O	**DAC (Digital-to-Analog Converter) Output** This pin is the analog output of DAC, and controlled by internal MCU register. Normally used for audio output.
16 15	ADCIN[0] ADCIN[1]	I	**ADC (Analog-to-Digital Converter) Input** These pins are the analog input of ADC, and controlled by internal MCU register.
20	WAKEUP	I	**Wake-up Input Signal** This is LT7689 wake-up input signal **SD card (SPI mode) detect signal** When SD card (SPI mode) is used, this signal can be used as the SD card (SPI mode) detection signal.
19	RST#	I	**Reset** When RST# = 0, the internal MCU will caused reset.

6.9 Power and Clock Signals

Table 6-9: Power and Clock Signals

Pin #	Pin Name	I/O	Pin Description
94	TFT_XI	I	**TFTC Crystal / External Clock Input** This pin is used to connect to a clock souce for the internal TFTC's PLL. If an external crystal or clock source is used, it should be connected to this pin. In general, this pin is connected to PWM_M1, and the susgested frequency is 8~12MHz.
95	TFT_XO	O	**TFTC Crystal Output** This is an output pin for internal crystal circuit of TFT Controller. It should be connected to the external crystal circuit.
37	32K_XI	I	**32.768Khz Crystal Input** This pin is connected to 32.768Khz X'tal.
36	32K_XO	O	**32.768Khz Crystal Output** This pin is connected to 32.768Khz X'tal.
29	XTAL	I	**12Mhz Crystal Input** This pin is connected to 12Mhz X'tal for USB.
28	EXTAL	O	**12Mhz Crystal Output** This pin is connected to12Mhz X'tal for USB.
35	VBAT	PWR	**3.3V~3.6V RTC Battery Power**
21	VCC35	PWR	**3~5V Power Supply Input**
51, 52, 55, 57, 59, 67, 88, 96	VDD_1	PWR	**3.3V Power Input for TFT Controller**
9, 23	VDD_2	PWR	**3.3V Power Input for MCU**
14, 17, 38	AVDD		**3.3V Power Input for Internal Analog Circuit**
24	LDO1_V11	PWR	**Internal LDO Output #1 (1.1V, for Flash)** These pins must connect 1uF and 0.1uF capacitor to ground.
40	LDO2_V11	PWR	**Internal LDO Output #2 (1.1V for USB)** These pins must connect 1uF and 0.1uF capacitor to ground.
22	LDO3_V12	PWR	**Internal LDO Output #3 (1.2V, for Core)** These pins must connect 1uF and 0.1uF capacitor to ground.
1, 54, 58, 78,	LDO4_V18	PWR	**Internal LDO Output #3 (1.8V, for TFTC)** These pins must connect 1uF and 0.1uF capacitor to ground.
53, 66	VSS	PWR	**Ground Pins**
0	Thermal Pad	-	The back of LT7689 Heat Sink Pad must be grounded.

7. Electrical Characteristics

7.1 Absolute Maximum Ratings

Table 7-1: Absolute Maximum Ratings

Symbol	Parameter	**Value**	**Unit**
V_{DD}	Supply Voltage Range	-0.3 ~ 3.6	V
V_{IN}	Input Voltage Range	-0.3 ~ VDD+0.3	V
V_{OUT}	Output Voltage Range	-0.3 ~ VDD+0.3	V
P_D	Power Dissipation	≦300	mW
T_{OPR}	Operation Temperature Range	-40 ~ 85	℃
T_{ST}	Storage Temperature	-40 ~ 125	℃
T_{SOL}	Soldering Temperature	260	℃

Note:

If used beyond the absolute maximum ratings, LT768x may be permanently damaged. It is strongly recommended that the device is used within the electrical characteristics. If exposed to the condition not within the electrical characteristics, it may affect the reliability of the device. This specification does not guarantee the accuracy of the parameters without a given upper and lower limit value, but it's typical value reasonably reflects the device performance.

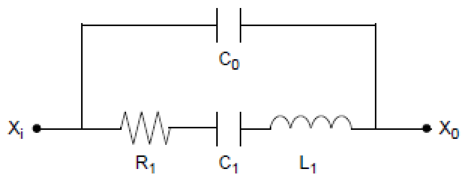
7.2 Electrical Parameter

Table 7-2: Electrical Parameter

Symbol	Parameter	Condition	Min.	Typ.	Max.	Unit
V_{DD1} , V_{DD2}	System Voltage		3.0	3.3	3.6	V
C_{VDD}	Loading Capacitor		1	-	10	uF
I_{OPR}	Operation Current	Note 1		60		mA
I_{STB}	Standby Mode	Note 1		30		mA
I_{SUSP}	Suspend Mode	Note 1		10		mA
I_{SLP}	Sleep Mode	Note 1		7		mA
T_{RMP}	Power Ramp Up Time	VDD Ramp Up to 3.3 V	3.5		35	ms
OSC / PLL						
F_{OSC}	Oscillator Clock	V_{DD} = 3.3 V, Note 2		10		MHz
F_{VCO}	VCO Output Clock Frequency		100		500	MHz
T_{LOCK}	Lock Time	Note 3			500	us
CLK_{MPLL}	MPLL Output Clock (MCLK)	VDD = 3.3 V			133	MHz
CLK_{CPLL}	CPLL Output Clock (CCLK)	VDD = 3.3 V			100	MHz
CLK_{PPLL}	PPLL Output Clock (PCLK)	VDD = 3.3 V			80	MHz
Serial Host Interface						
CLK_{SPI}	SPI Input Clock				50	MHz
Input/Output (CMOS 3-state Output pad with Schmitt Trigger Input, Pull-Up/Down)						
V_{IH}	Input High Voltage		2		3.6	V
V_{IL}	Input Low Voltage		-0.3		0.8	V

Symbol	Parameter	Condition	Min.	Typ.	Max.	Unit
V_{OH}	Output High Voltage		2.4			V
V_{OL}	Output Low Voltage				0.4	V
R_{PU}	Pull up Resistance		34	41	64	KΩ
R_{PD}	Pull down Resistance		33	44	79	KΩ
V_{TP}	Schmitt Trigger Low to High Threshold		1.5		2.1	V
V_{TN}	Schmitt Trigger High to Low Threshold		0.8		1.3	V
V_{HVS}	Hysteresis Voltage		200			mV
I_{LEAK}	Input Leakage Current		-10		+10	μA
V_{SLEW}	Rise/Fall Slew Rate			1.5		V/ns

Note 1: Measured on tester with 8 bit MPU interface and without extra load.



Standard Value : R1 = 50Ω (25-100Ω) , L1 = 3.4mH, C1 = 13fF, C0 = 2.8pF

Figure 7-1: Equivalent Circuit

8. Function Description

LT7689 combines high-performance Cortex-M4 MCU and TFT graphics accelerators. Its internal MCU's main structure is the same as Levetop's LT32U03, and its functions can be referenced to LT32U03 specification and application manual. In addition, the TFT Graphics Accelerator applies Levetop's LT768 hardware architecture, its circuit configuration to the MCU is shown in the following figure.

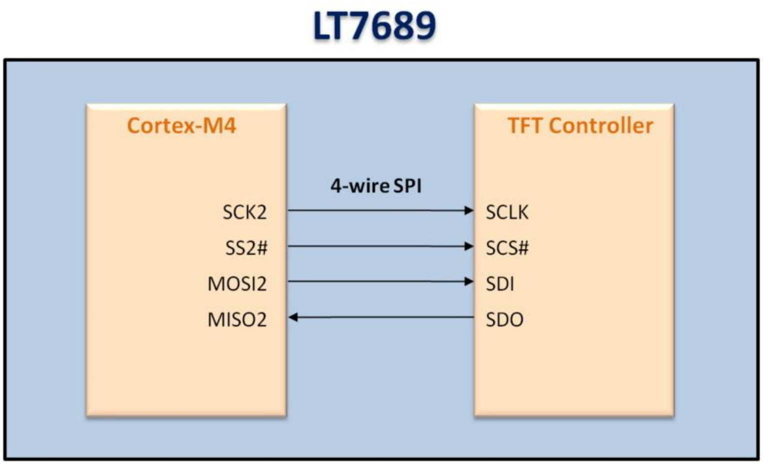


Figure 8-1: The connection of Internal MCU and TFT Controller

LT7689 internal MCU program already contains a Uart protocol that supports about 70 Levetop's Serial TFT panel instructions. Users can design their display flow and contents simply by using the development software provided by Levetop Semiconductor to gather pictures, animation, and other display contents together. Basically, there is no need to modify **LT7689** internal MCU program, nor need to understand **LT7689** internal register and control method. The host MCU only needs to send instructions to **LT7689** through Uart Interface, and to receive and interpret the feedback message sent by **LT7689**. Therefore, using **LT689** can save users a lot of time on display developmpent tasks. For more details about development software and external application schematic, please refer to the LT7689 Uart TFT panel application note - "LT7689_UartTFT_AP Note_Vxx_EN.pdf".

.

Figure 8-2: Serial Usrt TFT Module

8.1 Host MCU and TFT Protocol List

Table 8-1: Protocol List

Main Function	Detail Function	Host Transfer (Panel Receive)						Host Receive (Panel Send)					
		Initial Code (1Byte)	Comm. Code (1Byte)	No. (1Byte)	parameter	CRC Code (2Bytes)	End Code (4Bytes)	Initial Code (1Byte)	Comm. Code (1Byte)	No. (1Byte)	Parameter (1Byte)	CRC Code (2Bytes)	End Code (4Bytes)
Picture Display	Display Pictures	Start	80h	nn		CRC	End	Start	80h	nn	info code	CRC	End
	Cancel Display	Start	85h	nn		CRC	End	Start	84h	nn	info code	CRC	End
	Single/Multi Picture	Start	8Ah	nn		CRC	End	Start	8Ah	nn	info code	CRC	End
	Display Single Picture	Start	8Fh	nn	X, Y, PNG, Pnn	CRC	End	Start	8Fh	nn	info code	CRC	End
	Play In Loop	Start	81h	nn		CRC	End	Start	81h	nn	info code	CRC	End
	Cancel Play In Loop	Start	84h	nn		CRC	End	Start	84h	nn	info code	CRC	End
	Transparent Pictures	Start	82h	nn		CRC	End	Start	82h	nn	info code	CRC	End
	Display GIF	Start	88h	nn		CRC	End	Start	88h	nn	info code	CRC	End
	Cancel GIF	Start	89h	nn		CRC	End	Start	89h	nn	info code	CRC	End
	Buffer Area	Start	8Eh		0, 1	CRC	End	Start	8Eh	00	info code	CRC	End
	Pop Up Picture	Start	D8h	nn		CRC	End	Start	D8h	nn	info code	CRC	End
	Scroll Picture	Start	D9h	nn		CRC	End	Start	D9h	nn	info code	CRC	End
	Cancel Scroll In Loop	Start	DBh	nn		CRC	End	Start	DBh	nn	info code	CRC	End
	Numeric Picture	Start	90h	nn	ddd.d	CRC	End	Start	90h	nn	info code	CRC	End
	True Color Numeric Picture	Start	91h	nn	ddd.d	CRC	End	Start	91h	nn	info code	CRC	End
	Ascii Picture	Start	9Fh	nn	ddd.d	CRC	End	Start	9Fh	nn	信息码	CRC	End
Display Control Button	Full Screen Slide Picture	Start	B4h	nn		CRC	End	Start	B4h	nn	info code	CRC	End
	Display Single Control Button	Start	A0h	nn		CRC	End	Start	A0h	nn	info code	CRC	End
		When Press Control Button						Start	A0h	nn	31h	CRC	End
		When Release Control Button						Start	A0h	nn	30h	CRC	End
	Cancel Display	Start	A1h	Nn		CRC	End	Start	A1h	nn	info code	CRC	End
	Virtual Control Display	Start	A2h	Nn		CRC	End	Start	A2h	nn	info code	CRC	End
		When Press Virtual Button						Start	A2h	nn	31h	CRC	End
		When Release Virtual Button						Start	A2h	nn	30h	CRC	End
	Cancel Display	Start	A3h	nn		CRC	End	Start	A3h	nn	info code	CRC	End
	Display Base Map &All Control Button	Start	9Ch	00		CRC	End	Start	9Ch	00	info code	CRC	End
		After Slide Screen						Start	9Ch	Page No.	info code	CRC	Start
		When Press Control Button						Start	9Bh	Icon ID	31h	CRC	End
		When Release Control Button						Start	9Bh	Icon ID	30h	CRC	End

Main Function	Detail Function	Host Transfer (Panel Receive)						Host Receive (Panel Send)					
		Initial Code (1Byte)	Comm. Code (1Byte)	No. (1Byte)	parameter	CRC Code (2Bytes)	End Code (4Bytes)	Initial Code (1Byte)	Comm. Code (1Byte)	No. (1Byte)	Parameter (1Byte)	CRC Code (2Bytes)	End Code (4Bytes)
Index Picture	Progress Bar	Start	B0h	nn	Value (2 Bytes)	CRC	End	Start	B0h	nn	info code	CRC	End
	Pointer	Start	B1h	nn	Angle (2 Bytes)	CRC	End	Start	B1h	nn	info code	CRC	End
	Ring Pointer	Start	DCh	nn	S_Angle, A_Angle	CRC	End	Start	DCh	nn	info code	CRC	End
	QR Code	Start	98h	nn	String	CRC	End	Start	98h	nn	info code	CRC	End
	Curve Buffer	Start	B2h	Chanel (0~3)	Data (n*2bytes)	CRC	End	Start	B2h	Chanel (0~3)	信息码	CRC	End
	Display Curve	Start	B3h	nn		CRC	End	Start	B3h	nn	信息码	CRC	End
Touch Slider	Set Slider	Start	94h	nn		CRC	End	Start	94h	nn	info code	CRC	End
		When Press Slider						Start	94h	nn	Value (1 Byte)	CRC	End
	Remove Slider	Start	95h	Nn		CRC	End	Start	95h	nn	info code	CRC	End
	Set Ring Slider	Start	96h	Nn		CRC	End	Start	96h	nn	info code	CRC	End
		When Press Ring Slider						Start	96h	nn	Value (1 Byte)	CRC	End
	Remove Ring Slider	Start	97h	nn		CRC	End	Start	97h	nn	info code	CRC	End
Font	Font-1	Start	C0h	nn	String	CRC	End	Start	C0h	nn	info code	CRC	End
	Font-2	Start	C1h	nn	String	CRC	End	Start	C1h	nn	info code	CRC	End
	Font-3	Start	C2h	nn	String	CRC	End	Start	C2h	nn	info code	CRC	End
	Font-4	Start	C3h	nn	String	CRC	End	Start	C3h	nn	info code	CRC	End
	Bold Font-1	Start	D0h	nn	String	CRC	End	Start	D0h	nn	info code	CRC	End
	Bold Font-2	Start	D1h	nn	String	CRC	End	Start	D1h	nn	info code	CRC	End
	Bold Font-3	Start	D2h	nn	String	CRC	End	Start	D2h	nn	info code	CRC	End
	Bold Font-4	Start	D3h	nn	String	CRC	End	Start	D3h	nn	info code	CRC	End
Cursor	Cursor On/Off	Start	86h		00/01/02	CRC	End	Start	86h	nn	info code	CRC	End
	Display Cursor	Start	87h	N	X, Y	CRC	End	Start	87h	N	info code	CRC	End
Backlight Brightness	Set Brightness	Start	BAh		BL (00~0Fh)	CRC	End	Start	BAh	BL (00~0Fh)	info code	CRC	End
	On/Off	Start	BCh		00 or 01	CRC	End	Start	BCh	00 or 01	info code	CRC	End
Wav	Play	Start	B8h		REP(Bit7) + WAV No.	CRC	End	Start	B8h	REP(Bit7) + WAV No	info code	CRC	End
	Stop	Start	B9h			CRC	End	Start	B9h	00	info code	CRC	End
Boot	Boot	Start	9Ah	00		CRC	End	Start	9Ah	00	info code	CRC	End
Combine	Command Combine	Start	9Ah	nn		CRC	End	Start	9Ah	nn	info code	CRC	End
Verify	TFT Verify	Start	8Bh			CRC	End	Start	8Bh	00	info code	CRC	End

Main Function	Detail Function	Host Transfer (Panel Receive)						Host Receive (Panel Send)					
		Initial Code (1Byte)	Comm. Code (1Byte)	No. (1Byte)	parameter	CRC Code (2Bytes)	End Code (4Bytes)	Initial Code (1Byte)	Comm. Code (1Byte)	No. (1Byte)	Parameter (1Byte)	CRC Code (2Bytes)	End Code (4Bytes)
Tft Reset	Reset LT7689	Start	BDh			CRC	End	Start	BDh	00	info code	CRC	End
Geometric	Dot	Start	DFh	nn	X,Y	CRC	End	Start	DFh	nn	info code	CRC	End
	Line	Start	E0h	nn		CRC	End	Start	E0h	nn	info code	CRC	End
	Hollow Circle	Start	E1h	nn		CRC	End	Start	E1h	nn	info code	CRC	End
	Solid Circle	Start	E2h	nn		CRC	End	Start	E2h	nn	info code	CRC	End
	Solid Circle With Frame	Start	E3h	nn		CRC	End	Start	E3h	nn	info code	CRC	End
	Hollow Ellipse	Start	E4h	nn		CRC	End	Start	E4h	nn	info code	CRC	End
	Solid Ellipse	Start	E5h	nn		CRC	End	Start	E5h	nn	info code	CRC	End
	Solid Ellipse With Frame	Start	E6h	nn		CRC	End	Start	E6h	nn	info code	CRC	End
	Hollow Rectangle	Start	E7h	nn		CRC	End	Start	E7h	nn	info code	CRC	End
	Solid Rectangle	Start	E8h	nn		CRC	End	Start	E8h	nn	info code	CRC	End
	Rectangle With Frame	Start	E9h	nn		CRC	End	Start	E9h	nn	info code	CRC	End
	Hollow Rounded Rectangle	Start	EAh	nn		CRC	End	Start	EAh	nn	info code	CRC	End
	Solid Rounded Rectangle	Start	EBh	nn		CRC	End	Start	EBh	nn	info code	CRC	End
	Rounded Rectangle With Frame	Start	ECh	nn		CRC	End	Start	ECh	nn	info code	CRC	End
	Hollow Triangle	Start	EDh	nn		CRC	End	Start	EDh	nn	info code	CRC	End
	Solid Triangle	Start	EEh	nn		CRC	End	Start	EEh	nn	info code	CRC	End
	Triangle With Frame	Start	EFh	nn		CRC	End	Start	EFh	nn	info code	CRC	End
	Hollow Quadrangle	Start	F0h	nn		CRC	End	Start	F0h	nn	info code	CRC	End
	Solid Quadrangle	Start	F1h	nn		CRC	End	Start	F1h	nn	info code	CRC	End
	Hollow Pentagon	Start	F2h	nn		CRC	End	Start	F2h	nn	info code	CRC	End
	Solid Pentagon	Start	F3h	nn		CRC	End	Start	F3h	nn	info code	CRC	End
	Cylinder	Start	F4h	nn		CRC	End	Start	F4h	nn	info code	CRC	End
	Square Cylinder	Start	F5h	nn		CRC	End	Start	F5h	nn	info code	CRC	End
	Table Window	Start	F6h	nn		CRC	End	Start	F6h	nn	info code	CRC	End
Numeric Keyboard	Display Numeric Keyboard	Start	A4h	00		CRC	End	Start	A4h	nn	info code	CRC	End
		When Press Keyboard						Start	A4h	nn	ASCII + info code	CRC	End
		When Press CR Key						Start	A4h	nn	ASCII + info code+ content	CRC	End
	Cancel Keyboard	Start	A5h	00		CRC	End	Start	A5h	nn	info code	CRC	End

Main Function	Detail Function	Host Transfer (Panel Receive)						Host Receive (Panel Send)					
		Initial Code (1Byte)	Comm. Code (1Byte)	No. (1Byte)	parameter	CRC Code (2Bytes)	End Code (4Bytes)	Initial Code (1Byte)	Comm. Code (1Byte)	No. (1Byte)	Parameter (1Byte)	CRC Code (2Bytes)	End Code (4Bytes)
Full Keyboard	Display Full Keyboard	Start	A6h	00		CRC	End	Start	A6h	nn	info code	CRC	End
		When press CR key						Start	A6h	nn	info code+ (ASCII+GB2312)	CRC	End
	Cancel Full Keyboard	Start	A7h	00		CRC	End	Start	A7h	nn	info code	CRC	End
TFT Detect	Online Detect	Start	BEh			CRC	End	Start	BEh	00	5Ah, or 55h	CRC	End
	Version Detect	Start	BFh			CRC	End	Start	BFh	MCU Code(5) + Module Info. (42)	info code	CRC	End
Clock Setting	Set Clock	Start	8Ch		Y, M, D, H, M, S, W (7 Bytes)	CRC	End	Start	8Ch	00	info code	CRC	End
	Read Clock	Start	8Dh			CRC	End	Start	8Dh	Y, M, D, H, M, S, W (8)	info code	CRC	End
Display Clock	Display Digital Time, Date	Start	92h	nn		CRC	End	Start	92h	nn	info code	CRC	End
	Display Analog Time, Date	Start	93h	nn		CRC	End	Start	93h	nn	info code	CRC	End
	Display Week	Start	9Dh	nn		CRC	End	Start	9Dh	nn	info code	CRC	End
Register	Execute 9A Command	Start	CAh	Reg		CRC	End	Start	CAh	nn	info code	CRC	End
	Set Register	Start	CBh	Reg		CRC	End	Start	CBh	nn	info code	CRC	End
	Write Data	Start	CCh	Data		CRC	End	Start	CCh	nn	info code	CRC	End
	Read Data	Start	CDh	00		CRC	End	Start	CDh	Data	info code	CRC	End
	Register Data Plus 1	Start	CEh	Reg		CRC	End	Start	CEh	nn	info code	CRC	End
	Register Data Minus 1	Start	CFh	Reg		CRC	End	Start	CFh	nn	info code	CRC	End

8.2 RS-232(UART) Communication Protocol

When the host MCU sends command to LT7689 through the UART port, in addition to the Command Code, Serial Number, and Command Parameters, it should also add 1 Byte of Start Code (fixed as 0xAA), 2 Bytes of CRC Code and 4 Bytes of End Code (fixed as 0xE4, 0x1B, 0x11 and 0xEE). The command information is shown in Table 8-2:

Table 8-2: Receive Command Information

Start Code	Command Code	Serial Number	Command Parameters	CRC	End Code
0xAA (1 Byte)	1 Byte	1 Byte	n Bytes	2 Bytes	**0xE4, 0x1B, 0x11, 0xEE** (4 Bytes)

LT7689 will respond 10 Bytes messages after receiving commands from host MCU, including Start Code, Command Code, Serial Number, Information Code, CRC Code, and End Code. The first Byte is the Start Code, followed by the command received, the third is Serial Number. Then the forth is Information Code that returns the execution result of the panel, the fifth and sixth are CRC codes, and the last four Bytes are the End Codes.

Table 8-3: Feedback Information

Start Code	Command Code	Serial Number	Information Code	CRC Code	End Code
0xAA (1 Byte)	1 Byte	Normal command (1 Byte) 8Dh command (8 Bytes) BFh command (47 Bytes)	1Byte 0x00: Command executed 0x01: Command parameter error 0x02: Non-exist command 0x03: Command's configuration Overflow 0x04: CRC correction error 0x05: Flash Data Exception BEh command: 0x5A: Ready 0x55: Not Ready 94h 94h Command: the percentage position of whole progress bar A0h, 9Ch/9Bh Command: 0x31: Press Icon 0x30: Release Icon	2 Bytes	0xE4, 0x1B, 0x11, 0xEE (4 Bytes)

In the feedback information, serial numbers may represent different meanings in some commands. For example,in 9Ch command, it means the page number; in BAh command, it means the backlight brightness; in B8h, it means WAV No., in 8Dh, it has 8 Bytes and means the clock information; in BFh command, it has 47 Bytes which represent the TFT panel information. Please refer to LT7689 Application Note for detail.

9. Clock and Reset Signals

9.1 Clock Signals

The internal clock signals of the LT7689 are shown in Figure 9-1 below. The MCU section consists of an external 32KHz crystal oscillator circuit with a separated power supply, and two internal high precision clock circuits (150MHz). The TFT controller section consists of an external 10MHz crystal oscillator circuit and three internal PLL circuits. The clock signal can be generated from a PWM outputs of the MCU (such as PWM_M1) or using an external 10MHz crystal oscillator circuit (shown in Figures 9-2 below).

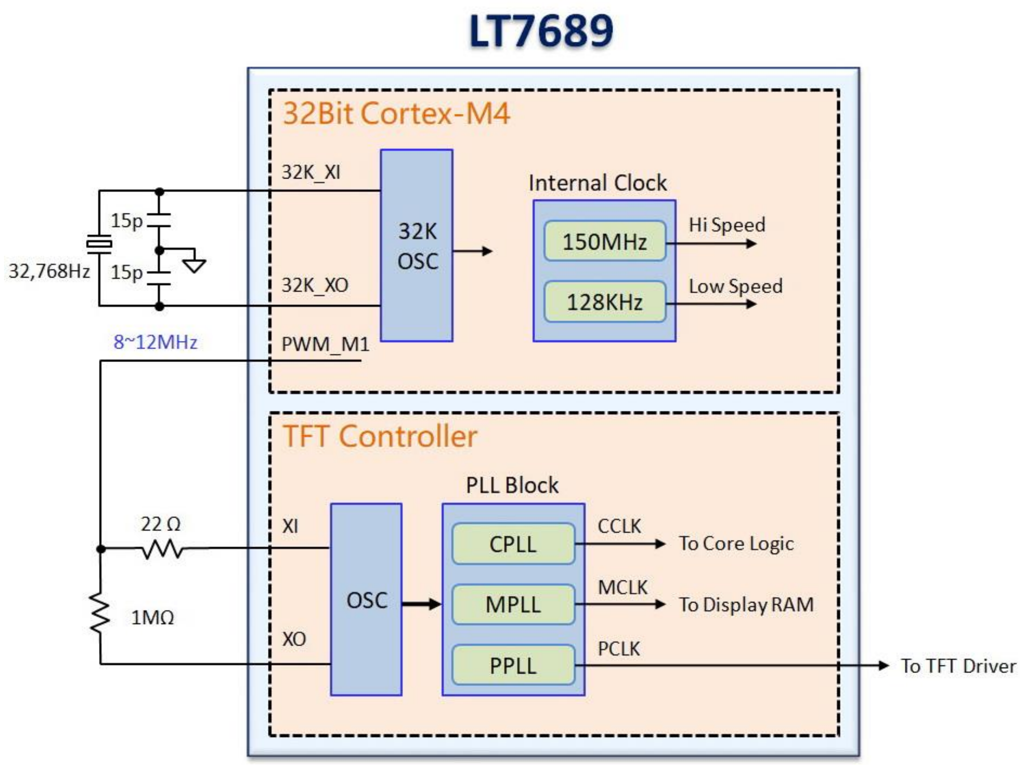


Figure 9-1: LT7689's Clock Structure

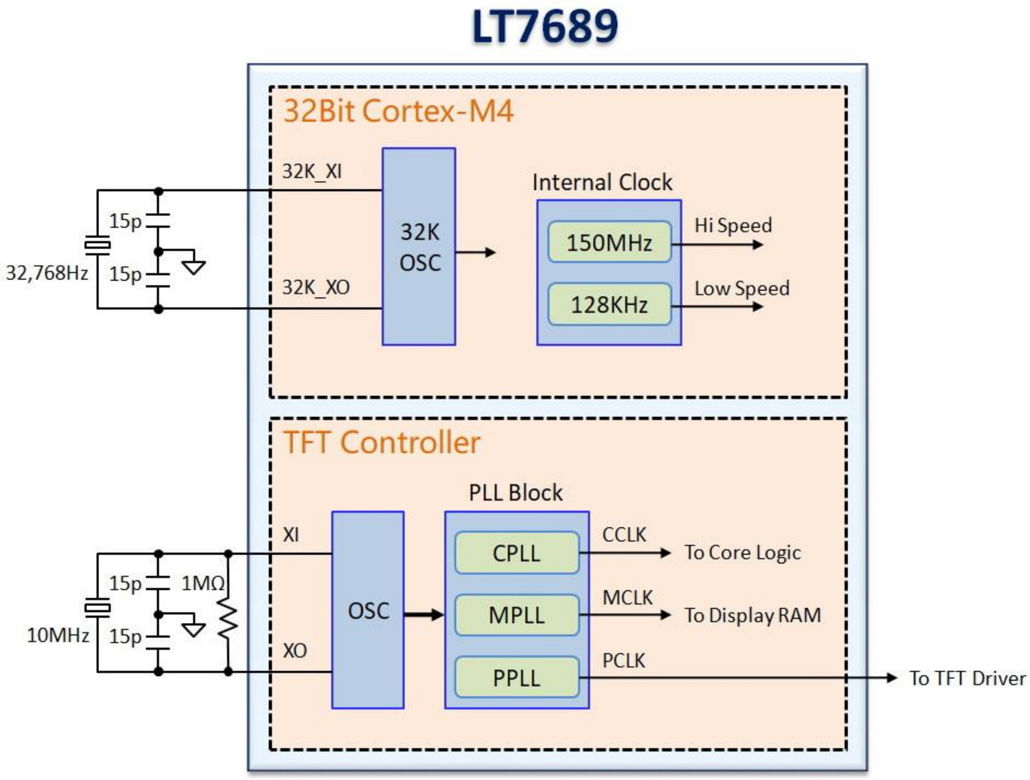


Figure 9-2: TFT Use External Xtal Oscillator

The external 32KHz crystal of the MCU is independently powered and can be used for RTC. The system clock can run at 72MHz at full speed or at 128KHz to reduce the power consumption. For more information, please refer to the LT32U03 specification and application manual.

The internal TFT Controller has three embedded PLL circuits to generate three clock source for internal circuit operations:

- CPLL : Provide **CCLK** for Host interface, BTE Engine, Graphics Engine, and Text DMA data transfer etc…
- MPLL : Provide **MCLK** for internal Display RAM
- PPLL : Provide **PCLK** for TFT-LCD's Scan Clock.

The three PLL are operated independendly. The PLL output frequency is calculated from the following formula:

$$F_{OUT} = XI * (N / R) \div OD$$

In the above formula, XI is the external Oscillator / Clock input. The input frequency "XI/R" is no less than 1MHz, and the default value is 1MHz. "R" is Input Divider Ratio, its value is between 2 ~ 31. "OD" is Output Divider Ratio that must be 1, 2 or 4. "N" is the Feedback Divider Ratio of Loop that indicated by 9bits which between 2 ~ 511.

Table 9-1: PLL Register Setting (1)

R[4:0]	Input Divider Ratio (R)	N[8:0]	Feedback Divider Ratio (N)
00010	2	000000010	2
00011	3	000000011	3
00101	4	000000101	4
: : :	: : :	: : :	: : :
11101	29	111111101	509
11110	30	111111110	510
11111	31	111111111	511

Table 9-2: PLL Register Setting (2)

OD[1:0]	Input Divider Ratio (OD)
00	1
01	2
10	3
11	4

For example, XI is 10MHz, R[4:0] is 01010 (i.e. 10), N[8:0] is 100000000 (i.e. 256), OD[1:0] is 11 (i.e. 4), then:

$$F_{OUT} = 10MHz * (256 / 10) \div 4 = 64MHz$$

The design rule of three clock are:

1. **CCLK * 2 >= MCLK >= CCLK**
2. **CCLK >= PCLK * 1.5**

In general, TFT manufacturers will advise the Pixel Clock(PCLK) for the best display effect, based on their TFT characteristics.Therefore, user can setup the register according to the requirements of the PCLK, and then apply the above rules to set up CCLK and MCLK. According to the different resolution of LCD panel, the PLL output should generate different clock frequency. For example, if LCD panel resolution is 640*480, the recommend values are: PCLK = 20MHz, MCLK = 40MHz, CCLK = 40MHz, then the register settings are as following:

PCLK	= XI * (N / R) ÷ OD = 10MHz * (80 / 10) ÷ 4 = 20MHz	REG[05h] : OD = 11b, R = 01010b REG[06h] : N = 01010000b
MCLK	= XI * (N / R) ÷ OD = 10MHz * (160 / 10) ÷ 4 = 40MHz	REG[07h] : OD = 11b, R = 01010b REG[08h] : N = 10100000b
CCLK	= XI * (N / R) ÷ OD = 10MHz * (160 / 10) ÷ 4 = 40MHz	REG[09h] : OD = 11b, R = 01010b REG[0Ah] : N = 10100000b

Another example, if resolution is 800*480, the recommend values are: PCLK = 25MHz, MCLK = 50MHz, CCLK = 50MHz. We can setup the registers' values as following:

PCLK	= XI * (N / R) ÷ OD = 10MHz * (100 / 10) ÷ 4 = 25MHz	REG[05h] : OD = 11b, R = 01010b REG[06h] : N = 01100100b
MCLK	= XI * (N / R) ÷ OD = 10MHz * (200 / 10) ÷ 4 = 50MHz	REG[07h] : OD = 11b, R = 01010b REG[08h] : N = 11001000b
CCLK	= XI * (N / R) ÷ OD = 10MHz * (200 / 10) ÷ 4 = 50MHz	REG[09h] : OD = 11b, R = 01010b REG[0Ah] : N = 11001000b

Table 9-3: PLL Register Setting Example

Registers	640*480	800*480
REG[05h], PPLLC1	11010100b	11010100b
REG[06h], PPLLC2	01010000b	01100100b
REG[07h], MPLLC1	11010100b	11010100b
REG[08h], MPLLC2	10100000b	11001000b
REG[09h], CPLLC1	11010100b	11010100b
REG[0Ah], CPLLC1	10100000b	11001000b

Attention:

1. The value of Register R[4:0] must be greater than Register OD.
2. When the Pixel Clock "PCLK" is less than or equal to 8MHz, set R directly to 4, and OD to 2.

9.2 Reset

LT7689 has five reset sources. The built- inMCU and TFT controllers have their own Power -On Rset, External Reset input, and Software Reset. The “Watchdog Timer reset” and “Voltage Detection Reset” are unique reset states for MCU. For its operation details, please refer to the LT32U03 specification and application manual.

- Power on Reset
- External Reset Pin
- Software Reset
- Watchdog Timer Reset
- Programmable Voltage Detect Reset

9.2.1 Power-on Reset

The MCU and TFT controllers in LT7689 have their own POR (Power On Reset) circuit. It is an active low signal and may output to external circuits by RST# pin to synchronize the whole system. When system power (3.3V) on, internal reset will be activated until internal power is stable and then de-active after 256 OSC(X’tal Oscillator) clocks.

9.2.2 External Reset

LT7689 has two external reset input signals, RST1# is the reset signal for the TFT controller, and RST2# is the reset signal of the MCU. The external reset signal RST1# can synchronize LT7689 with the external system. The external reset signal must be stable for at least 256 crystal clocks (OSCs) before it can be recognized. Before starting to set LT7689, the MCU should check the bit1-operating mode status indicator of the status register STSR to ensure that LT7689 is currently in a "normal operating state".

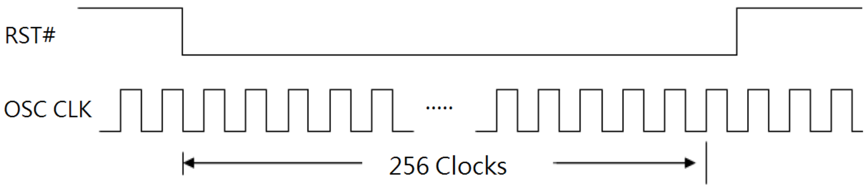


Figure 9-3: External Reset Signal

9.2.3 Software Reset

If the MCU write 1 to register REG[00h] bit0 of TFT Controller, LT7689 will be reset by software. The software reset will only reset the internal state machine of LT7689, and the other registers values will not be affected or cleared. After the software reset is complete, the REG[00h] bit0 will automatically be cleared to 0.

10. I/O Interface and Data Format

10.1 Communication Interface of System

The communication mode between LT7689 and host MCU is through the UART interface. The host MCU is responsible for sending display commands, while LT7689 displays information such as pictures on the TFT Panel according to the display command. Levetop provides complete development and simulation environment. All display sequences are planned and formulated by Levetop. For more information, please refer to our application manual.

In addition, host MCU can update LT7689 internal program or SPI Flash data via the USB interface.

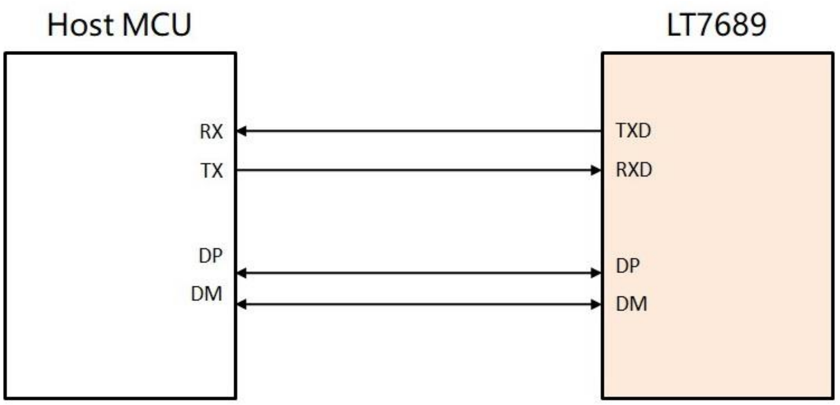


Figure 10-1: The Connection of System MCU and LT7689

10.2 IO Interface of MCU

LT7689's internal MCU module also offers many IO interfaces, as shown below in the left half of Figure 10-2. It includes 2 analog inputs, multiple interrupt signal inputs and 3 PWM signal outputs. These interfaces are set by the internal MCU registers and can also be used as normal GPIO. The Pin names are consistent with Levetop's LT32U03. The detailed operation can be found in the LT32U03 specification and application manual. And in the right half of Figure 10-2, the TFT controller also provides some GPIO interface. It can be used as an extension of the MCU interface. Usually these GPIO interfaces are shared with other control signals, as shown in Table 10-1 below. The GPIO pins can be used only when these control signals are prohibited.

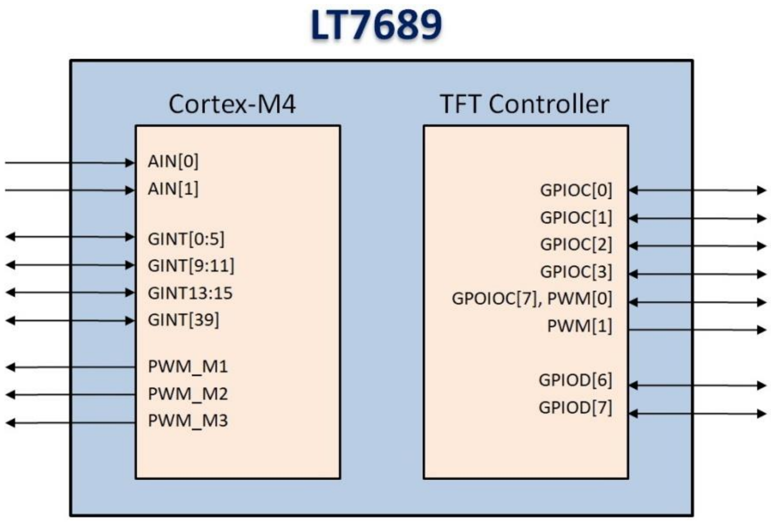


Figure 10-2: Internal I/O Interface

10.3 IO Interface of TFTC

Some GPIO interfaces are also provided within the TFT controller, as shown in the right half of the Figure 10-2 as above. These IO ports can be used as an extension of the MCU interface. Usually these GPIO signlas are shared with other control signals. Please refer to Table 10-1 below. These control signals can only be used as GPIO ports if their control functions are prohibited.

Table 10-1: The Alternate Pins of GPIO and Control Signals

GPIO Pins	Alternate Signals
GPIOC[7]	TFT_PWM[0]
GPIOC[4:0],	{ SFCS1#, SFCS0#, SFDI, SFDO, SFCLK }
GPIOD[7:6],	{ PD[18], PD[2] }

The setting of these GPIO interfaces is determined by the Register REG F0h~F6h of the TFT Controller. Please refer to the Chapter 19 “Register Description”.

10.4 Pulse Width Modulation (PWM) of TFTC

TFT Controller has built-in PWM functions, and provides two PWM signals output: TFT_PWM[0] and TFT_PWM[1]. It also has embedded two 16bits counters Timer-0 and Timer-1, and its action is related to the output state of the PWM signals. For example, before using the TFT_PWM[0], the Host must set Timer-0 count registers (TCNTB0, REG[8Ah-8Bh],) and Timer-0 count comparison registers (TCMPB0, REG[88h-89h]). After the PWM function is enabled, the Timer-0 counter will first load the TCNTB0 value and start counting down according to the PWM clock frequency. When the value of the Timer-0 counter equals the value of the TCMPB0 register, PWM will active. This means if the original state of TFT_PWM[0] is 0, it will change to 1. The Timer-0 counter will continue to count down, and when Timer-0 equals to 0 then an interrupt will be generated. The TFT_PWM[0]'s state will be back to the original 0, and also automatically reload TCNTB0 value. The above procedure represents a complete PWM cycle.

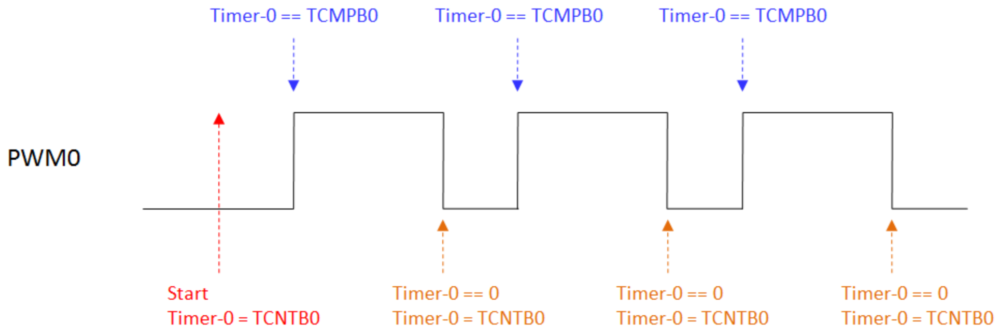


Figure 10-3: TFT_PWM[0] Output Wave Form

Based on the above waveform and description, the duty cycle of TFT_PWM[0] is determined by comparison registers (TCMPB0, REG[88h-89h]). For example, to generate a voltage of approximate DC potential by TFT_PWM[0], when the TFT_PWM[0] is initially set to 0, the larger value of the TCMPB0 is set, the higher equivalent voltage on TFT_PWM[0] will be. On the other hand, when the TFT_PWM[0] is initially set to 1, the smaller value of the TCMPB0 is set, the higher equivalent voltage on TFT_PWM[0] will be.

Note: The automatic reload feature of TFT_PWM[0] (REG[86h] bit1) must be turned on, and when Timer-0 equals to 0, it will reload the value of the TCNTB0 automatically. Therefore, if the MCU changes TCNTB0 or TCMPB0 value before Timer-0 equals to 0, PWM can produce different duty-cycle waveform.

10.4.1 TFT_PWM Clock Source

The TFT_PWM's clock source comes from CCLK(System Clock), while the Timer-0 and Timer-1 base frequencies are determined by register PSCLR (REG[84h]):

PWM_CLK = CCLK / (Prescaler + 1)

The clock source of Timer is determined by the respective frequency registers (REG[85h]). The Timer Divisor provides four options: 1, 1/2, 1/4, 1/8 for the Timer's clock. For example the REG[85h] Bit[5:4] = 10b, then the Timer-0 count Clock = System_clock/4. Please refer to the Register Description of Chapter 19 for REG[84H] and REG[85h].

10.4.2 TFT_PWM Output

The output of PWM can also be set to a fixed high or low level. For TFT_PWM[0] , first turn off the automatic reload feature (REG[86h] bit1 = 0), and then stop counting down Timer-0 (REG[86h] bit0 = 0), if Timer-0 < TCMP0, then PWM output high. If Timer-0 > TCMP0, then PWM output low (assuming the reverse phase is closed, REG[86h] bit2=0). The output state of the TFT_PWM[0] can be set to the inverse phase by REG[86h] bit2.

In addition, TFT_PWM[0] and TFT_PWM[1] are shared output pins that can be used for other purposes, please refer to the following description of Register REG[85h] bit[3:0].

Table 10-2: REG[85h] Description

Bit	Description
3-2	**TFT_PWM[1] Function Control** 0xb: TFT_PWM[1] output system error flag (Scan FIFO POP error or Memory access out of range) 10b: TFT_PWM[1] output PWM timer 1 event or invert of PWM timer 0 11b: TFT_PWM[1] output Oscillator Clock
1-0	**TFT_PWM[0] Function Control** 0xb: TFT_PWM[0] becomes GPIOC[7] 10b: TFT_PWM[0] output PWM Timer 0 11b: TFT_PWM[0] output Core Clock (CCLK)

TFT_PWM[0] and TFT_PWM[1] can also be set to complementary outputs. In this case, the output state of TFT_PWM[1] follows the setting and control of TFT_PWM[0] , except that it is a TFT_PWM[0] reverse output state:

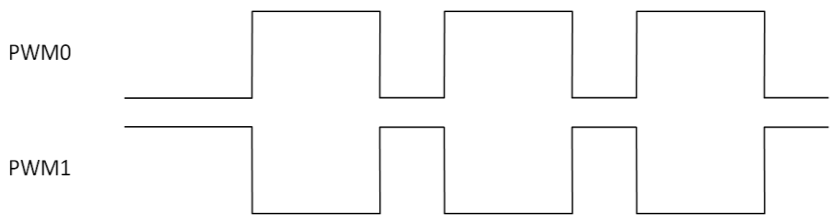


Figure 10-4: The Complementary Outputs of TFT_PWM[0] and TFT_PWM[1]

In some applications, when using Complementary Outputs of TFT_PWM[0] and TFT_PWM[1] , changing state of TFT_PWM[0] and TFT_PWM[1] at the same time may cause excessive current. LT768x provides a dead-zone timing control to stagger the output state of TFT_PWM[0] and TFT_PWM[1]. The dead-zone length of PWM is set by register REG[87h] (DZ_LENGTH):

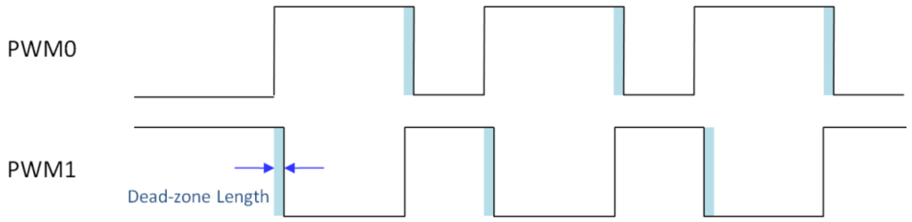


Figure 10-5: Dead-zone of TFT_PWM[0] and TFT_PWM[1]

10.5 Display Input Data Format

LT7689 supports Monochrome, 256 color, 65K color, 256K color and 16.7M color for TFT panel. The data for RGB colors are arranged in Display RAM as follows:

- 1bpp : Monochrome (1bit/pixel)
- 8bpp : Color RGB 3:3:2 (1 byte/pixel)
- 16bpp : Color RGB 5:6:5 (2bytes/pixel)
- 24bpp : Color RGB 8:8:8 (3bytes/pixel, or 4bytes/pixel)

The following examples are based on 8bits MCU, 16bits MCU and display color (RGB) in different data format.

10.5.1 Input Data without Opacity (RGB)

Table 10-3: 8bits MCU, 1bpp Monochrom Mode

Order	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0
2	P_{15}	P_{14}	P_{13}	P_{12}	P_{11}	P_{10}	P_9	P_8
3	P_{23}	P_{22}	P_{21}	P_{20}	P_{19}	P_{18}	P_{17}	P_{16}
4	P_{31}	P_{30}	P_{29}	P_{28}	P_{27}	P_{26}	P_{25}	P_{24}
5	P_{39}	P_{38}	P_{37}	P_{36}	P_{35}	P_{34}	P_{33}	P_{32}
6	P_{47}	P_{46}	P_{45}	P_{44}	P_{43}	P_{42}	P_{41}	P_{40}

Table 10-4: 8bits MCU, 8bpp Mode (RGB 3:3:2)

Order	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	R_0^7	R_0^6	R_0^5	G_0^7	G_0^6	G_0^5	B_0^7	B_0^6
2	R_1^7	R_1^6	R_1^5	G_1^7	G_1^6	G_1^5	B_1^7	B_1^6
3	R_2^7	R_2^6	R_2^5	G_2^7	G_2^6	G_2^5	B_2^7	B_2^6
4	R_3^7	R_3^6	R_3^5	G_3^7	G_3^6	G_3^5	B_3^7	B_3^6
5	R_4^7	R_4^6	R_4^5	G_4^7	G_4^6	G_4^5	B_4^7	B_4^6
6	R_5^7	R_5^6	R_5^5	G_5^7	G_5^6	G_5^5	B_5^7	B_5^6

Table 10-5: 8bits MCU, 16bpp Mode (RGB 5:6:5)

Order	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	G_0^4	G_0^3	G_0^2	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3
2	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	G_0^7	G_0^6	G_0^5
3	G_1^4	G_1^3	G_1^2	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3
4	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	G_1^7	G_1^6	G_1^5
5	G_2^4	G_2^3	G_2^2	B_2^7	B_2^6	B_2^5	B_2^4	B_2^3
6	R_2^7	R_2^6	R_2^5	R_2^4	R_2^3	G_2^7	G_2^6	G_2^5

Table 10-6: 8bits MCU, 24bpp Mode (RGB 8:8:8)

Order	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3	B_0^2	B_0^1	B_0^0
2	G_0^7	G_0^6	G_0^5	G_0^4	G_0^3	G_0^2	G_0^1	G_0^0
3	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	R_0^2	R_0^1	R_0^0
4	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3	B_1^2	B_1^1	B_1^0
5	G_1^7	G_1^6	G_1^5	G_1^4	G_1^3	G_1^2	G_1^1	G_1^0
6	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	R_1^2	R_1^1	R_1^0

Table 10-7: 16bits MCU, 1bpp Mono Mode -1

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0
2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	P_{15}	P_{14}	P_{13}	P_{12}	P_{11}	P_{10}	P_9	P_8
3	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	P_{23}	P_{22}	P_{21}	P_{20}	P_{19}	P_{18}	P_{17}	P_{16}
4	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	P_{31}	P_{30}	P_{29}	P_{28}	P_{27}	P_{26}	P_{25}	P_{24}
5	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	P_{39}	P_{38}	P_{37}	P_{36}	P_{35}	P_{34}	P_{33}	P_{32}
6	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	P_{47}	P_{46}	P_{45}	P_{44}	P_{43}	P_{42}	P_{41}	P_{40}

Table 10-8: 16bits MCU, 1bpp Mono Mode -2

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	P_{15}	P_{14}	P_{13}	P_{12}	P_{11}	P_{10}	P_9	P_8	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0
2	P_{31}	P_{30}	P_{29}	P_{28}	P_{27}	P_{26}	P_{25}	P_{24}	P_{23}	P_{22}	P_{21}	P_{20}	P_{19}	P_{18}	P_{17}	P_{16}
3	P_{47}	P_{46}	P_{45}	P_{44}	P_{43}	P_{42}	P_{41}	P_{40}	P_{39}	P_{38}	P_{37}	P_{36}	P_{35}	P_{34}	P_{33}	P_{32}
4	P_{63}	P_{62}	P_{61}	P_{60}	P_{59}	P_{58}	P_{57}	P_{56}	P_{55}	P_{54}	P_{53}	P_{52}	P_{51}	P_{50}	P_{49}	P_{48}
5	P_{79}	P_{78}	P_{77}	P_{76}	P_{75}	P_{74}	P_{73}	P_{72}	P_{71}	P_{70}	P_{69}	P_{68}	P_{67}	P_{66}	P_{65}	P_{64}
6	P_{95}	P_{94}	P_{93}	P_{92}	P_{91}	P_{90}	P_{89}	P_{88}	P_{87}	P_{86}	P_{85}	P_{84}	P_{83}	P_{82}	P_{81}	P_{80}

Table 10-9: 16bits MCU, 8bpp Mode -1 (RGB 3:3:2)

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_0^7	R_0^6	R_0^5	G_0^7	G_0^6	G_0^5	B_0^7	B_0^6
2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_1^7	R_1^6	R_1^5	G_1^7	G_1^6	G_1^5	B_1^7	B_1^6
3	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_2^7	R_2^6	R_2^5	G_2^7	G_2^6	G_2^5	B_2^7	B_2^6
4	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_3^7	R_3^6	R_3^5	G_3^7	G_3^6	G_3^5	B_3^7	B_3^6
5	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_4^7	R_4^6	R_4^5	G_4^7	G_4^6	G_4^5	B_4^7	B_4^6
6	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_5^7	R_5^6	R_5^5	G_5^7	G_5^6	G_5^5	B_5^7	B_5^6

Table 10-10: 16bits MCU, 8bpp Mode -2 (RGB 3:3:2)

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	R_1^7	R_1^6	R_1^5	G_1^7	G_1^6	G_1^5	B_1^7	B_1^6	R_0^7	R_0^6	R_0^5	G_0^7	G_0^6	G_0^5	B_0^7	B_0^6
2	R_3^7	R_3^6	R_3^5	G_3^7	G_3^6	G_3^5	B_3^7	B_3^6	R_2^7	R_2^6	R_2^5	G_2^7	G_2^6	G_2^5	B_2^7	B_2^6
3	R_5^7	R_5^6	R_5^5	G_5^7	G_5^6	G_5^5	B_5^7	B_5^6	R_4^7	R_4^6	R_4^5	G_4^7	G_4^6	G_4^5	B_4^7	B_4^6
4	R_7^7	R_7^6	R_7^5	G_7^7	G_7^6	G_7^5	B_7^7	B_7^6	R_6^7	R_6^6	R_6^5	G_6^7	G_6^6	G_6^5	B_6^7	B_6^6
5	R_9^7	R_9^6	R_9^5	G_9^7	G_9^6	G_9^5	B_9^7	B_9^6	R_8^7	R_8^6	R_8^5	G_8^7	G_8^6	G_8^5	B_8^7	B_8^6
6	R_{11}^7	R_{11}^6	R_{11}^5	G_{11}^7	G_{11}^6	G_{11}^5	B_{11}^7	B_{11}^6	R_{10}^7	R_{10}^6	R_{10}^5	G_{10}^7	G_{10}^6	G_{10}^5	B_{10}^7	B_{10}^6

Note: The Mode-1 and Mode 2 is determined by bit[7:6] of Register[02h], please refer to the Cpater-13 Reregister Description.

Table 10-11: 16bits MCU, 16bpp Mode (RGB 5:6:5)

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	G_0^7	G_0^6	G_0^5	G_0^4	G_0^3	G_0^2	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3
2	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	G_1^7	G_1^6	G_1^5	G_1^4	G_1^3	G_1^2	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3
3	R_2^7	R_2^6	R_2^5	R_2^4	R_2^3	G_2^7	G_2^6	G_2^5	G_2^4	G_2^3	G_2^2	B_2^7	B_2^6	B_2^5	B_2^4	B_2^3
4	R_3^7	R_3^6	R_3^5	R_3^4	R_3^3	G_3^7	G_3^6	G_3^5	G_3^4	G_3^3	G_3^2	B_3^7	B_3^6	B_3^5	B_3^4	B_3^3
5	R_4^7	R_4^6	R_4^5	R_4^4	R_4^3	G_4^7	G_4^6	G_4^5	G_4^4	G_4^3	G_4^2	B_4^7	B_4^6	B_4^5	B_4^4	B_4^3
6	R_5^7	R_5^6	R_5^5	R_5^4	R_5^3	G_5^7	G_5^6	G_5^5	G_5^4	G_5^3	G_5^2	B_5^7	B_5^6	B_5^5	B_5^4	B_5^3

Table 10-12: 16bits MCU, 24bpp Mode1 (RGB 8:8:8)

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	G_0^7	G_0^6	G_0^5	G_0^4	G_0^3	G_0^2	G_0^1	G_0^0	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3	B_0^2	B_0^1	B_0^0
2	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3	B_1^2	B_1^1	B_1^0	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	R_0^2	R_0^1	R_0^0
3	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	R_1^2	R_1^1	R_1^0	G_1^7	G_1^6	G_1^5	G_1^4	G_1^3	G_1^2	G_1^1	G_1^0
4	G_2^7	G_2^6	G_2^5	G_2^4	G_2^3	G_2^2	G_2^1	G_2^0	B_2^7	B_2^6	B_2^5	B_2^4	B_2^3	B_2^2	B_2^1	B_2^0
5	B_3^7	B_3^6	B_3^5	B_3^4	B_3^3	B_3^2	B_3^1	B_3^0	R_2^7	R_2^6	R_2^5	R_2^4	R_2^3	R_2^2	R_2^1	R_2^0
6	R_3^7	R_3^6	R_3^5	R_3^4	R_3^3	R_3^2	R_3^1	R_3^0	G_3^7	G_3^6	G_3^5	G_3^4	G_3^3	G_3^2	G_3^1	G_3^0

Table 10-13: 16bits MCU, 24bpp Mode2 (RGB 8:8:8)

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	G_0^7	G_0^6	G_0^5	G_0^4	G_0^3	G_0^2	G_0^1	G_0^0	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3	B_0^2	B_0^1	B_0^0
2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	R_0^2	R_0^1	R_0^0
3	G_1^7	G_1^6	G_1^5	G_1^4	G_1^3	G_1^2	G_1^1	G_1^0	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3	B_1^2	B_1^1	B_1^0
4	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	R_1^2	R_1^1	R_1^0
5	G_2^7	G_2^6	G_2^5	G_2^4	G_2^3	G_2^2	G_2^1	G_2^0	B_2^7	B_2^6	B_2^5	B_2^4	B_2^3	B_2^2	B_2^1	B_2^0
6	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	R_2^7	R_2^6	R_2^5	R_2^4	R_2^3	R_2^2	R_2^1	R_2^0

10.5.2 Input Data with Opacity (αRGB)

LT7689 provides a palette of 64 colors from a total of 4096 different colors with opacity attribute for OSD (On-Screen-Display) application. User may load preferred color into embedded color palette then pick it up by index color. The α value stands for opacity.

Table 10-14: 8bits MCU, 8bpp Mode (αIndex 2:6)

Order	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	α_1^3	α_1^2	Index color of pixel 0					
2	α_3^3	α_3^2	Index color of pixel 1					
3	α_5^3	α_5^2	Index color of pixel 2					
4	α_7^3	α_7^2	Index color of pixel 3					
5	α_9^3	α_9^2	Index color of pixel 4					
6	α_{11}^3	α_{11}^2	Index color of pixel 5					

$\alpha_x^3\ \alpha_x^2$: 0→100%, 1→20/32, 2→11/32, 3→0

Table 10-15: 8bits MCU, 16bpp Mode (αRGB 4:4:4:4)

Order	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	G_0^7	G_0^6	G_0^5	G_0^4	B_0^7	B_0^6	B_0^5	B_0^4
2	α_0^3	α_0^2	α_0^1	α_0^0	R_0^7	R_0^6	R_0^5	R_0^4
3	G_1^7	G_1^6	G_1^5	G_1^4	B_1^7	B_1^6	B_1^5	B_1^4
4	α_1^3	α_1^2	α_1^1	α_1^0	R_1^7	R_1^6	R_1^5	R_1^4
5	G_2^7	G_2^6	G_2^5	G_2^4	B_2^7	B_2^6	B_2^5	B_2^4
6	α_2^3	α_2^2	α_2^1	α_2^0	R_2^7	R_2^6	R_2^5	R_2^4

$\alpha_x^3\ \alpha_x^2\ \alpha_x^1\ \alpha_x^0$: 0→100%, 1→30/32, 2→28/32, 3→26/32, 4→24/32, ……., 12→8/32, 13→6/32, 14→4/32, 15→0.

Table 10-16: 16bits MCU, Index Mode (Index 2:6)

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	α_0^3	α_0^2	Index color of pixel 0					
2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	α_1^3	α_1^2	Index color of pixel 1					
3	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	α_2^3	α_2^2	Index color of pixel 2					
4	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	α_3^3	α_3^2	Index color of pixel 3					
5	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	α_4^3	α_4^2	Index color of pixel 4					
6	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	α_5^3	α_5^2	Index color of pixel 5					

$\alpha_x^3\ \alpha_x^2$: 0→0, 1→11/32, 2→20/32, 3→100%

Table 10-17: 16bits MCU, 12bpp Mode (αRGB 4:4:4:4)

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	α_0^3	α_0^2	α_0^1	α_0^0	R_0^7	R_0^6	R_0^5	R_0^4	G_0^7	G_0^6	G_0^5	G_0^4	B_0^7	B_0^6	B_0^5	B_0^4
2	α_1^3	α_1^2	α_1^1	α_1^0	R_1^7	R_1^6	R_1^5	R_1^4	G_1^7	G_1^6	G_1^5	G_1^4	B_1^7	B_1^6	B_1^5	B_1^4
3	α_2^3	α_2^2	α_2^1	α_2^0	R_2^7	R_2^6	R_2^5	R_2^4	G_2^7	G_2^6	G_2^5	G_2^4	B_2^7	B_2^6	B_2^5	B_2^4
4	α_3^3	α_3^2	α_3^1	α_3^0	R_3^7	R_3^6	R_3^5	R_3^4	G_3^7	G_3^6	G_3^5	G_3^4	B_3^7	B_3^6	B_3^5	B_3^4
5	α_4^3	α_4^2	α_4^1	α_4^0	R_4^7	R_4^6	R_4^5	R_4^4	G_4^7	G_4^6	G_4^5	G_4^4	B_4^7	B_4^6	B_4^5	B_4^4
6	α_5^3	α_5^2	α_5^1	α_5^0	R_5^7	R_5^6	R_5^5	R_5^4	G_5^7	G_5^6	G_5^5	G_5^4	B_5^7	B_5^6	B_5^5	B_5^4

$\alpha_x^3\ \alpha_x^2\ \alpha_x^1\ \alpha_x^0$: 0→0, 1→2/32, 2→4/32, 3→6/32, 4→8/32, ……, 12→24/32, 13→26/32, 14→28/32, 15→100%.

11. Display Memory

LT7689 has an embedded 128Mb Display RAM. The internal MCU saves the displayed data to internal Display RAM through commands. LT7689's internal display engine will then continue to read the data of the memory, and then send those data to the TFT driver. The capacity of the Display RAM is also related to the resolutions and image layers supported. Please refer to the following table:

Table 11-1: LT7689' Model vs. Embedded Display RAM Capacity

Model	Display RAM Capacity	Resolution (Max.)	Color (Max.)	Image Layer (@Max Color)
LT7689	128Mb	1280*1024	16.7M	4
		1024*600	16.7M	9
		800*480	16.7M	14
		1280*1024	65K	6
		1024*600	65K	13
		800*480	65K	21

The embedded Display RAM of LT7689 is a High-Speed SDRAM (Synchronous Dynamic Random Access Memory). Before the Display RAM can be accessed, the relevant register initialization must be done based on the type of Display RAM. Please refer to the initialization steps as following:

- Setup Register REG[E0h] according to the Display RAM capacity and model. The value of REG[E0h] must be set according to the LT7689 model used to avoid display anomalies and image confusion. Please refer to Table 19-7 of Chapter 19.
- Setup Register REG[E1H], REG[E2h], REG[E3h] including the CAS latency, refresh interval etc. The typical Display RAM refresh interval is 64ms. The value of these registers are recommended according to the LT7689 model used. Please refer to Table 19-8 of Chapter 19.
- Set Register REG[E4h] Bit0 to 1, start Display RAM initialization processing.
- Read Register REG[E4h] bit0, if it becomes 1, that means initialization is complete.

11.1 Display RAM Data Structure

The image data stored in the Display RAM will be stored in different arrangement formats depending on the color (1bpp, 8bpp, 16bpp, 24bpp). Therefore, when the Host writes the image data to the Display RAM, it must follow these formats, the following tables are 8/16/24bpp RGB data arrangement format:

11.1.1 8bpp Display Data (RGB 3:3:2)

Table 11-2: 8bpp Display Data (RGB 3:3:2)

Addr	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0000h	R_1^7	R_1^6	R_1^5	G_1^7	G_1^6	G_1^5	B_1^7	B_1^6	R_0^7	R_0^6	R_0^5	G_0^7	G_0^6	G_0^5	B_0^7	B_0^6
0002h	R_3^7	R_3^6	R_3^5	G_3^7	G_3^6	G_3^5	B_3^7	B_3^6	R_2^7	R_2^6	R_2^5	G_2^7	G_2^6	G_2^5	B_2^7	B_2^6
0004h	R_5^7	R_5^6	R_5^5	G_5^7	G_5^6	G_5^5	B_5^7	B_5^6	R_4^7	R_4^6	R_4^5	G_4^7	G_4^6	G_4^5	B_4^7	B_4^6
0006h	R_7^7	R_7^6	R_7^5	G_7^7	G_7^6	G_7^5	B_7^7	B_7^6	R_6^7	R_6^6	R_6^5	G_6^7	G_6^6	G_6^5	B_6^7	B_6^6
0008h	R_9^7	R_9^6	R_9^5	G_9^7	G_9^6	G_9^5	B_9^7	B_9^6	R_8^7	R_8^6	R_8^5	G_8^7	G_8^6	G_8^5	B_8^7	B_8^6
000Ah	R_{11}^7	R_{11}^6	R_{11}^5	G_{11}^7	G_{11}^6	G_{11}^5	B_{11}^7	B_{11}^6	R_{10}^7	R_{10}^6	R_{10}^5	G_{10}^7	G_{10}^6	G_{10}^5	B_{10}^7	B_{10}^6

11.1.2 16bpp Display Data (RGB 5:6:5)

Table 11-3: 16bpp Display Data (RGB 5:6:5)

Addr	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0000h	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	G_0^7	G_0^6	G_0^5	G_0^4	G_0^3	G_0^2	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3
0002h	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	G_1^7	G_1^6	G_1^5	G_1^4	G_1^3	G_1^2	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3
0004h	R_2^7	R_2^6	R_2^5	R_2^4	R_2^3	G_2^7	G_2^6	G_2^5	G_2^4	G_2^3	G_2^2	B_2^7	B_2^6	B_2^5	B_2^4	B_2^3
0006h	R_3^7	R_3^6	R_3^5	R_3^4	R_3^3	G_3^7	G_3^6	G_3^5	G_3^4	G_3^3	G_3^2	B_3^7	B_3^6	B_3^5	B_3^4	B_3^3
0008h	R_4^7	R_4^6	R_4^5	R_4^4	R_4^3	G_4^7	G_4^6	G_4^5	G_4^4	G_4^3	G_4^2	B_4^7	B_4^6	B_4^5	B_4^4	B_4^3
000Ah	R_5^7	R_5^6	R_5^5	R_5^4	R_5^3	G_5^7	G_5^6	G_5^5	G_5^4	G_5^3	G_5^2	B_5^7	B_5^6	B_5^5	B_5^4	B_5^3

11.1.3 24bpp Display Data (RGB 8:8:8)

Table 11-4: 24bpp Display Data (RGB 8:8:8)

Addr	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0000h	G_0^7	G_0^6	G_0^5	G_0^4	G_0^3	G_0^2	G_0^1	G_0^0	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3	B_0^2	B_0^1	B_0^0
0002h	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3	B_1^2	B_1^1	B_1^0	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	R_0^2	R_0^1	R_0^0
0004h	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	R_1^2	R_1^1	R_1^0	G_1^7	G_1^6	G_1^5	G_1^4	G_1^3	G_1^2	G_1^1	G_1^0
0006h	G_2^7	G_2^6	G_2^5	G_2^4	G_2^3	G_2^2	G_2^1	G_2^0	B_2^7	B_2^6	B_2^5	B_2^4	B_2^3	B_2^2	B_2^1	B_2^0
0008h	B_3^7	B_3^6	B_3^5	B_3^4	B_3^3	B_3^2	B_3^1	B_3^0	R_2^7	R_2^6	R_2^5	R_2^4	R_2^3	R_2^2	R_2^1	R_2^0
000Ah	R_3^7	R_3^6	R_3^5	R_3^4	R_3^3	R_3^2	R_3^1	R_3^0	G_3^7	G_3^6	G_3^5	G_3^4	G_3^3	G_3^2	G_3^1	G_3^0

11.1.4 Index Display with Opacity (αRGB 2:2:2:2)

Table 11-5: Index Display with Opacity (αRGB 2:2:2:2)

Addr	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0000h	α_1^3	α_1^2	Index color of pixel 1						α_0^3	α_0^2	Index color of pixel 0					
0002h	α_3^3	α_3^2	Index color of pixel 3						α_2^3	α_2^2	Index color of pixel 2					
0004h	α_5^3	α_5^2	Index color of pixel 5						α_4^3	α_4^2	Index color of pixel 4					
0006h	α_7^3	α_7^2	Index color of pixel 7						α_6^3	α_6^2	Index color of pixel 6					
0008h	α_9^3	α_9^2	Index color of pixel 9						α_8^3	α_8^2	Index color of pixel 8					
000Ah	α_{11}^3	α_{11}^2	Index color of pixel 11						α_{10}^3	α_{10}^2	Index color of pixel 10					

α_x^3 α_x^2 : 0→0, 1→11/32, 2→20/32, 3→100%

11.1.5 12bpp Display with Opacity (αRGB 4:4:4:4)

Table 11-6: 12bpp Display with Opacity (αRGB 4:4:4:4)

Addr	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0000h	α_0^3	α_0^2	α_0^1	α_0^0	R_0^7	R_0^6	R_0^5	R_0^4	G_0^7	G_0^6	G_0^5	G_0^4	B_0^7	B_0^6	B_0^5	B_0^4
0002h	α_1^3	α_1^2	α_1^1	α_1^0	R_1^7	R_1^6	R_1^5	R_1^4	G_1^7	G_1^6	G_1^5	G_1^4	B_1^7	B_1^6	B_1^5	B_1^4
0004h	α_2^3	α_2^2	α_2^1	α_2^0	R_2^7	R_2^6	R_2^5	R_2^4	G_2^7	G_2^6	G_2^5	G_2^4	B_2^7	B_2^6	B_2^5	B_2^4
0006h	α_3^3	α_3^2	α_3^1	α_3^0	R_3^7	R_3^6	R_3^5	R_3^4	G_3^7	G_3^6	G_3^5	G_3^4	B_3^7	B_3^6	B_3^5	B_3^4
0008h	α_4^3	α_4^2	α_4^1	α_4^0	R_4^7	R_4^6	R_4^5	R_4^4	G_4^7	G_4^6	G_4^5	G_4^4	B_4^7	B_4^6	B_4^5	B_4^4
000Ah	α_5^3	α_5^2	α_5^1	α_5^0	R_5^7	R_5^6	R_5^5	R_5^4	G_5^7	G_5^6	G_5^5	G_5^4	B_5^7	B_5^6	B_5^5	B_5^4

α_x^3 α_x^2 α_x^1 α_x^0 : 0→0, 1→2/32, 2→4/32, 3→6/32, 4→8/32, ……, 12→24/32, 13→26/32, 14→28/32, 15→100%.

.

11.2 Color Palette RAM

Table 11-7: Color Palette RAM

Addr	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0000h	R_0^7	R_0^6	R_0^5	R_0^4	G_0^7	G_0^6	G_0^5	G_0^4	B_0^7	B_0^6	B_0^5	B_0^4
0002h	R_1^7	R_1^6	R_1^5	R_1^4	G_1^7	G_1^6	G_1^5	G_1^4	B_1^7	B_1^6	B_1^5	B_1^4
0004h	R_2^7	R_2^6	R_2^5	R_2^4	G_2^7	G_2^6	G_2^5	G_2^4	B_2^7	B_2^6	B_2^5	B_2^4
0006h	R_3^7	R_3^6	R_3^5	R_3^4	G_3^7	G_3^6	G_3^5	G_3^4	B_3^7	B_3^6	B_3^5	B_3^4
0008h	R_4^7	R_4^6	R_4^5	R_4^4	G_4^7	G_4^6	G_4^5	G_4^4	B_4^7	B_4^6	B_4^5	B_4^4
000Ah	R_5^7	R_5^6	R_5^5	R_5^4	G_5^7	G_5^6	G_5^5	G_5^4	B_5^7	B_5^6	B_5^5	B_5^4

12. LCD Interface

LT7689 Supports 16, 18, and 24bits RGB interface of TFT Panel. No matter it is 24bpp (RGB 8:8:8), 16bpp (RGB 5:6:5) or 8bpp (RGB 3:3:2), the display data can be sent to the TFT Driver on the TFT panel through these RGB interfaces. The below Table12-1 shows how LT7689 LCD Display data corresponds to the RGB data. The MCU can setup REG[01h] Bit[4:3] to select 16bits, 18bits, or 24bits resolution.

Table 12-1: RGB Interface VS. RGB Display Data

Pin Name	TFT-LCD Interface		
	16bits	18bits	24bits
PD[0]	GPIOD[0]		B0
PD[1]	GPIOD[1]		B1
PD[2]	GPIOD[6]	B0	B2
PD[3]	B0	B1	B3
PD[4]	B1	B2	B4
PD[5]	B2	B3	B5
PD[6]	B3	B4	B6
PD[7]	B4	B5	B7
PD[8]	GPIOD[2]		G0
PD[9]	GPIOD[3]		G1
PD[10]	G0	G0	G2
PD[11]	G1	G1	G3
PD[12]	G2	G2	G4
PD[13]	G3	G3	G5
PD[14]	G4	G4	G6
PD[15]	G5	G5	G7
PD[16]	GPIOD[4]		R0
PD[17]	GPIOD[5]		R1
PD[18]	GPIOD[7]	R0	R2
PD[19]	R0	R1	R3
PD[20]	R1	R2	R4
PD[21]	R2	R3	R5
PD[22]	R3	R4	R6
PD[23]	R4	R5	R7

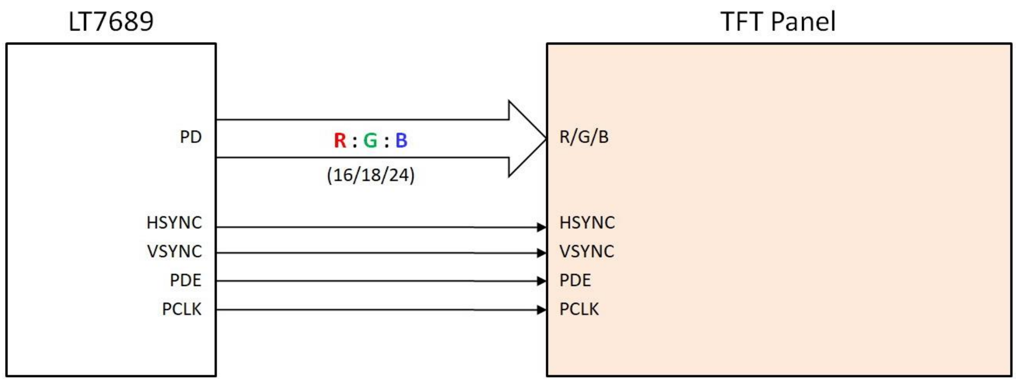


Figure 12-1: TFT-LCD Interface

Figure 12-1 is the interface diagram of LT7689 and TFT panel. Figure 12-2 is the timing diagram of LT7689 TFT-LCD output signals. In addition to the RGB data lines mentioned above, LT7689 also provides PCLK (Panel Scan Clock), VSYNC Pulse, HSYNC Pulse and Data Enable signal. The frequency of PCLK is setup by REG[05h] and REG[06h]. Please refer to the description in Chapter 19th.

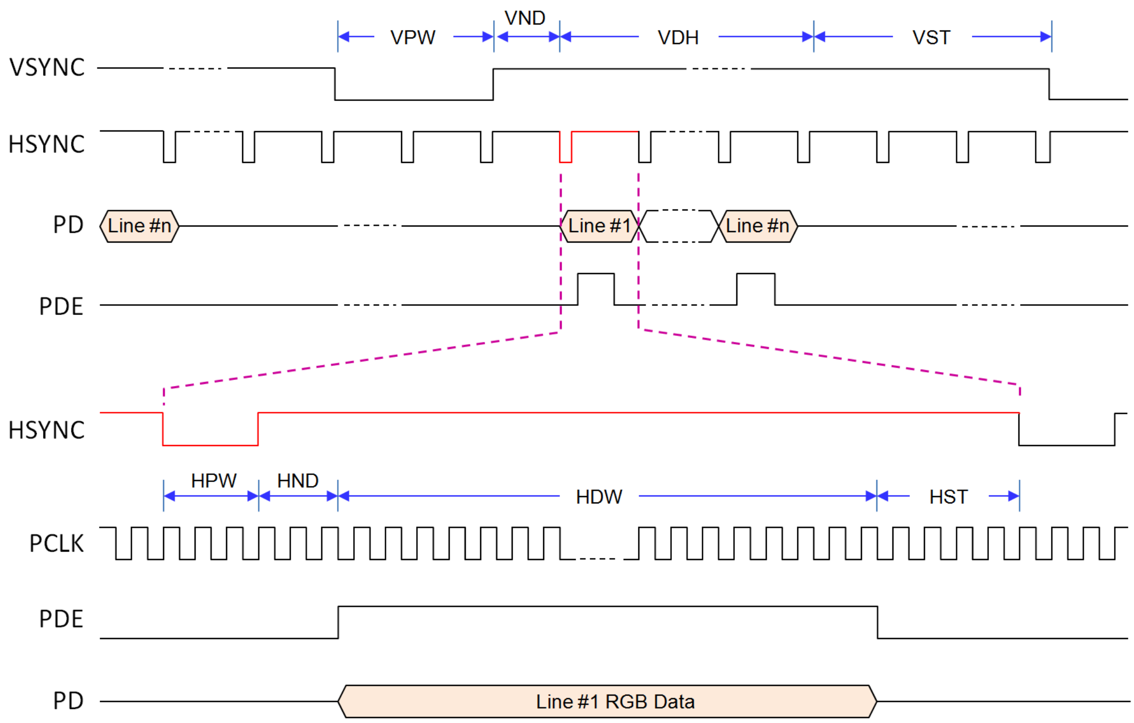


Figure 12-2: TFT-LCD Interface Timing

13. Display Function

13.1 Color Bar

LT7689 provides a color bar display, which can be used as a display test and does not require display memory. The function can be performed by setting REG[12h] bit5 to 1.

Figure 13-1: Color Bar

13.2 Main Window

The LCD main window size can be defined by setting Registers REG[14h] ~ REG[1Fh]. Users can save images to memory buffers first. By setting up the related Registers (REG[20h] ~ REG[29h]), users can then show the specific images they want.

13.2.1 Configure Display Image Buffer

Display RAM is used to store images, and how many images can be stored is determined by the image resolution and the color depth. For example, if the image resolution is 800*480, with 65K color (16bits), then 21 images can be stored in 128Mbits Display RAM:

Number of Image = (128*1024*1024) / 800*480*16) = 21.8

If the image resolution is 1024*600, with 65K color (16bits), then 13 images can be stored in 128Mbits Display RAM:

Number of Image = (128*1024*1024) / 1024*600*16) = 13.6

In LT7689, the Starting Position and Width of the Canvas, and the Working window range must be set before writing the image data to Display RAM. Please refer the registers REG[50H] to REG[5Eh] for detail description.

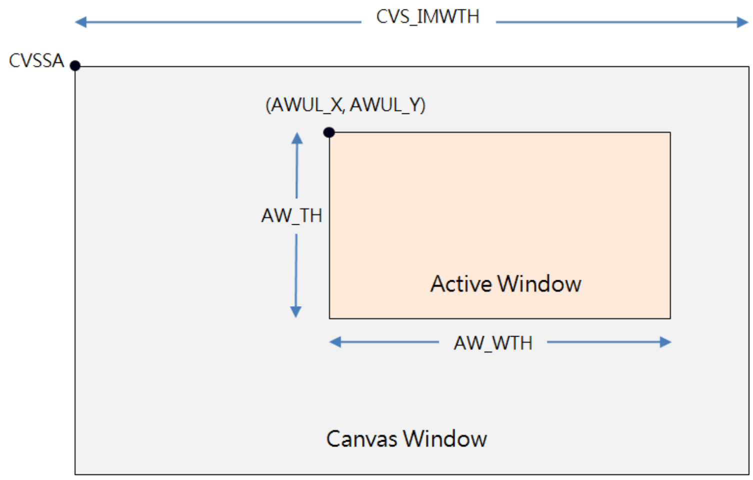


Figure 13-2: Canvas Window and Active Window

13.2.2 Write Image Data to Display Image Buffer

The following diagram is a flowchart about writing image data to Display RAM and displaying the main window image on an LCD screen:

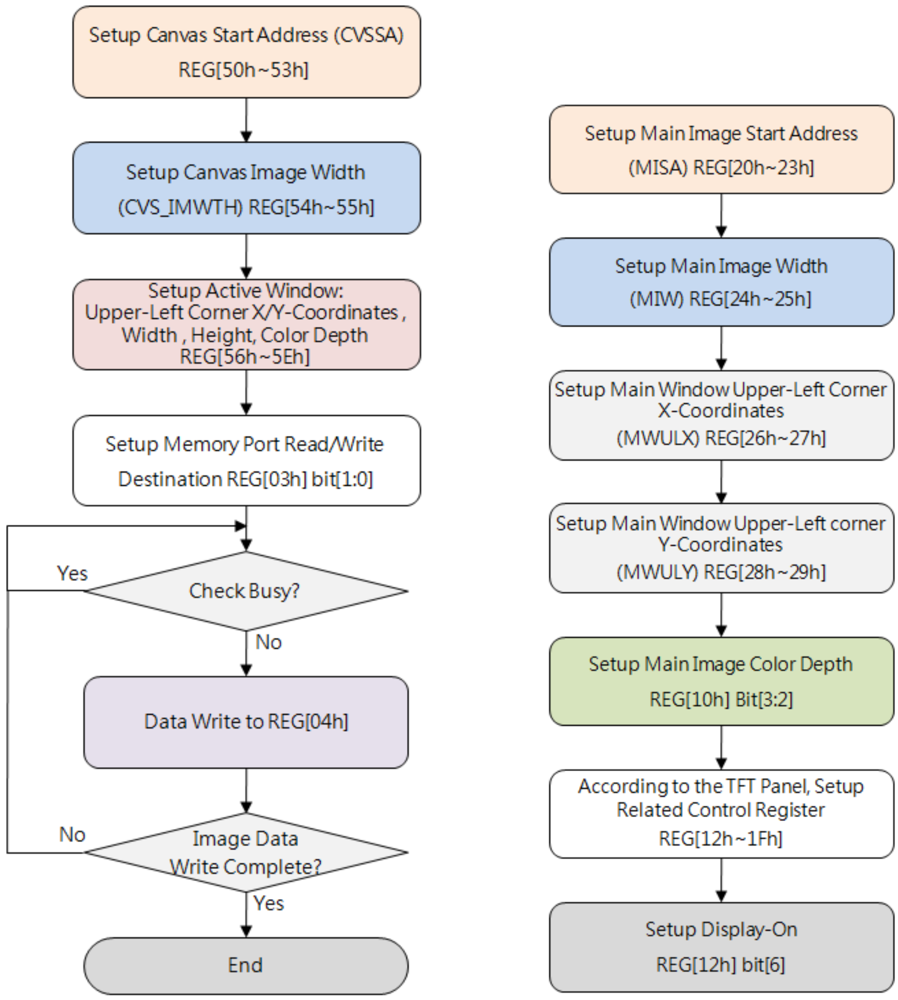


Figure 13-3: Write Image Data to Display Image Buffer

13.2.3 Display Main Window Image

Main window displays the selected image by setting the Registers REG[20h] ~ REG[29h]. The following figure is the process for setting up the main window:

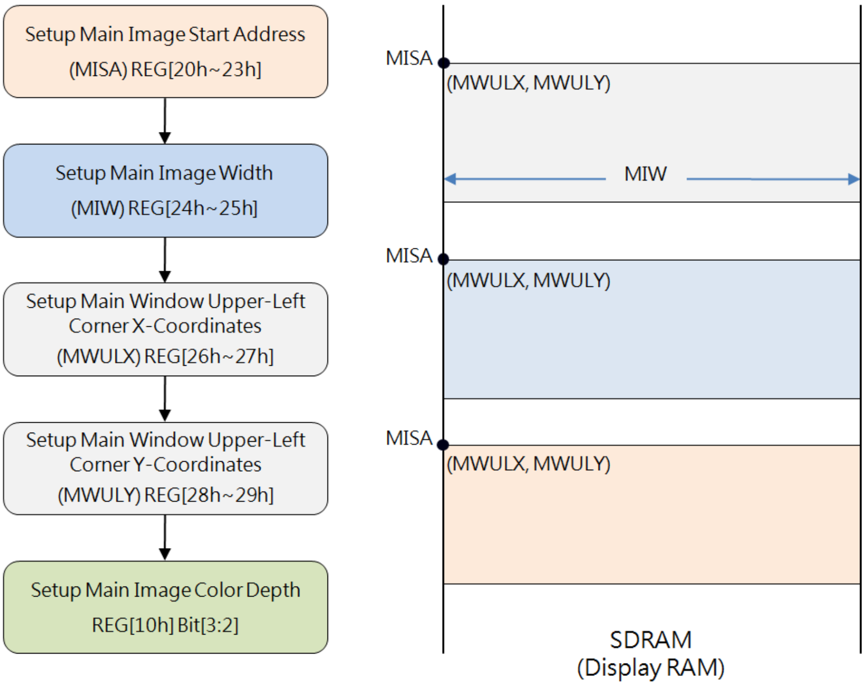


Figure 13-4: Setup and Select Main Window Image

13.3 Picture-In-Picture (PIP)

LT7689 supports Picture-in-Picture feature. User can display secondary image(PIP-1) on the main screen(PIP-2) without overwriting the image data of the main display window. If both images are overlapping, then the PIP-1 image is always on the top of PIP-2.

The size and location of the picture window is set by the register REG[2Ah] ~ REG[3BH] and REG[11h]. The parameters of PIP-1 and PIP-2 are using the same registers, while REG[10H] Bit4 is used to decide whether the parameters are for PIP-1 or PIP-2. In addition, before using the PIP function, the relevant parameters of the display window must be set.

The unit of PIP windows sizes and start positions is 4 pixels in horizontal, and 1 pixel in vertical. In PIP mode, LT7689 does not support the overlapping of transparent display, and when REG[12h] Bit3 VDIR = 1, then PIP windows, graphic cursors, and text cursors will be automatically prohibited.

13.3.1 PIP Window Setting

For using PIP feature, Host has to setup the PIP Image Start Address, Image Width, Display X/Y coordinates, Image X/Y coordinates, PIP Windows Color Depth, PIP Window Width and PIP Window Height registers. The following figure is the flowchart for setting the PIP, and showing the corresponding display to the main window.

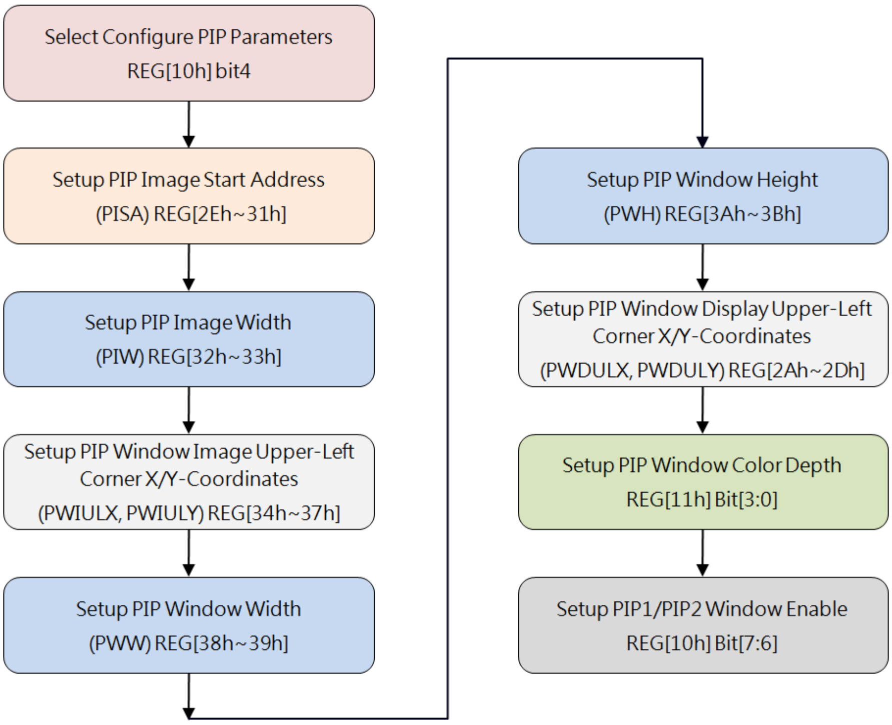


Figure 13-5: Initialize PIP Function

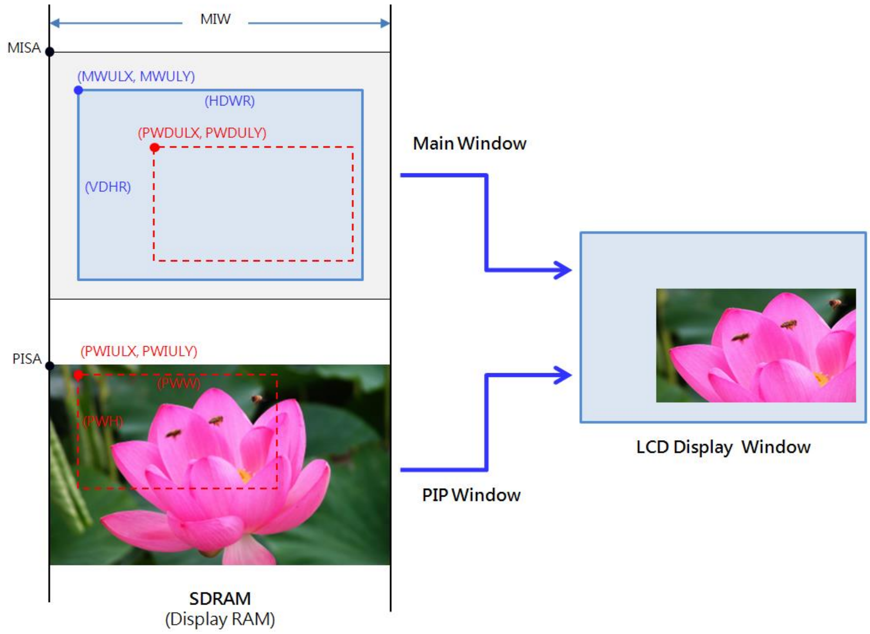


Figure 13-6: PIP Example

13.3.2 PIP Display Position and PIP Image Position

As described in the previous section of the PIP display flowchart, it is known that the final position of the PIP on the LCD screen can be changed by setting up PWDULX and PWDULY. Setting PISA, PIW, PWIULX, PWIULY can change the position of the PIP images that you want to display. These actions do not change any image data that exists in Display RAM, but can easily change the displayed picture on the panel. The following example shows a main window with a PIP window. You can show the different PIP image by changing the PIP Position Register(PWIULX and PWIULY) of the display window.

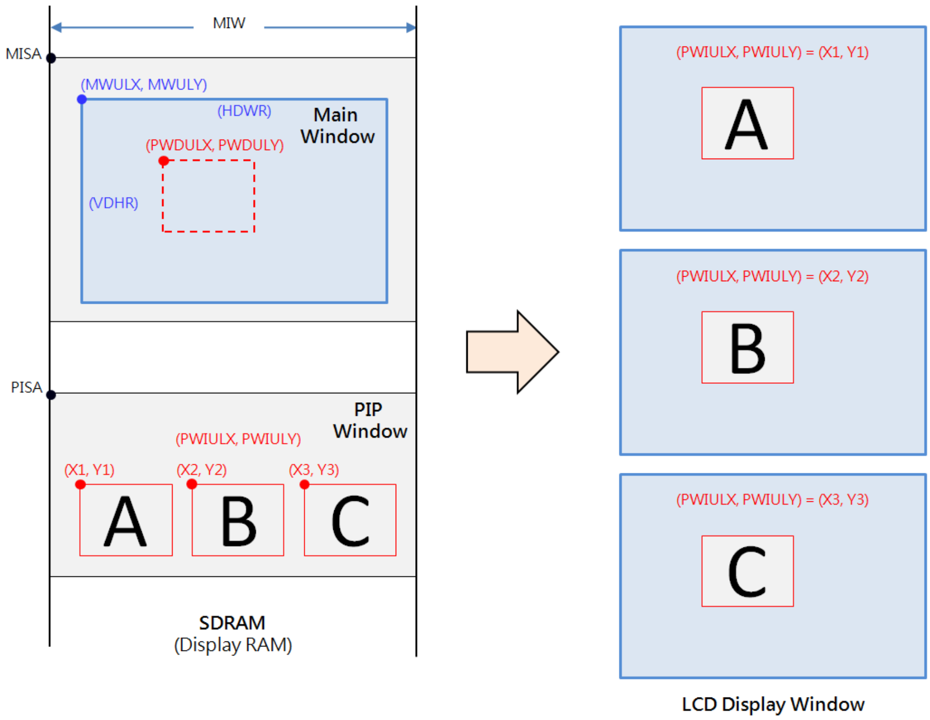


Figure 13-7: Select Different PIP image on the Screen

13.4 Image Rotate and Mirror

Usually LCD displays are refreshed horizontally (from left to right and from top to bottom), and the displayed images are stored in the same way. LT7689 provides Rotate and Mirror functions. The rotate feature rotates the displayed image in the counterclockwise direction of 90° or 180°. The mirror feature is the image that is displayed from right to left. LT7689 has embedded a hardware controller to execute Rotation and Mirroring features, therefore it can greatly save Host processing time.

The REG[02H] bit[2:1] are used to control the Memory Store Direction. These two bits are available only for Graphic Mode.

00b: Left → Right, then Top → Bottom (Original)

01b: Right → Left, then Top → Bottom (Horizontal flip)

10b: Top → Bottom, then Left → Right (Rotate right 90°& Horizontal flip)

11b: Bottom → Top, then Left → Right (Rotate left 90°)

The followings are some examples for Image Rotate and Mirror:

Figure 13-8: Original Image – without Rotation

1. **When VDIR (REG[12h] bit3)= 0**

   When REG[02H] bit[2:1] is set to 00b, its definition is to write image data from left to right and then top to bottom. This will show the original image as Figure 13-8. If REG[02H] bit[2:1] is set to 01b, it means writing image data from right to left and then from top to bottom. So the image displayed will be a horizontal mirror image as following:

Figure 13-9: Horizontal Mirror Image

When REG[02H] bit[2:1] is set to 10b, it means writing image data from top to bottom and then left to right. So the displayed image will be rotated to the right 90° and then flipped horizontally as following Figure:

Figure 13-10: Rotated to the Right 90° and Flipped Horizontally

When REG[02H] bit[2:1] is set to 11b, the image is written from bottom to top and then left to right. So the displayed image will rotate 90° to the left as following Figure:

Figure 13-11: Rotate 90° to the Left

2. When VDIR (REG[12h] bit3) = 1,

When REG[02h] bit[2:1] is set to 00b, the displayed image will be as following Figure:

Figure 13-12: Flip with Vertical

When REG[02h] bit[2:1] is set to 01b, the displayed image will be rotated 180°:

Figure 13-13: Rotated 180°

When REG[02h] bit[2:1] is set to 10b, The displayed image will rotate 90° to the left:

Figure 13-14: Rotate 90° to the Left

When REG[02h] bit[2:1] is set to 11b, the displayed image will be as following Figure:

Figure 13-15: Rotated to the Left 90° then Vertically Mirrored

14. Geometric Drawing Engine

14.1 Drawing Circles and Ellipses

LT7689 supports the function of drawing Circles and Ellipses. As long as the Host sets the Circle or the Ellipse Center (REG[7Bh ~ 7Eh]), Radius (REG[77h ~ 7Ah]), and the Color of the Circle (REG[D2h ~ D4h]), then specify REG[76h] bit[5:4] to 00b,. finally enable the Drawing Circle function (REG[76h] bit7 = 1), then LT7689 will draw a Circle or Ellipse on the screen automatically.

Host can also set REG[76h] Bit6 to 1 to fill a Circle or an Ellipse. Therefore, Host does not need to consume many resources to calculate the location or to write series of data to the display memory. The Ellipse has two radius. To draw a Circle, just set the long radius and short radius register to the same value (R1 = R2).

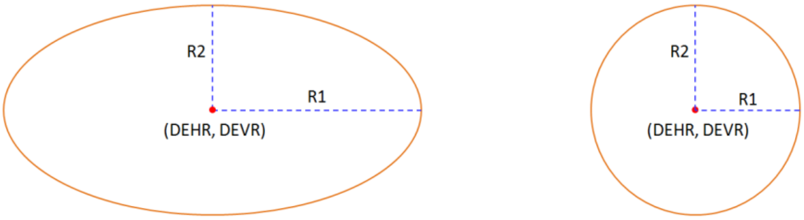


Figure 14-1: Drawing Ellipse and Circle

The flowchart for drawing circles and drawing ellipses is as following. The left flowchart is to draw a circle or an ellipse without fill, and the right flowchart is with fill. Please note the center point of the Circle must be located in the Active Window.

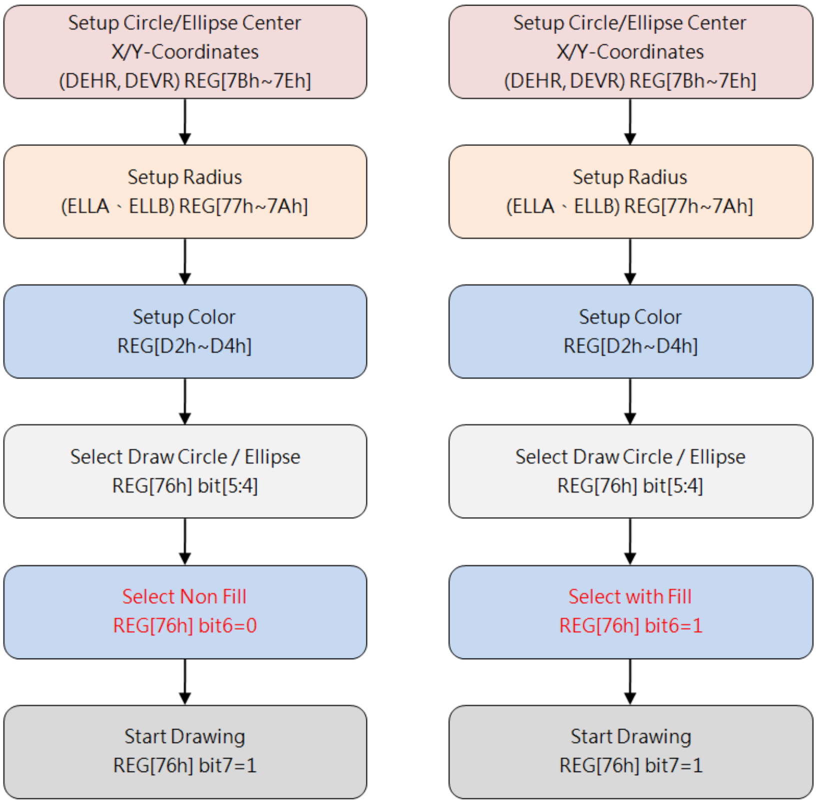


Figure 14-2: Flowchart of Drawing Circle and Ellipse

14.2 Drawing Curves

LT7689 supports the function of drawing Curves. As long as the Host sets the Curve Center (REG[7Bh ~ 7Eh]), Radius (REG[77h ~ 7Ah]), and the Color of the Curve (REG[D2h ~ D4h]), then specify REG[76h] bit[5:4] to 01b, finally enable the Drawing Curve function (REG[76h] bit7 = 1), then LT7689 will draw a one-fourth Curve on the screen automatically.

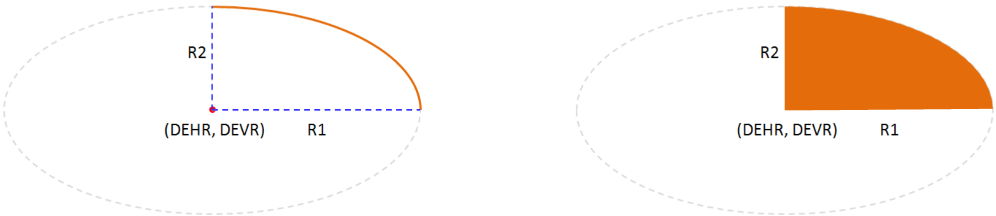


Figure 14-3: Drawing Curve and one-fourth Ellipse

The flowchart for drawing a Curve or an one-fourth Ellipse is as following. The left flowchart is to draw a Curve or an one-fourth Ellipse without fill, and the right flowchart is with fill. Please note the center point of the Curve must be located in the Active Window.

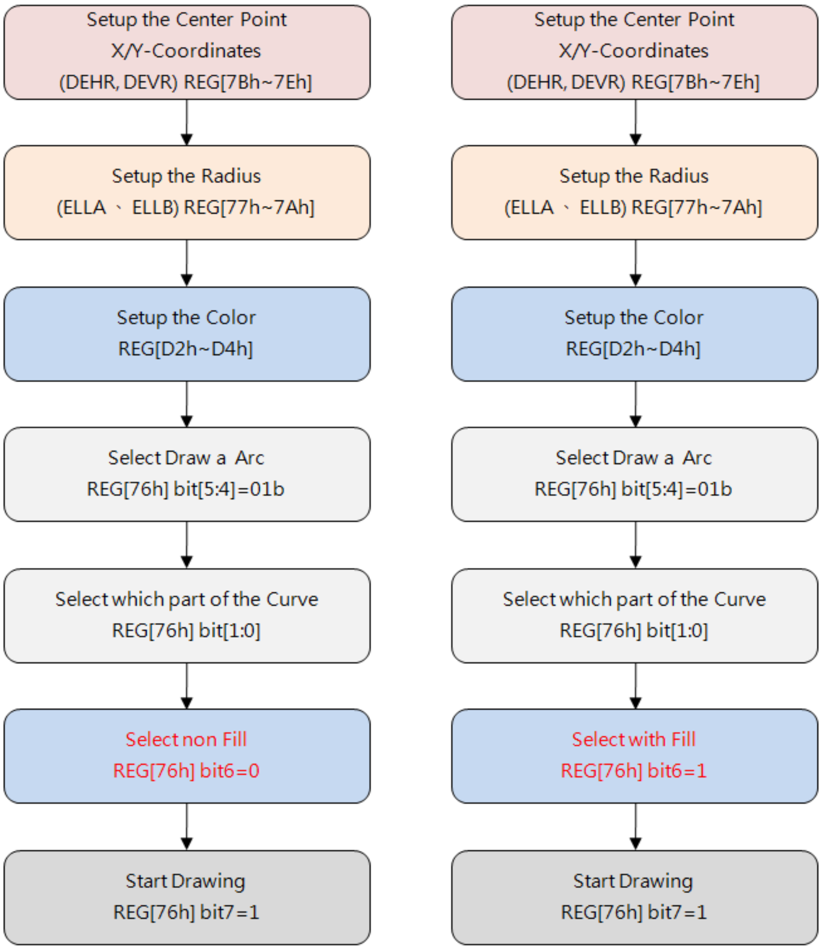


Figure 14-4: Flowchart of Drawing Curve and one-fourth Ellipse

14.3 Drawing Rectangles

To draw a rectangle, the Host has to set the Start Point Coordinates (REG[68h ~ 6Bh]), Stop Point Coordinates (REG[6Ch ~ 6Fh]), and the Color of the Rectangle (REG[D2h ~ D4h]), then specify REG[76h] bit[5:4] to 10b, finally enable the drawing function (REG[76h] bit7 = 1), then LT7689 will draw a Rectangle on the screen automatically. Host can also set REG[76h] Bit6 to 1 to fill the Rectangle with specified color.

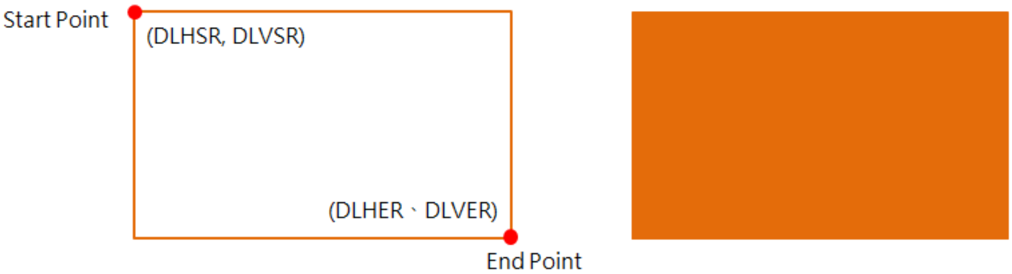


Figure 14-5: Drawing Rectangle

The following figure is the flowchart of drawing rectangles. The left flowchart is to draw a Rectangle without fill, and the right flowchart is with fill. Please note the Start and Stop point must be located in the Active Window.

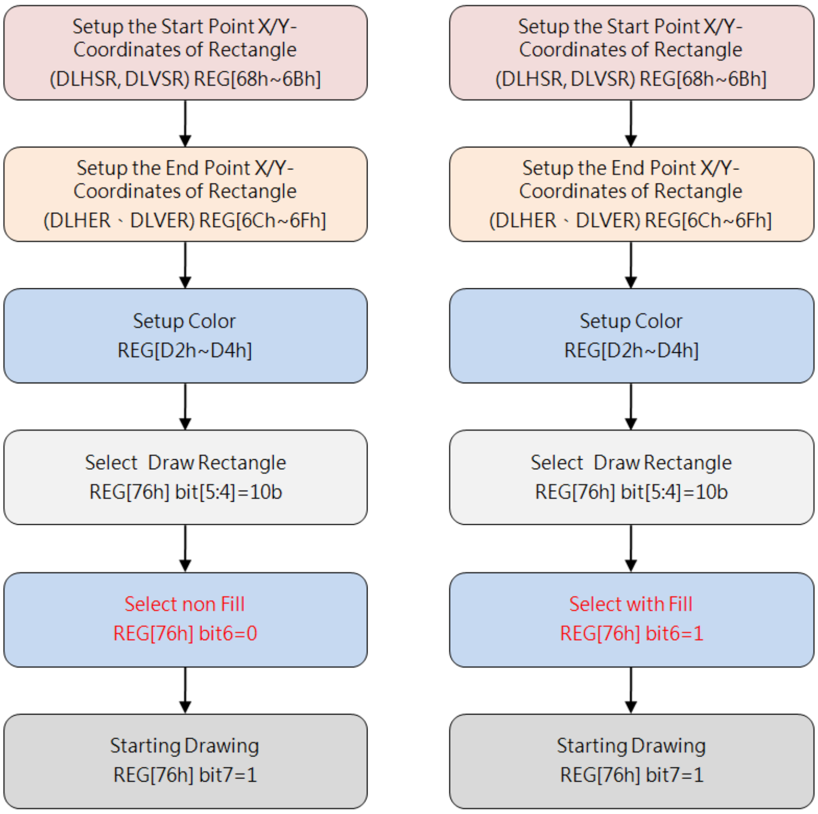


Figure 14-6: Flowchart of Drawing a Rectangle

14.4 Draw Lines

To draw a Line, the Host has to set the Start Point Coordinates (REG[68h ~ 6Bh]), Stop Point Coordinates (REG[6Ch ~ 6Fh]), and the Color of the Line (REG[D2h ~ D4h]), then specify and specify REG[67h] bit1 to 0, finally enable the Line Drawing function (REG[67h] bit7 = 1), then LT7689 will draw a Line on the screen automatically.

The following figure is the flowchart of drawing lines. Please note the Start and Stop point must be located in the Active Window.

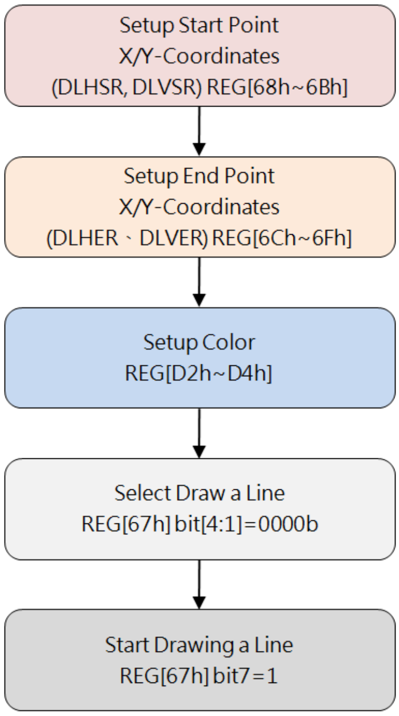


Figure 14-7: Flowchart of Drawing a Line

14.5 Drawing Triangles

To draw a triangle, the Host has to sets the 1ST Point Coordinate (REG[68h ~ 6Bh]) of Triangle, 2nd Point Coordinate (REG[6Ch ~ 6Fh]), 3rd Point Coordinate (REG[70h ~ 73h]), and the Color of the Triangle (REG[D2h ~ D4h]), then specify REG[67h] bit1 to 1, finally enable the drawing function (REG[67h] bit7 = 1), then LT7689 will draw a Triangle on the screen automatically. Host can also set REG[67h] bit5 to 1 to fill the Rectangle with specified color.

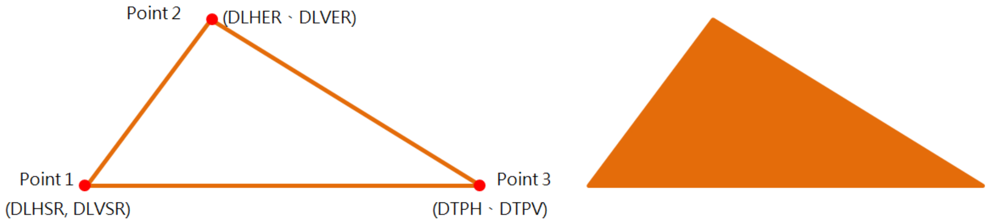


Figure 14-8: Drawing Triangle

The following figure is the flowchart of drawing triangles. The left flowchart is to draw a triangle without fill, and the right flowchart is with fill. Please note the three points must be located in the Active Window.

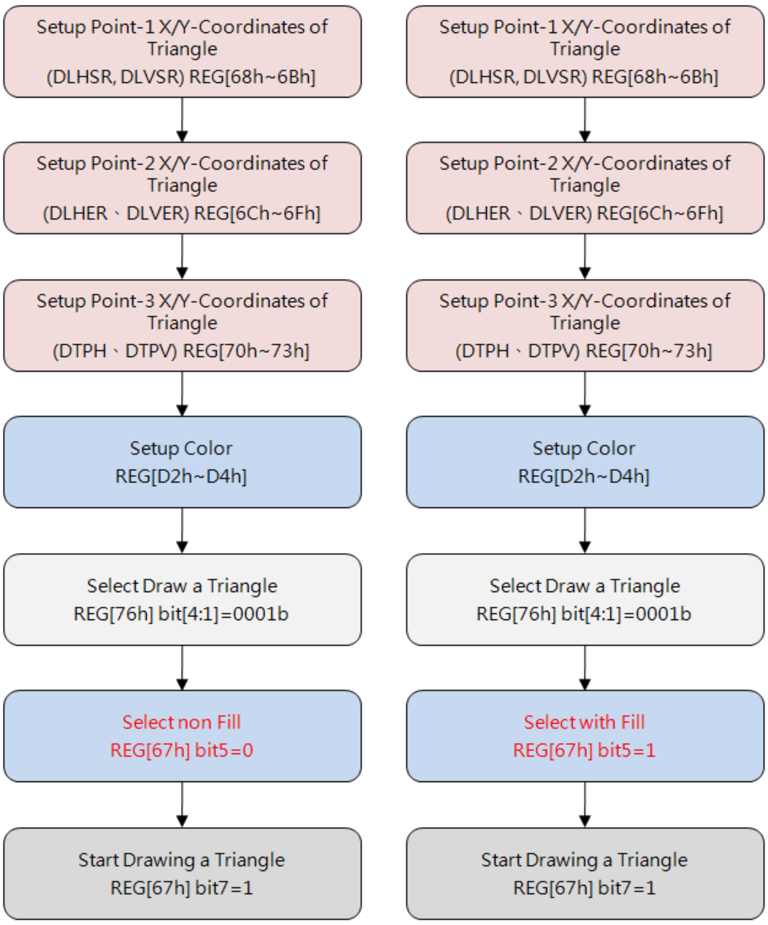


Figure 14-9: Flowchart of Drawing a Triangle

14.6 Drawing Rounded-Rectangles

To draw a rounded-rectangle, the Host has to set the Start Point Coordinates (REG[68h ~ 6Bh]), Stop Point Coordinates (REG[6Ch ~ 6Fh]), the Rounded Radius (REG[77h ~ 7Ah]) and the Color (REG[D2h ~ D4h]), then specify REG[76h] bit[5:4] to 11b, finally enable the drawing function (REG[76h] bit7 = 1), then LT7689 will draw a Rounded-Rectangle on the screen automatically. Host can also set REG[76h] Bit6 to 1 to fill the Rounded-Rectangle with specified color. The following figure is a schematic of a Rounded-Rectangle, in which the R1 represents the long axis radius, and the R2 represents the short axis radius.

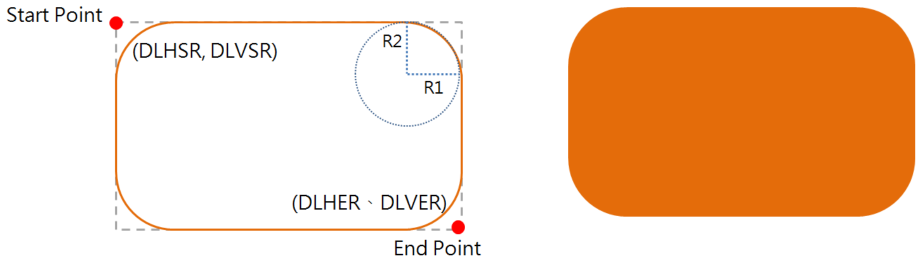


Figure 14-10: Drawing Rounded Triangle

Note1: DLHER-DLHSR must be larger than 2*R1 +1

Note2: DLVER-DLVSR must be larger than 2*R2 +1

The following figure is the flowchart of drawing Rounded-Rectangle. The left flowchart is to draw a Triangle without fill, and the right flowchart is with fill. Please note the Start and Stop point must be located in the Active Window.

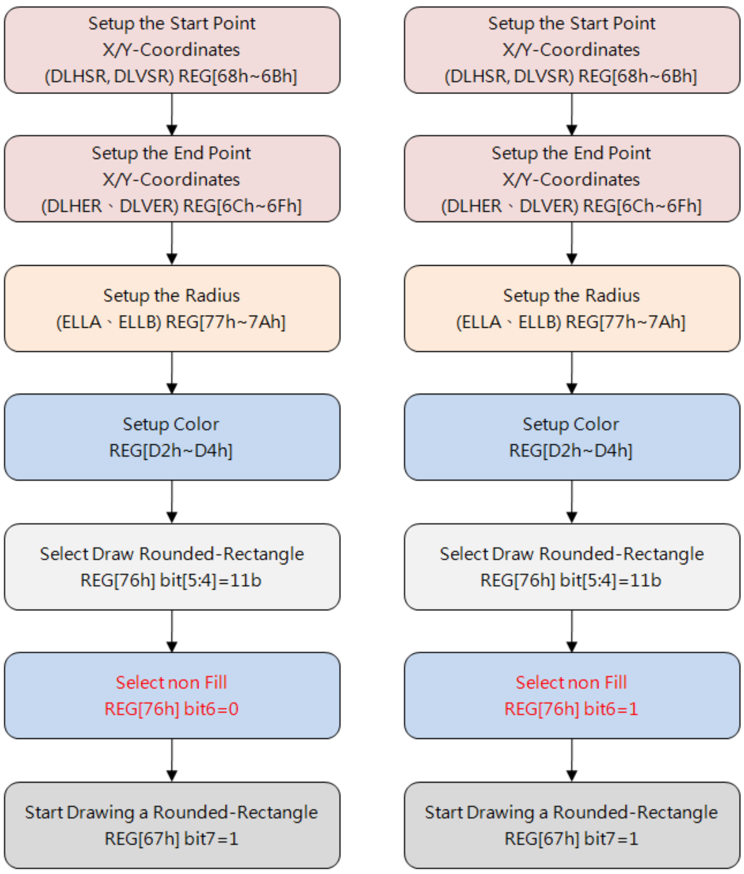


Figure 14-11: Flowchart of Drawing a Rounded-Rectangle

15. Block Transfer Engine (BTE)

LT7689 has an embedded 2D Block Transfer Engine, it can ehance the efficiency of the block transferring. In the transferring operation of designated block data combining with logic operation, BTE can accelerate data transferring, and simplify MCU programing. This chapter is mainly to explain the functions and operation modes of BTE.

Befroe using BTE, users must assign the operation mode of BTE. Please refer to Table15-1 for the details of the operation modes. Also, LT7689 supports 16 Raster Operations (ROP, Table15-2) that allow users to combine source data (source 0 and source 1) into desitnation data in various ways.

There are two ways to get BTE working status: (1) check Register BTE_CTRL0(REG[90h]) Bit4 or Status Register (STSR) Bit3; (2) check hardware interrupt. When hardware interrupt is triggered, the interruption source can be confirmed by checking REG[0Ch].

Table 15-1: BTE Operation Modes

BTE Operation Code REG[91h] Bits [3:0]	BTE Operation Description
0000b	MCU Write with ROP.
0010b	Memory Copy (move) with ROP.
0100b	MCU Write with chroma keying (without ROP)
0101b	Memory Copy (move) with chroma keying (without ROP)
0110b	Pattern Fill with ROP
0111b	Pattern Fill with chroma keying (without ROP)
1000b	MCU Write with Color Expansion (without ROP)
1001b	MCU Write with Color Expansion and chroma keying (without ROP)
1010b	Memory Copy with opacity (without ROP)
1011b	MCU Write with opacity (without ROP)
1100b	Solid Fill (without ROP)
1110b	Memory Copy with Color Expansion (without ROP)
1111b	Memory Copy with Color Expansion and chroma keying (without ROP)
Other Combinations	Reserved

Table 15-2: ROP Function

ROP Function Code REG[91h] bit[7:4]	Function Description (Boolean)
0000b	0 (Blackness)
0001b	~S0 · ~S1 or ~ (S0+S1)
0010b	~S0 · S1
0011b	~S0
0100b	S0 · ~S1
0101b	~S1
0110b	S0^S1
0111b	~S0+~S1 or ~ (S0 · S1)
1000b	S0 · S1
1001b	~ (S0^S1)
1010b	S1
1011b	~S0+S1
1100b	S0
1101b	S0+~S1
1110b	S0+S1
1111b	1 (Whiteness)

Note:

1. Source 0 (S0) Data: comes from MCU Writing or Memory
2. Source 1 (S1) Data: comes from Memory,
3. Destination (DT) Data: be written to Memory
4. Memory means internal Display RAM
5. Memory, S0, S1, DT, these Short Names will be always quoted in this Chapter. For Example: If ROP function REG[91h] bit[7:4]=0xCh, then “DT = S0”, means “Transfer Source 0 data to Destination”; If ROP function REG[91h] bit[7:4]=0xEh, then “DT = S0 + S1”, means “Source 0 data + Source 1 data then transfer to Destination”

Table 15-3: Color Expansion Function

ROP Function REG[91h] bit[7:4]	Start Bit Position for Color Expansion BTE operation code = 1000/1001/1110/1111	
	16bits MCU Interface	8bits MCU Interface
0000b	Bit0	Bit0
0001b	Bit1	Bit1
0010b	Bit2	Bit2
0011b	Bit3	Bit3
0100b	Bit4	Bit4
0101b	Bit5	Bit5
0110b	Bit6	Bit6
0111b	Bit7	Bit7
1000b	Bit8	Invalid
1001b	Bit9	Invalid
1010b	Bit10	Invalid
1011b	Bit11	Invalid
1100b	Bit12	Invalid
1101b	Bit13	Invalid
1110b	Bit14	Invalid
1111b	Bit15	Invalid

15.1 BTE Basic Settings

The BTE basic settings, including Memory Start Address, Image Width, and Start Point Position (Coordinate) for each Source and Destination, can be defined by following registers:

1. S0 Address Registers: REG [93h], REG[94h], REG[95h], REG[96h], REG[97h], REG [98h], REG[99h], REG[9Ah], REG[9Bh], REG[9Ch]
2. S1 Address Registers: REG [9Dh], REG[9Eh], REG[9Fh], REG[A0h], REG [A1h], REG[A2h], REG[A3h], REG[A4h], REG[A5h], REG[A6h]
3. DT Address Registers: REG [A7h], REG[A8h], REG[A9h], REG[AAh], REG [ABh], REG[ACh], REG[ADh], REG[AEh], REG[AFh], REG[B0h]

15.2 Color Palette RAM

LT7689 has an embedded Color Palette RAM for 8-bits Alpha Blend function. Real colors can be retrieved by searching Color Palette RAM. As shown in Figure 15-1, Color Palette RAM is organized by 64*12bits. Since MCU writes 8 bits data at a time, when it writes every even byte, only the lower 4 bits will be stored to the RAM. When using Color Palette RAM, the Register REG[03h] bit [1:0] has to be set to 11b. Color Palette RAM must be written by 128 Bytes (8-bits per Byte) sequence at a time, and it is not readable. The diagram Figure 15-2 shows the initialization flow.

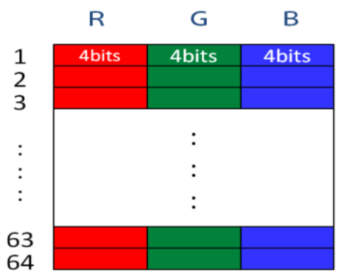


Figure 15-1: Color Palette RAM

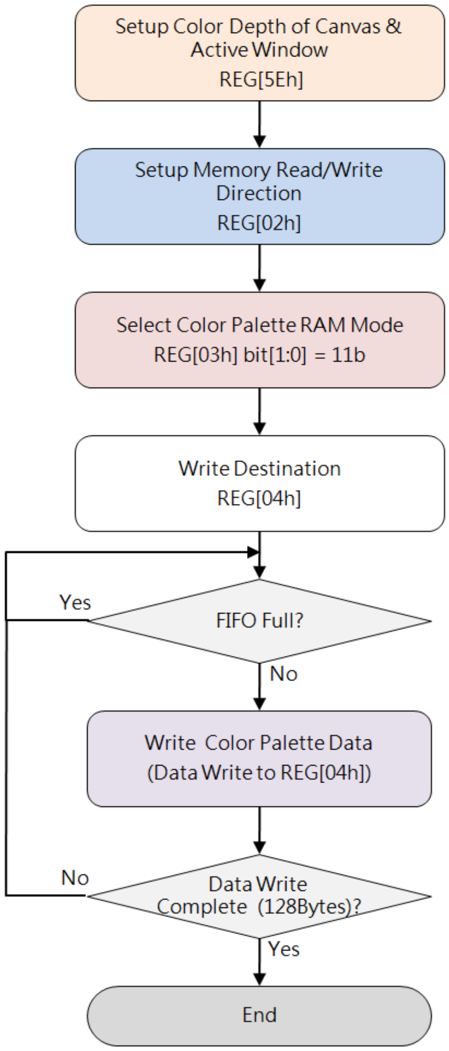


Figure 15-2: Flow Chart of Color Palette RAM Initialization

15.3 BTE Operation Overview

15.3.1 MCU Write with ROP

This Operation supports 16 ROP functions, BTE engine will perform the ROP function automatically then write the result data to DT.

15.3.2 Memory Copy with ROP

This operation supports 16 ROP functions, but supports data transfer in positive direction only.

15.3.3 Solid Fill

This operation fills a specified Rectangle Area of DT with a solid color data defined in the Foreground Color Register.

15.3.4 Pattern Fill

This operation fills DT with an 8*8 or 16*16 pixel Pattern.

15.3.5 Pattern Fill with Chroma Key

This operation fills DT with an 8*8 or 16*16 pixel Pattern, combined with Chroma Key function but ROP function. If the Pattern color is equal to the Chroma Key color, which is defined in Background Color Register, then the destination BTE Window will not be changed.

15.3.6 MCU Write with Chroma Key

This operation supports data transfer from S0 (MCU Write) to DT. If S0 color is equal to the Chroma Key color, which is defined in Background Color Register, then the destination BTE Window will not be changed, no ROP applied for this operation.

15.3.7 Memory Copy with Chroma Key

This operation supports data transfer in positive direction only, and requires all source data and destination data located in Memory. If S0 color is equal to Chroma Key color, which is defined in Background Color Register, then the destination BTE Window will not be changed, no ROP applied for this operation.

15.3.8 MCU Write with Color Expansion

This operation performs Color Expansion for the S0 (the data from MCU), from Monochrome 1 bpp format to Color 8/16/24 bpp format. The source data_1b will expand to the color data defined in the Foreground Color Register. The source data_0b will expand to the color data defined in the Background Color Register. If background transparency is enabled, then the Color of destination BTE Window will remain no change.

Note: No matter background transparency is enabled or not, the color data set in Foreground Color Register (D2h~D4h) must be different from the color data set in Background Color Register (D5h~D7h).

15.3.9 Memory Copy with Color Expansion

This operation performs Color Expansion for Source 0 data from Memory, please refer to 15.3.8 for other description and note.

15.3.10 Memory Copy with Opacity

This operation performs Alpha Blending for S0 data and S1 data and transferring the result data to the destination BTE Window, it requires S0, S1, DT located in Memory. The Alpha Blending has 2 modes, Picture Mode: for all pixels, blending by the same Alpha level. Pixel Mode: for each pixel, blending by individual Alpha Level of each pixel.

15.3.11 MCU Write with Opacity

This operation performs Alpha Blending for S0 data and S1 data and transferring the result data to the destination BTE Window, it requires S0 coming from MCU and S1/DT from Memory, please refer to 15.3.10 for the descriptions of the 2 modes.

15.4 BTE Memory Access Method

BTE accesses data of Sources and Destination by Block Method, and the size of the Data Block is defined by BTE Window. Below diagram shows how users can define S0 / S1 / DT / BTE_Window, and shows the directions of Memory Access:

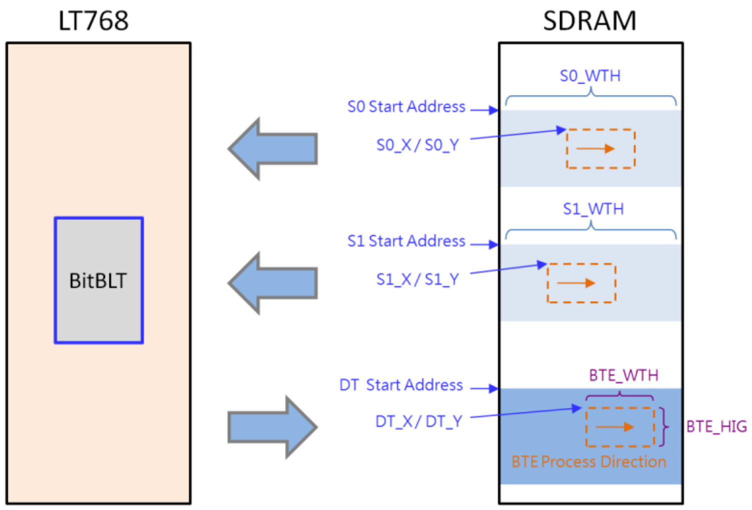


Figure 15-3: BTE Memory Access Method

15.5 BTE Chroma Key (Transparency Color) Function

If Chroma Key function is enabled, BTE will take Background Color (defined in Background Color Register) as the Chroma Key (Transparent Color), and compare the Chroma Key data against the S0 data one by one, if the compared result is equal, then BTE will not overwrite DT, otherwise BTE will write S0 data to DT.

The description below shows the available bits of the Background Color Registers against different Color Depth:

- **Source Color Depth = 256**

  Compare S0 Red color vs. REG[D5h] bit [7:5],

  Compare S0 Green color vs. REG [D6h] bit [7:5],

  Compare S0 Blue color vs. REG [D7h] bit [7:6]

- **Source Color Depth = 65,536**

  Compare S0 Red color vs. REG[D5h] bit [7:3],

  Compare S0 Green color vs. REG [D6h] bit [7:2],

  Compare S0 Blue color vs. REG [D7h] bit [7:3]

- **Source Color Depth = 16,777,216**

  Compare S0 Red color vs. REG[D5h] bit [7:0],

  Compare S0 Green color vs. REG [D6h] bit [7:0],

  Compare S0 Blue color vs. REG [D7h] bit [7:0]

15.6 BTE Operation Detail

15.6.1 MCU Write with ROP

This operation is to transfer S0 (form MCU Write) to DT, and supports all 16 ROP functions. Below diagram is an example for the operation result while BTE Control Register REG[91h] is set as 0x0Ch.

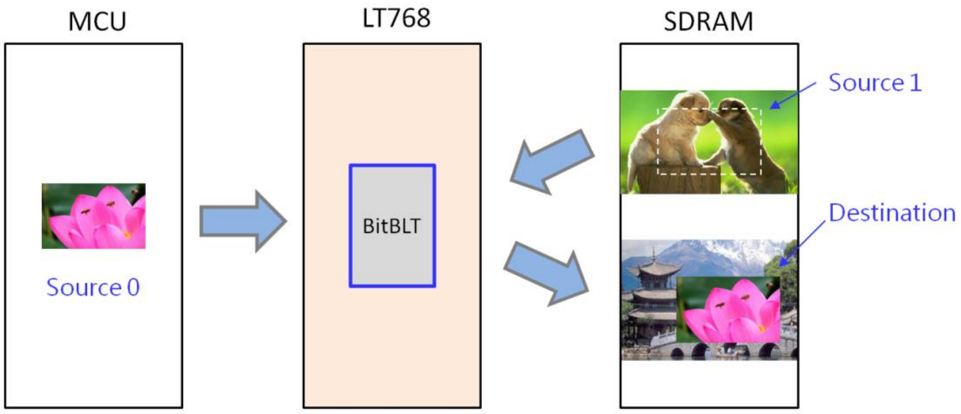


Figure 15-4: Example of MCU Write with ROP

The suggested programming steps and register settings are listed below as reference.

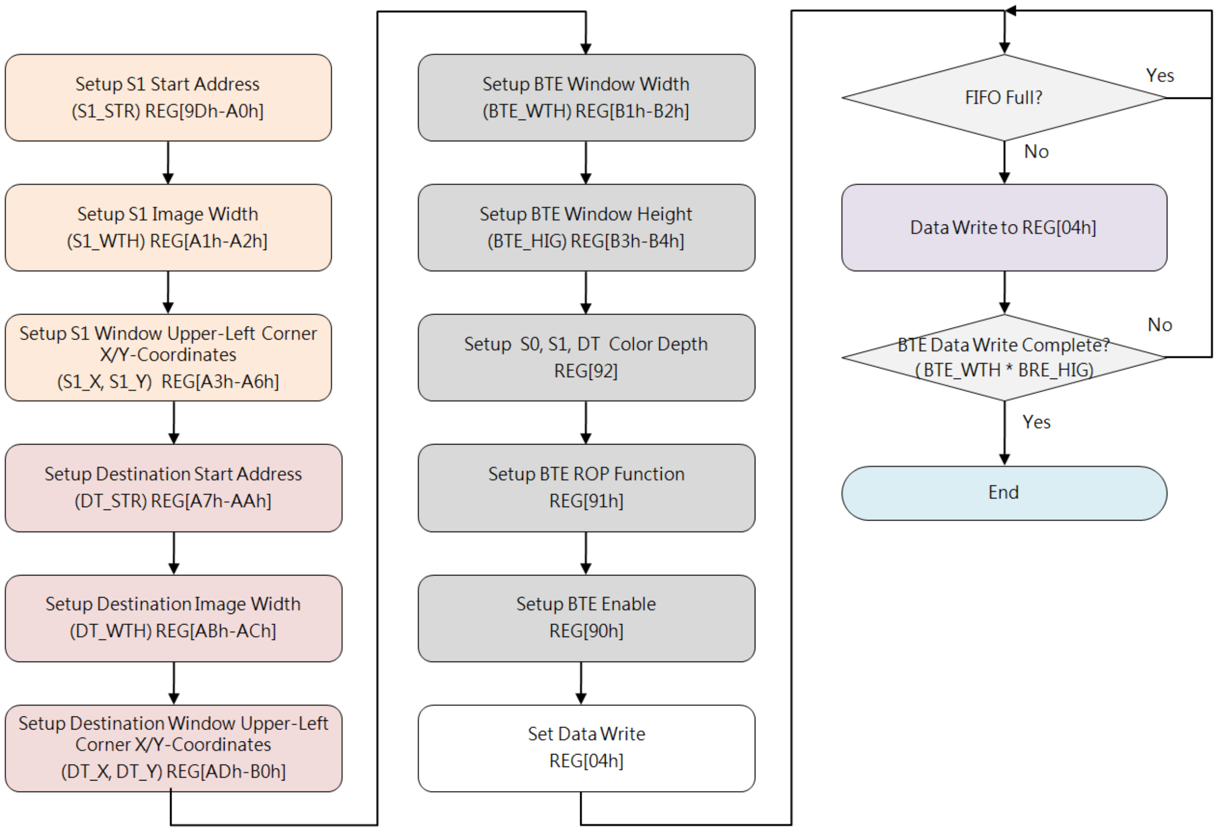


Figure 15-5: Flow Chart of MCU Wtite with ROP

15.6.2 Memory Copy (move) with ROP

This operation performs Memory Copy from S0 (from Memory) to DT. Below diagram is an example for the operation result while BTE Control Register REG[91h] is set as 0x0Ch.

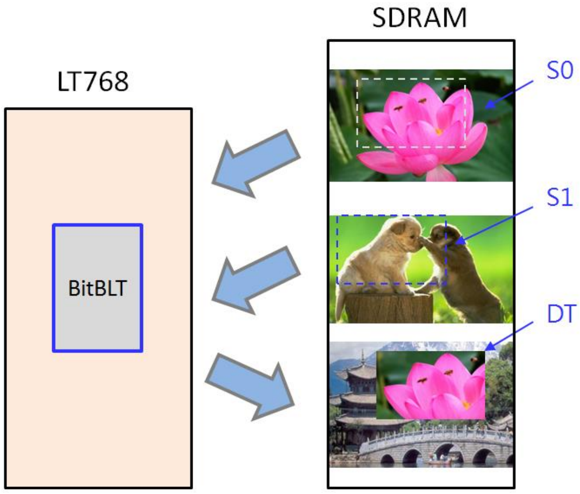


Figure 15-6: Example of Memory Copy with ROP

There are two ways to get BTE status. Figure 15-7 shows one way to get it by checking the Status Register STSR bit3, and Figure 15-8 shows another by checking hardware interruption.

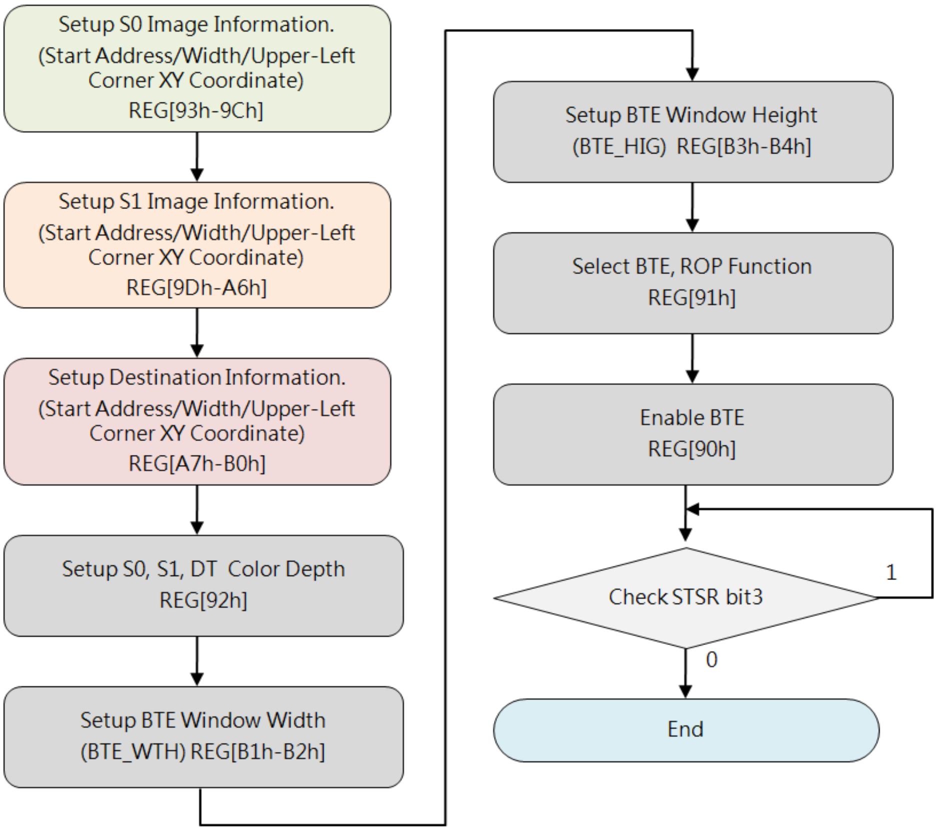


Figure 15-7: Flow Chart 1 of Memory Copy with ROP

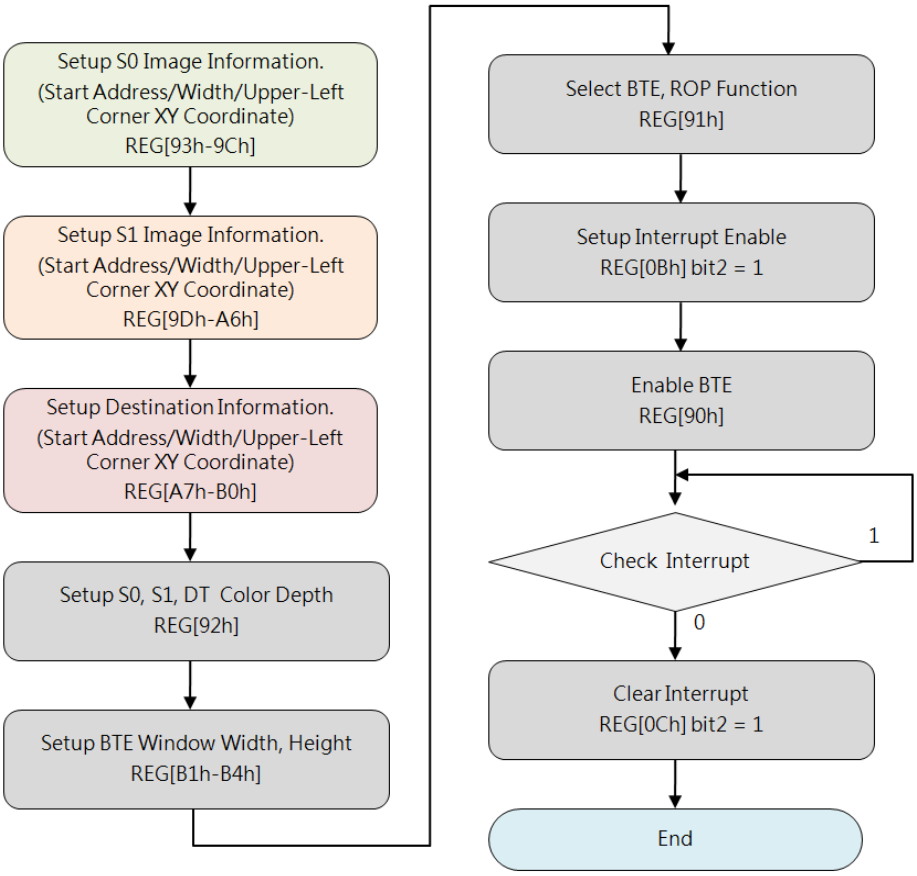


Figure 15-8: Flow Chart 2 of Memory Copy with ROP

15.6.3 MCU Write with Chroma Key (w/o ROP)

This operation is to transfer S0 (from MCU Write) to DT, and supports Chroma Key function (referring to 15.5).

If S0 color data is equal to the Chroma Key (the color data defined in the Background Color Registers REG[D5h-D7h]), BTE will take that color as transparent, which means BTE will not write that color data to DT. Below Example shows GREEN is the background color of RED "TOP", and GREEN is set as the Chroma Key, therefore BTE will write RED "TOP" only to DT:

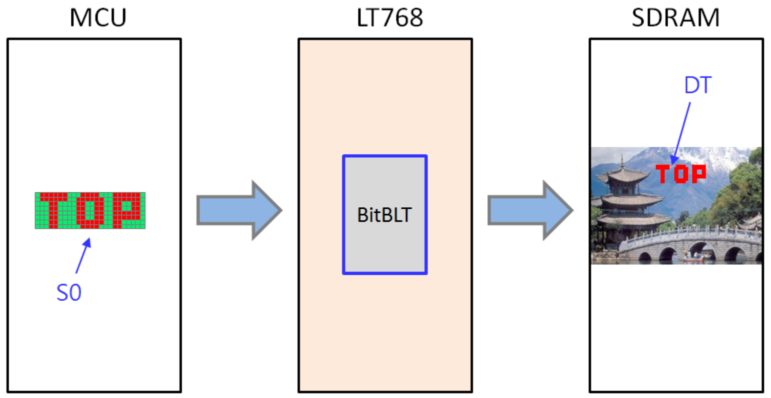


Figure 15-9: Example of MCU Write with Chroma Key

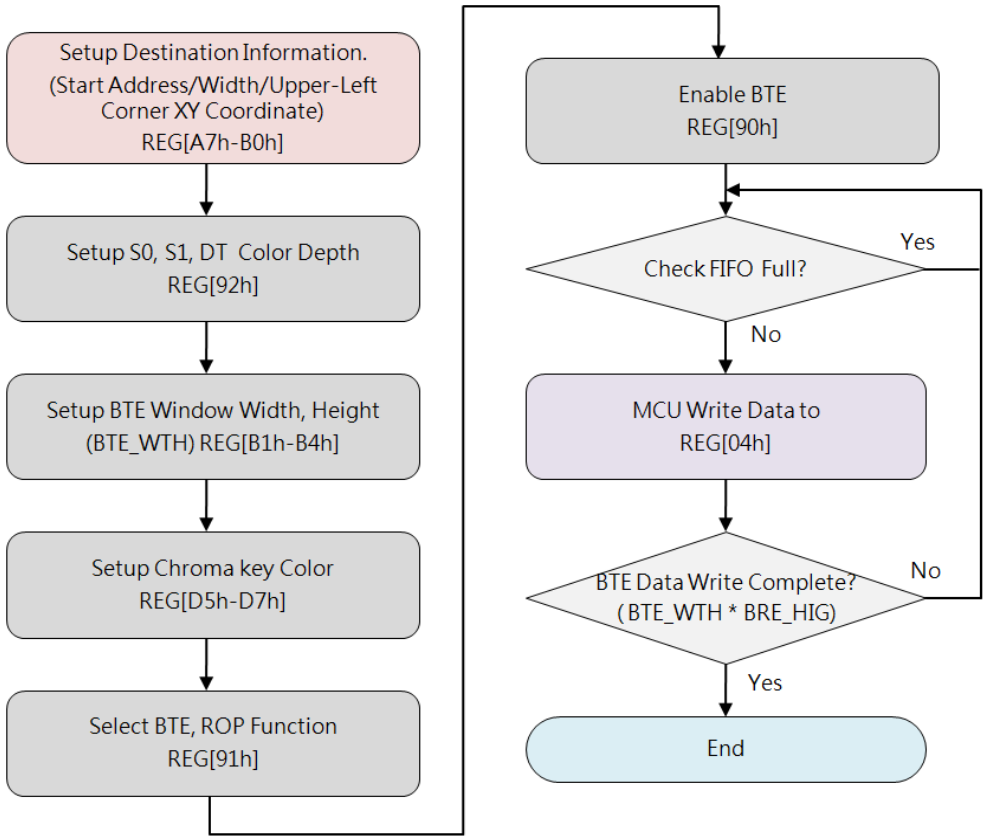


Figure 15-10: Flow Chart of MCU Write with Chroma Key

15.6.4 Memory Copy with Chroma Key (w/o ROP)

This operation performs Memory Copy from S0 (from Memory) to DT, and supports Chroma Key function (referring to Section 15.5).

The difference comparing with “MCU Write with Chroma Key” is that S0 comes from Memory not from MCU, please refer to Section 15.6.3 for more descriptions and Figure 15-11 for an example:

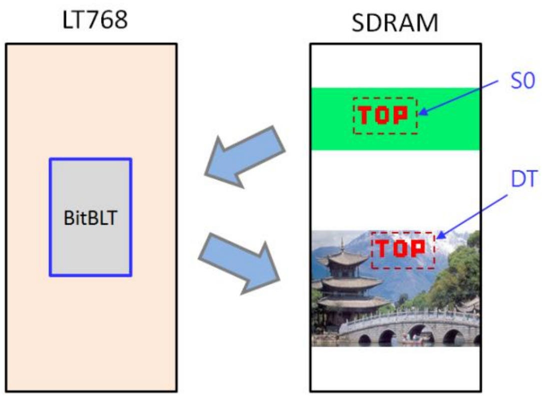


Figure 15-11: Example of Memory Copy with Chroma Key

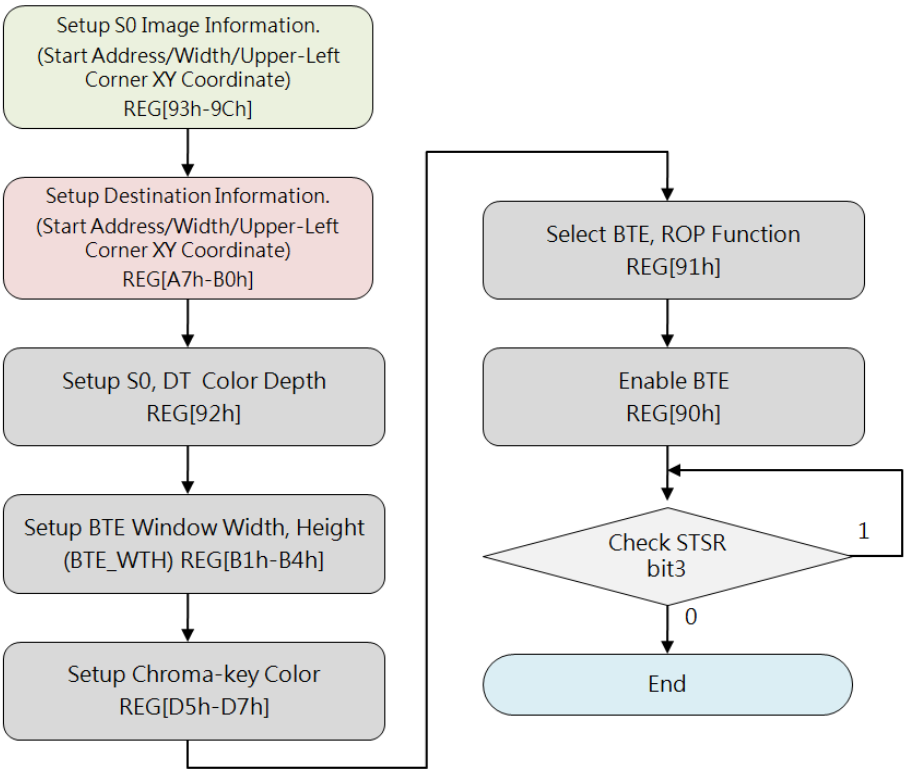


Figure 15-12: Flow Chart of Memory Copy with Chroma Key

15.6.5 Pattern Fill with ROP

This operation is to fill a pattern (as S0) repeatedly to a specified Rectangular Area of Destination (also known as BTE Window). The pattern should be an array of 8x8 or 16x16 pixels stored in Memory. The pattern can be logically processed with S1 by using one of the 16 ROP functions. This operation can be used to speed up duplicating a matrix of pattern into an area, such as background paste function.

Below example shows a six fills with 8x8 Pattern, ROP REG[91h] bit[7:4] is set as 0x0C:

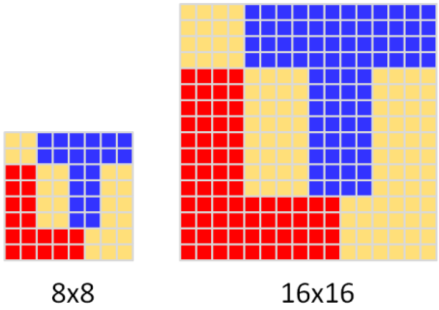


Figure 15-13: Pattern Format

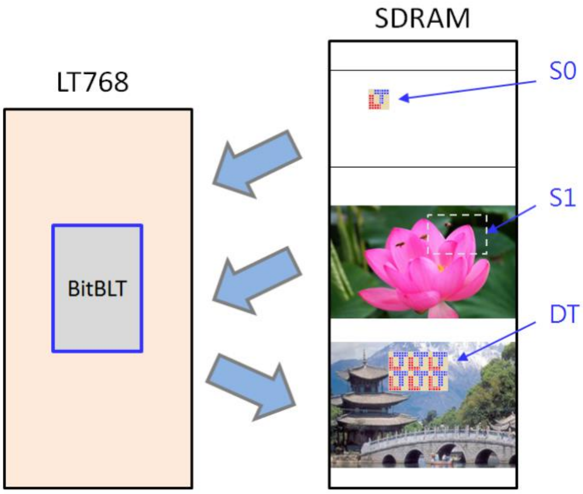


Figure 15-14: Example of Pattern Fill with ROP

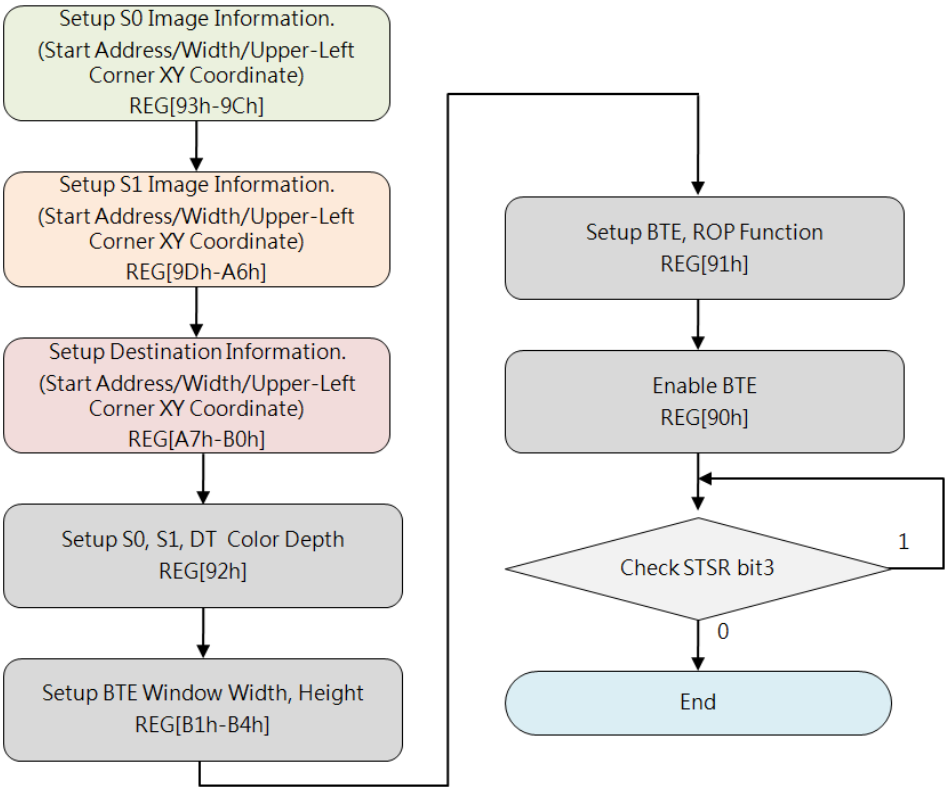


Figure 15-15: Flow Chart of Pattern Fill with ROP

15.6.6 Pattern Fill with Chroma Key

This operation is to fill a pattern (as S0) repeatedly to a specified Rectangular Area of DT (also known as BTE Window), and supports Chroma Key function (referring to Section 15.5). If any partial Color Data of the pattern is equal to the Chroma Key data, BTE will not write that part of data to DT.

Below Example shows ORANGE is background color of RED “T”, and ORANGE is set as Chroma Key, BTE will fill RED “T” only to DT one by one:

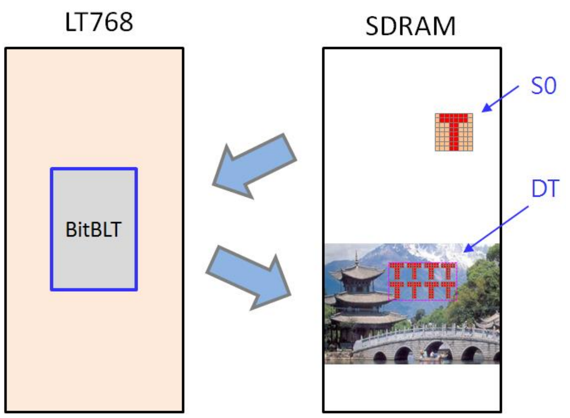


Figure 15-16: Example of Pattern Fill Chroma Key

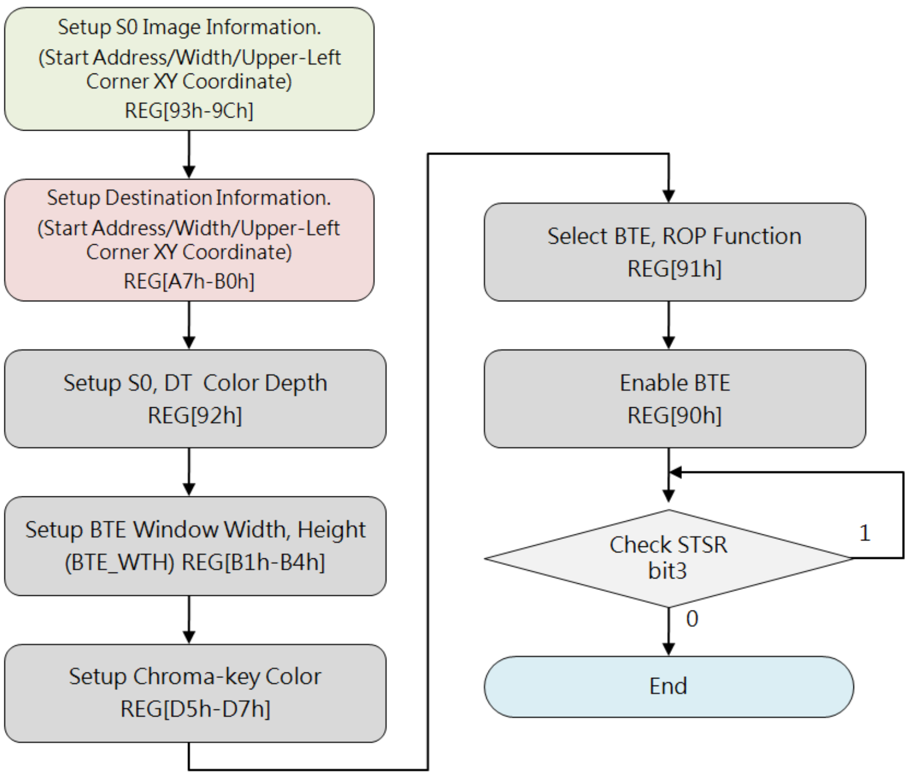


Figure 15-17: Flow Chart of Pattern Fill with Chroma Key

15.6.7 MCU Write with Color Expansion

This operation performs Color Expansion for S0 (from MCU Write), it is useful to translate bit-wise monochrome data to byte-wise color data. In this operation, S0 Color Depth REG[92h] Bit[6:5] is ignored, BTE takes MCU Interface Width as S0 Color Depth (Word Width), that is, if MCU Interface is 8 bits, then the Color Depth is 8-bits, and if MCU Interface is 16 bits, then the Color Depth is 16-bits. User should set needed Start Bit to REG[91h] Bit[7:4] against the corresponding Color Depth, so Bit[7]~Bit[0] are available for 8-bits depth and Bit[15]~Bit[0] are available for 16-bits depth. BTE disassembles Word by Word to Bit sequences (from LSB to MSB) against the ROW Line of source image (from left to right), and sequently expands bit by bit until the BTE Width reached ending. The result to DT is that data_1b expanded to Foreground Color and data_0b expanded to Background Color. Any Bit before the Start Bit and other Bits uncovered by the BTE Window will be discarded.

Below example shows expanding data_1b to RED (Foreground Color) and data_0b to BLUE (Background Color), so the result to DT is RED "TOP" together with BLUE background:

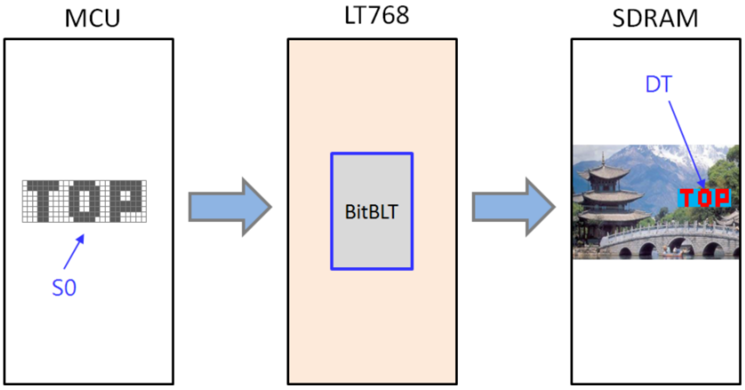


Figure 15-18: Example of MCU Write with Color Expansion

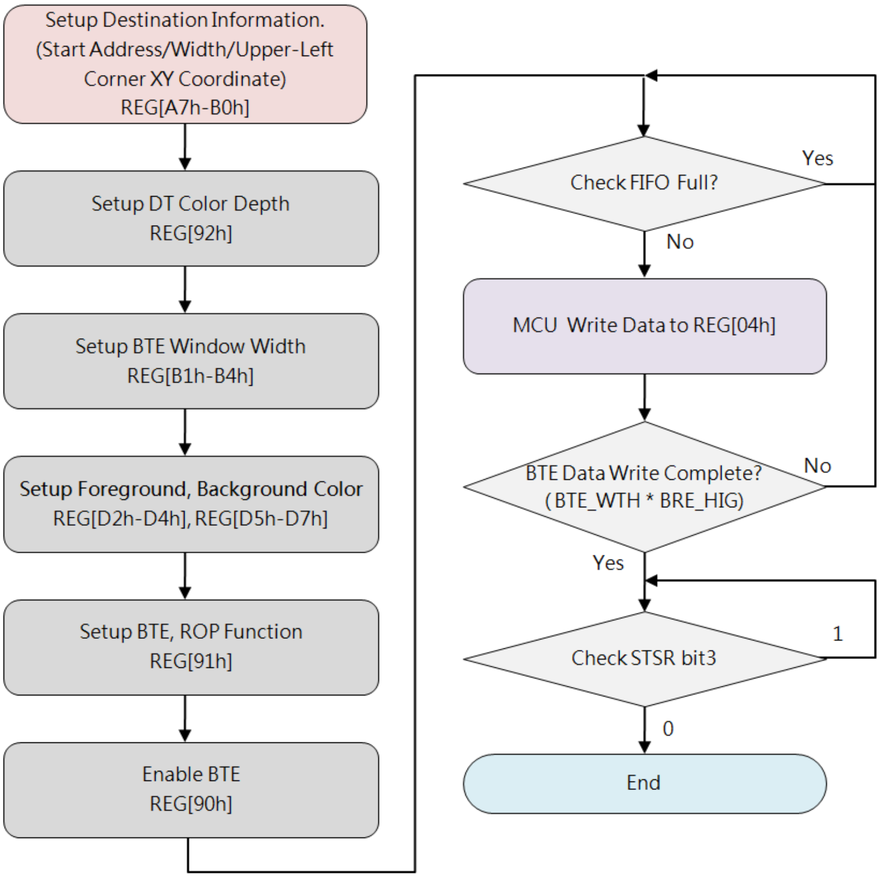


Figure 15-19: Flow Chart of MCU Write with Color Expansion

For 8-bits MCU Inerface, if ROP = 7, Foreground Color is set to RED, Background Color is set to KHAKI, and BTE Width is set to 23, then the color expansion result is as shown in Figure 15-20. If ROP = 3, then the result is shown as Figure 15-21.

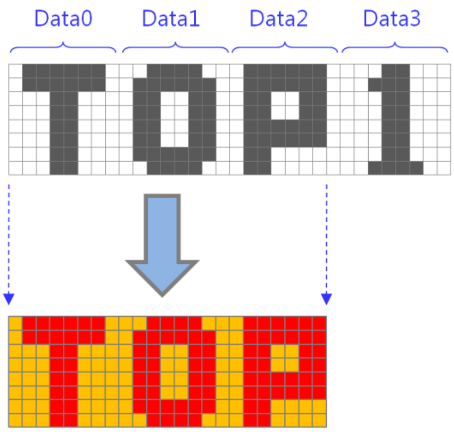


Figure 15-20: Example 1 of MCU Write with Color Expansion (ROP=7)

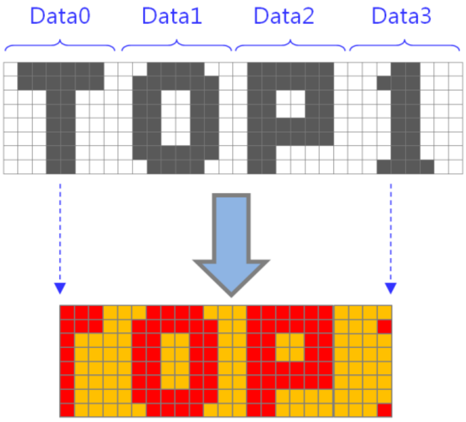


Figure 15-21: Example 2 of MCU Write with Color Expansion (ROP=3)

Note:

1. Sent Word_ Numbers_per_Row
   = [BTE_Width + (MCU_Interface_bits – Start_bit - 1)] / (MCU_ Interface _bits)
   (Take integer with unconditional carry)
2. Word_Number_Total = (Word_ Numbers_per_Row) * BTE Height

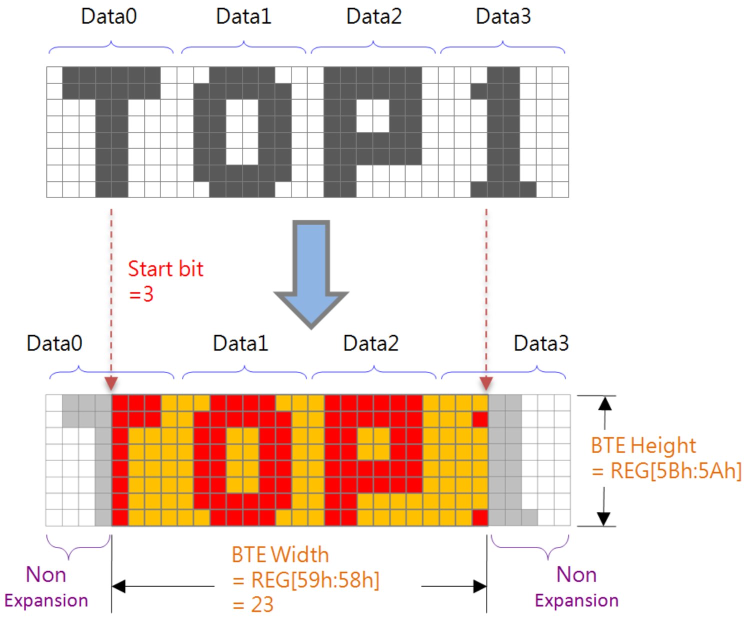


Figure 15-22: Data Format for Color Expansion

Example 1: "BTE Width"= 50, "MCU I/F bits" = 8 bits,

If Start bit=7, then:
Sent Data Numbers per Row = [{ 50 + (8 - 7 - 1) } / 8] = 7 Bytes

If Start bit=4, then:
Sent Data Numbers per Row = [{ 50 + (8 - 4 - 1) } / 8] = 7 Bytes

Example 2: "BTE Width"= 50, "MCU I/F bits" = 16 bits,

If Start bit =15, then:
Sent Data Numbers per Row = [{ 50 + (16 - 15 - 1) } / 16] = 4 Bytes

If Start bit=0, then:
Sent Data Numbers per Row = [{ 50 + (16 - 0 - 1) } / 16] = 5 Bytes

15.6.8 MCU Write with Color Expansion and Chroma key

This operation is similar to MCU Write with Color Expansion, but the difference is that data_0b will be discarded, BTE expands only data_1b to Foreground Color.

Below example shows expanding data_1b to RED (Foreground Color) and data_0b discarded, so the result to DT is RED “TOP” with TRANSPARENT background:

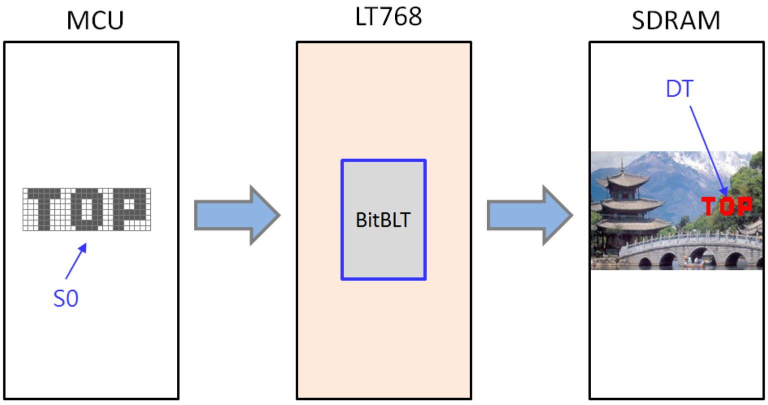


Figure 15-23: Example of MCU Write with Color Expansion and Chroma key

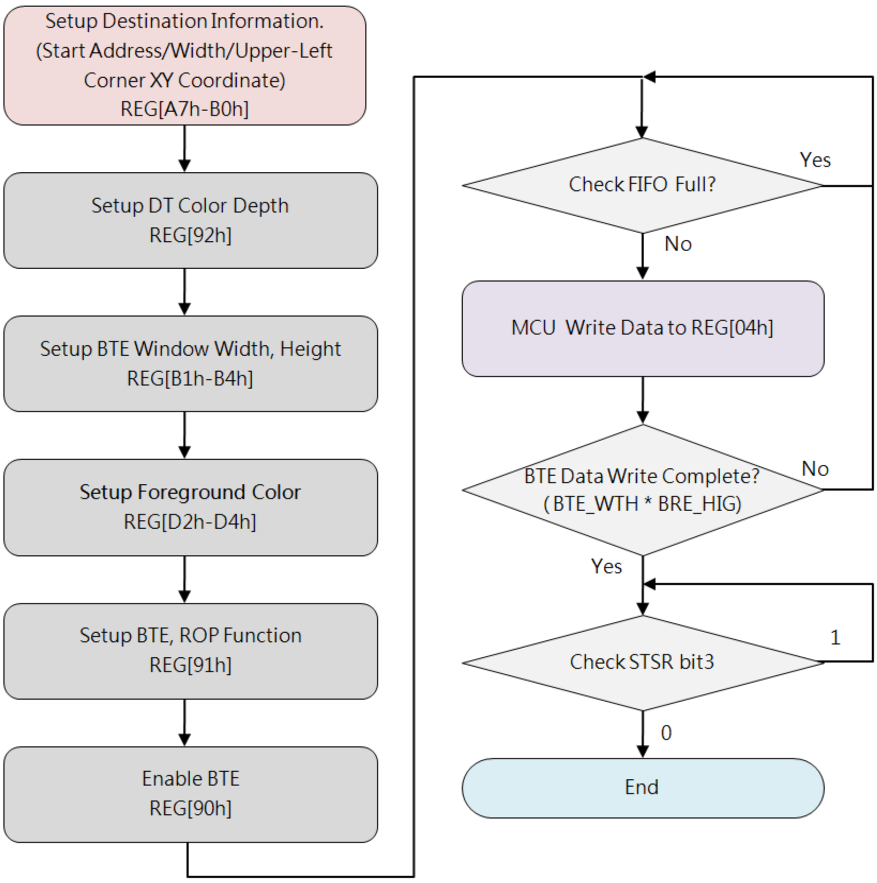


Figure 15-24: Flow Chart of MCU Write with Color Expansion and Chroma key

15.6.9 Memory Copy with Opacity

This operation performs blending S0 and S1 data and transferring the result to DT, two modes are available: Picture Mode and Pixel Mode.

Picture Mode is for 8 bpp/16bpp/24bpp format and it uses the same Opacity Value (Alpha Level) for whole bitmap picture. Opacity Value of Picture Mode is defined in REG[B5h].

Pixel Mode is for 8bpp/16bpp format and it uses individual Opacity Value of S1, each pixel of S1 has its own Opacity Value, such as: for each 16bpp data, the bit[15:12] is Opacity Value, the bit[11:0] is color data; for each 8bpp data of S1, the bit[7:6] is Opacity Value, the bit[5:0] is the Index (Address) of Palette Color RAM pointing to the initialized 12-bits Color Depth data.

- **Picture Mode:**
  Alpha_Level = REG[B5h]
  DT Data = (S0 * Alpha_Level) + (S1 * (1- Alpha_Level))
- **Pixel Mode 8bpp**:
  Alpha_Level = S1_Bit[7:6]
  DT Data = (S0 * Alpha_Level) + (Palette_Color_RAM[S1_Bit[5:0]] * (1 – Alpha_ Level))
- **Pixel Mode 16bpp**:
  Alpha_Level = S1_Bit[15:12]
  DT Data = (S0 * Alpha_Level) + (S1_Bit[11:0] * (1 – Alpha_Level))

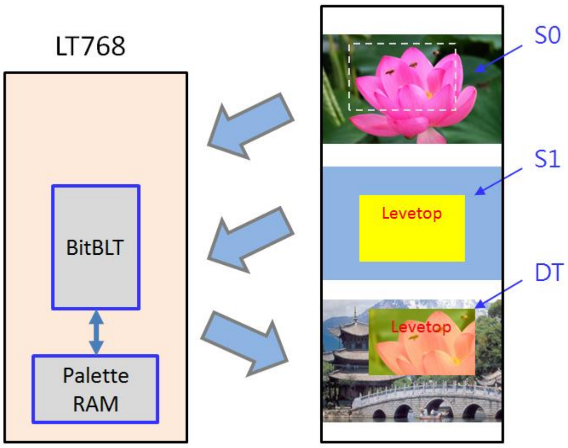


Figure 15-25: Example of Pixel Mode - 8bpp

Table 15-4: Alpha Blending of Pixel Mode - 8bpp

Bit[7:6]	Alpha Level
0h	0
1h	10/32
2h	21/32
3h	1

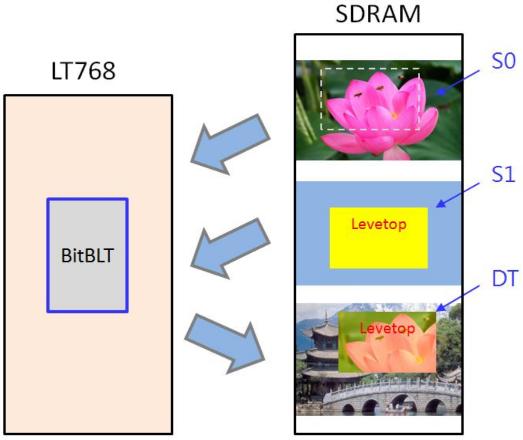


Figure 15-26: Example of Pixel Mode - 16bpp

Table 15-5: Alpha Level of Pixel Mode - 16bpp

Bit[15:12]	Alpha Level
0h	0
1h	2/32
2h	4/32
3h	6/32
4h	8/32
5h	10/32
6h	12/32
7h	14/32
8h	16/32
9h	18/32
Ah	20/32
Bh	22/32
Ch	24/32
Dh	26/32
Eh	28/32
Fh	1

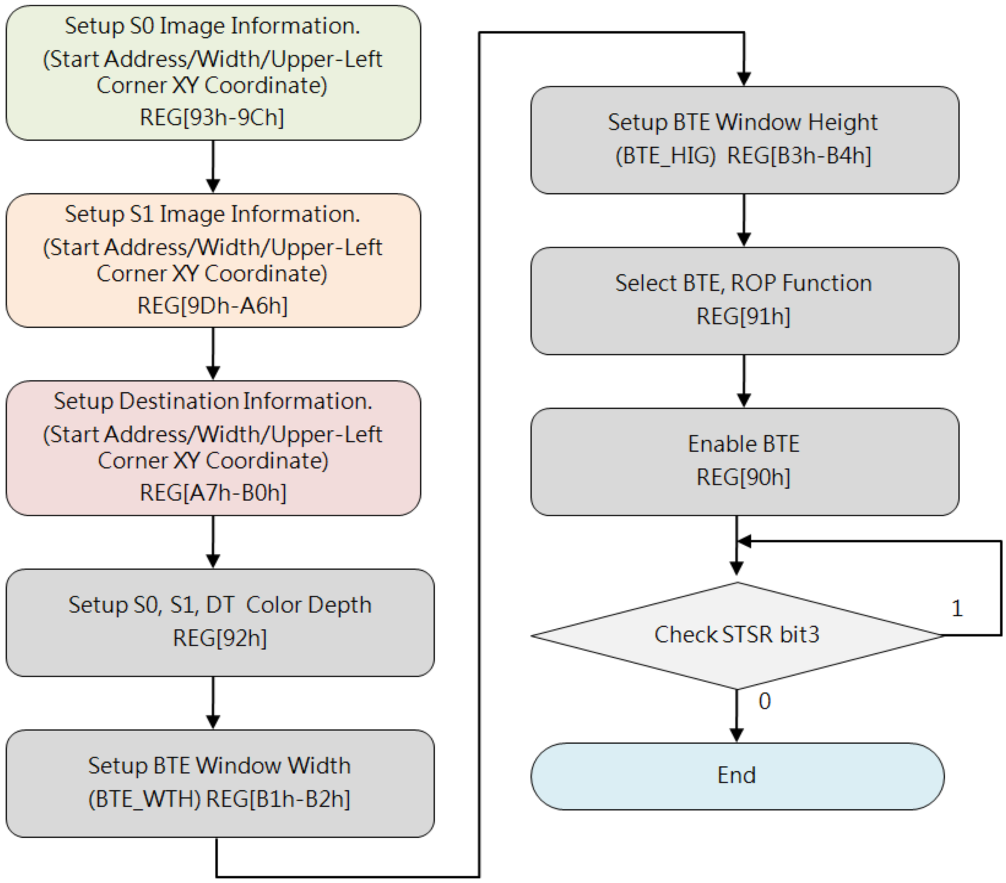


Figure 15-27: Flow Chart of Memory Copy with Opacity -Pixel Mode

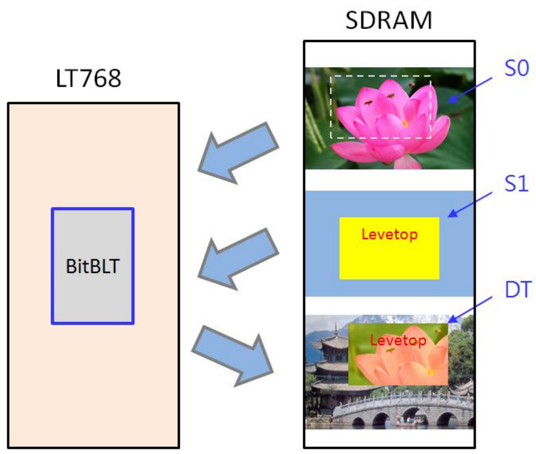


Figure 15-28: Example of Memory Copy with Opacity - Picture Mode

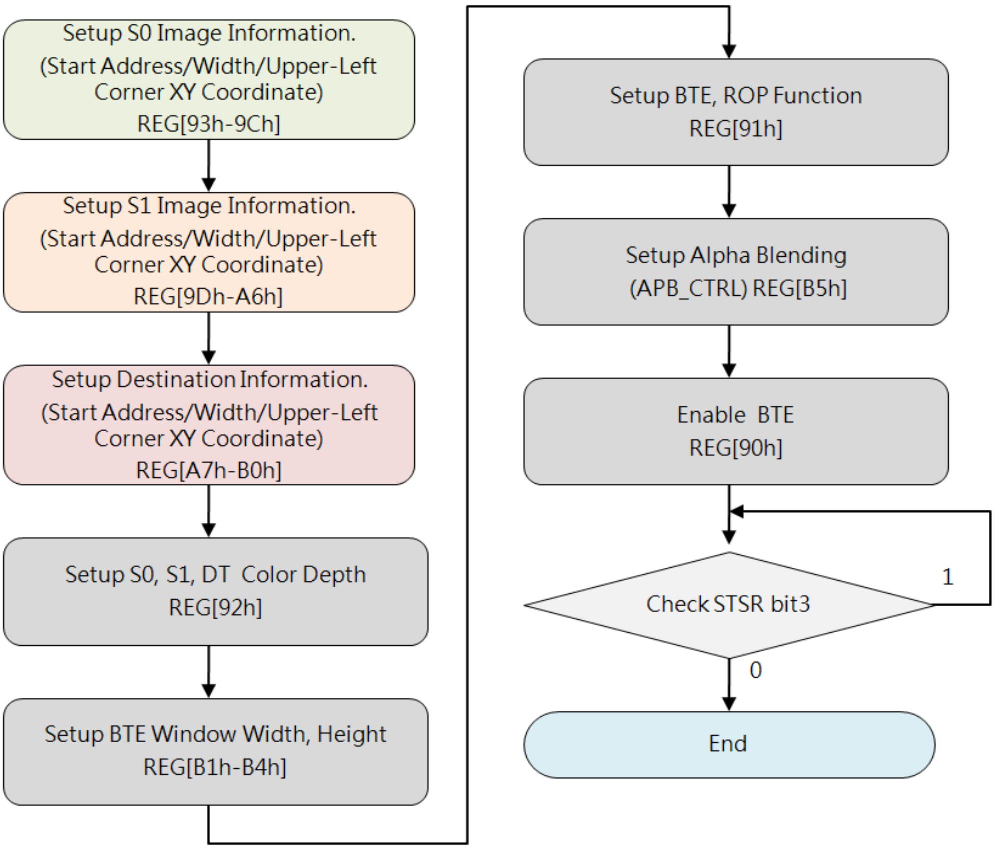


Figure 15-29: Flow Chart of Memory Copy with Opacity - Picture Mode

15.6.10 MCU Write with Opacity

This operation is similar to Memory Copy with Opacity, but the difference is that S0 is from MCU Write, and it also has Picture Mode and Pixel Mode, for more descriptions please refer to Section 15.6.9.

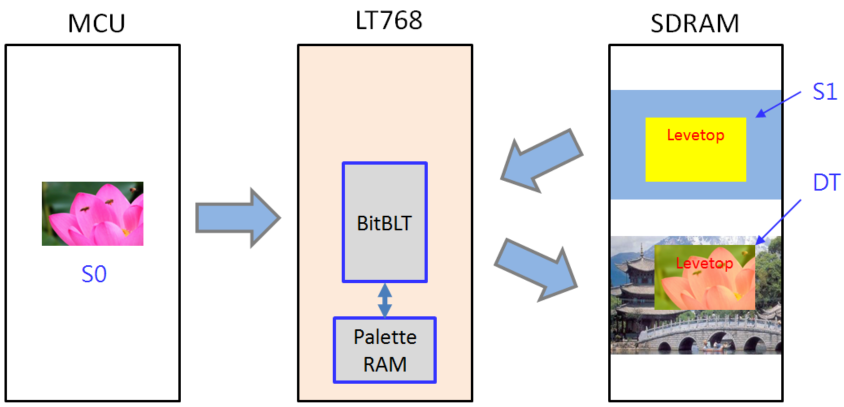


Figure 15-30: Example of MCU Write with Opacity

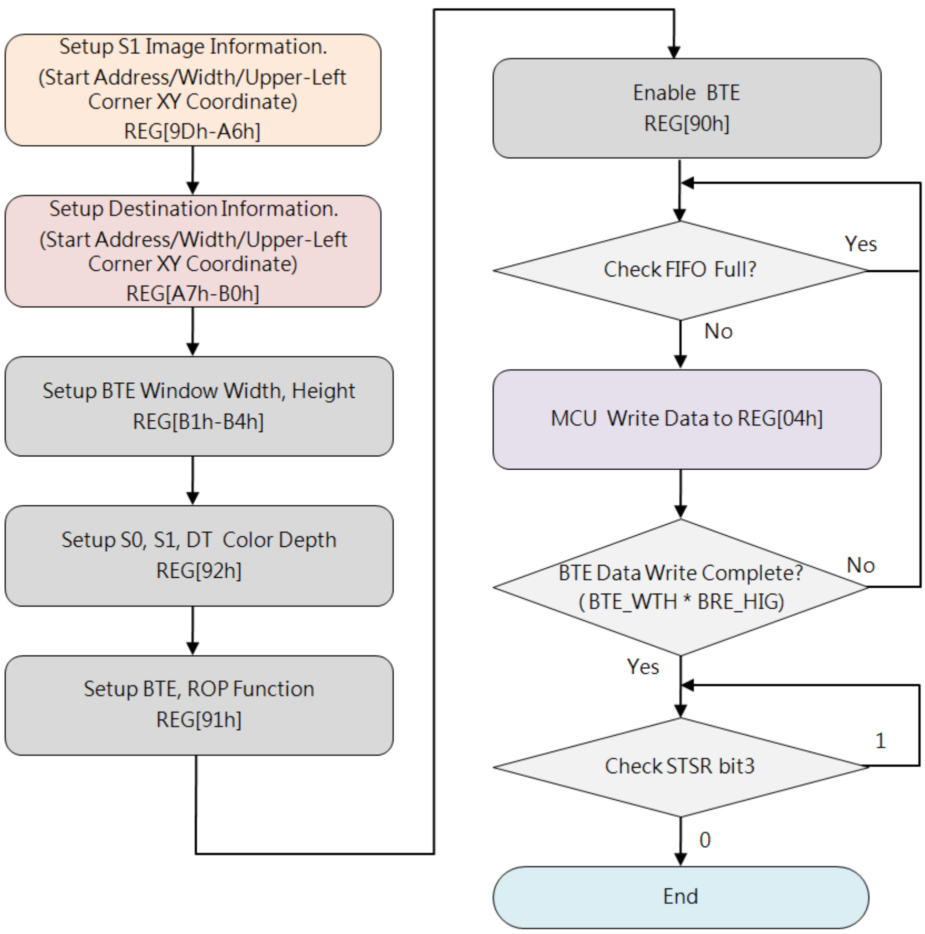


Figure 15-31: Flow Chart of MCU Write with Opacity

15.6.11 Memory Copy with Color Expansion

This operation performs Color Expansion for the S0 data (from Memory), it is useful to translate bit-wise monochrome data to byte-wise color data. In this operation, S0 data Color Depth (Word Width) is defined in REG[92h] Bit[6:5]. Users should set needed Start Bit to REG[91h] Bit[7:4] based on the corresponding Color Depth, Bit[7]~Bit[0] are available for 8-bits depth and Bit[15]~Bit[0] are available for 16-bits depth. BTE disassembles Word by Word to Bit sequences (from MSB to LSB) based on the ROW Line of source image (from left to right), and sequently expands bit by bit until the BTE Width reached ending as well. The result to DT is that data_1b expanded to Foreground Color and data_0b expanded to Background Color. Any Bit before the Start Bit and other Bits uncovered by the BTE Window will be discarded.

Below example shows expanding data_1b to RED (Foreground Color) and data_0b to BLUE (Background Color), so the result to DT is RED "TOP" together with BLUE background:

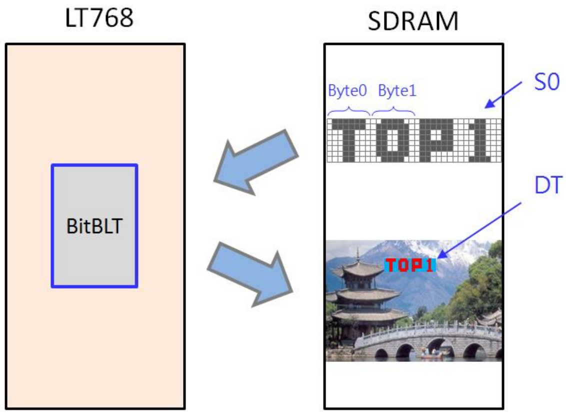


Figure 15-32: Example of Memory Copy with Color Expansion

While: S0 Color Depth = 8, Foreground Color = RED, Background Color = KHAKI, BTE window Width = 23, please refer to Figure 15-33 (ROP = 7) and Figure 15-34 (ROP = 4) as examples.

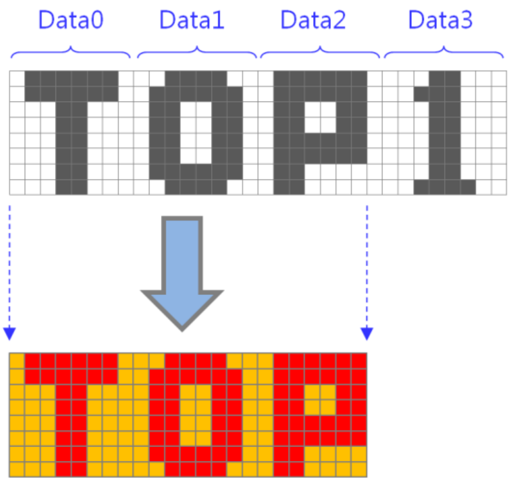


Figure 15-33: Example 1 of Memory Copy with Color Expansion (ROP=7)

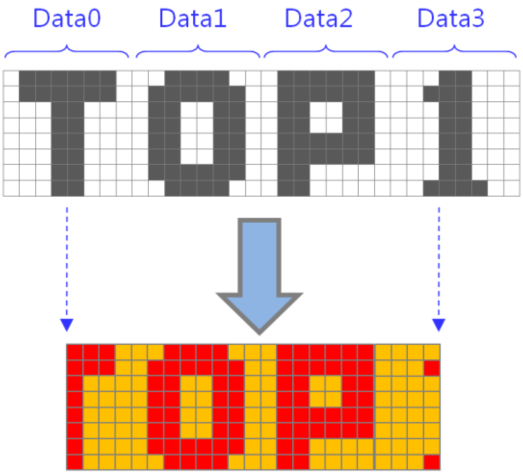


Figure 15-34: Example 2 of Memory Copy with Color Expansion (ROP=4)

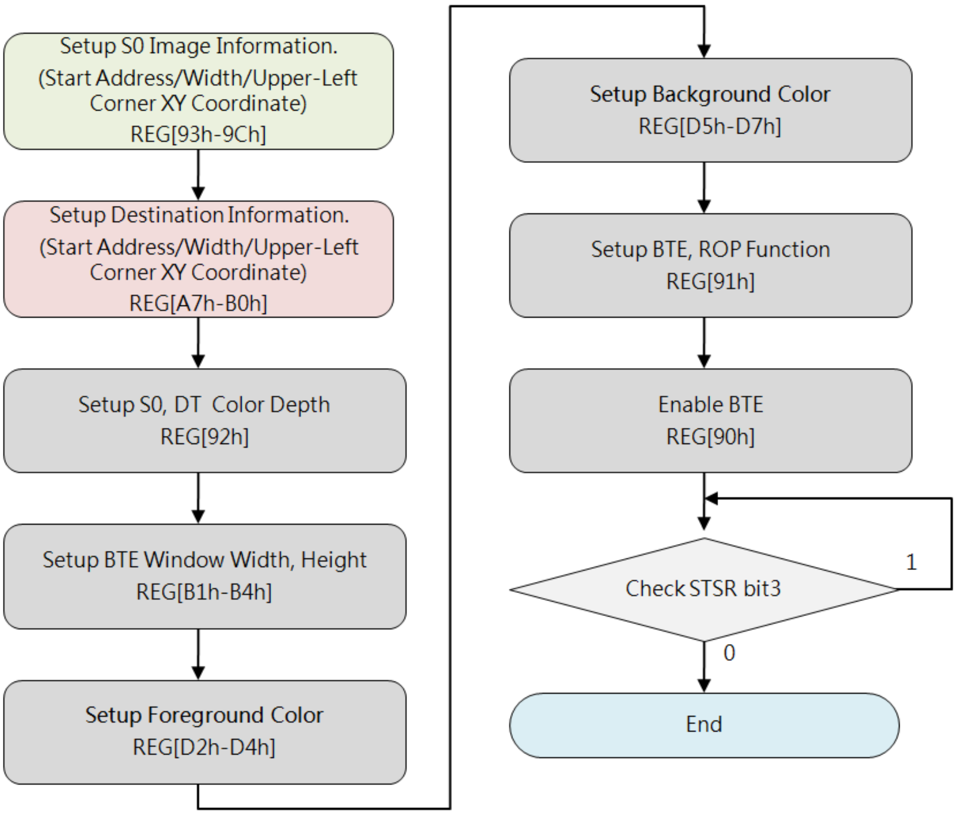


Figure 15-35: Flow Chart of Memory Copy with Color Expansion

15.6.12 Memory Copy with Color Expansion and Chroma Key

This operation is similar to Memory Copy with Color Expansion, but the difference is that data_0b will be discarded, BTE expands only data_1b to Foreground Color.

Below example shows data_1b is expanded to RED (Foreground Color) and data_0b is discarded, so the result to DT is RED "TOP" with TRANSPARENT background:

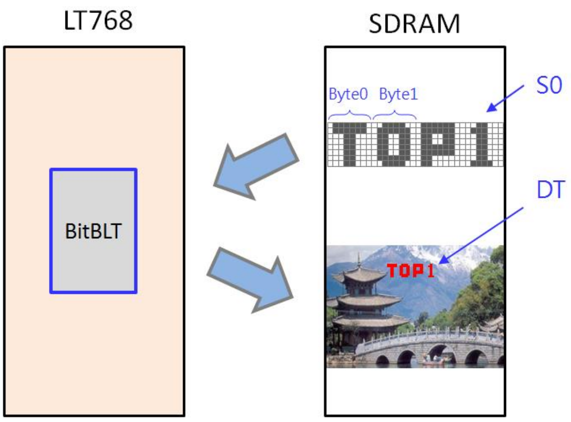


Figure 15-36: Example of Memory Copy with Color Expansion and Chroma Key

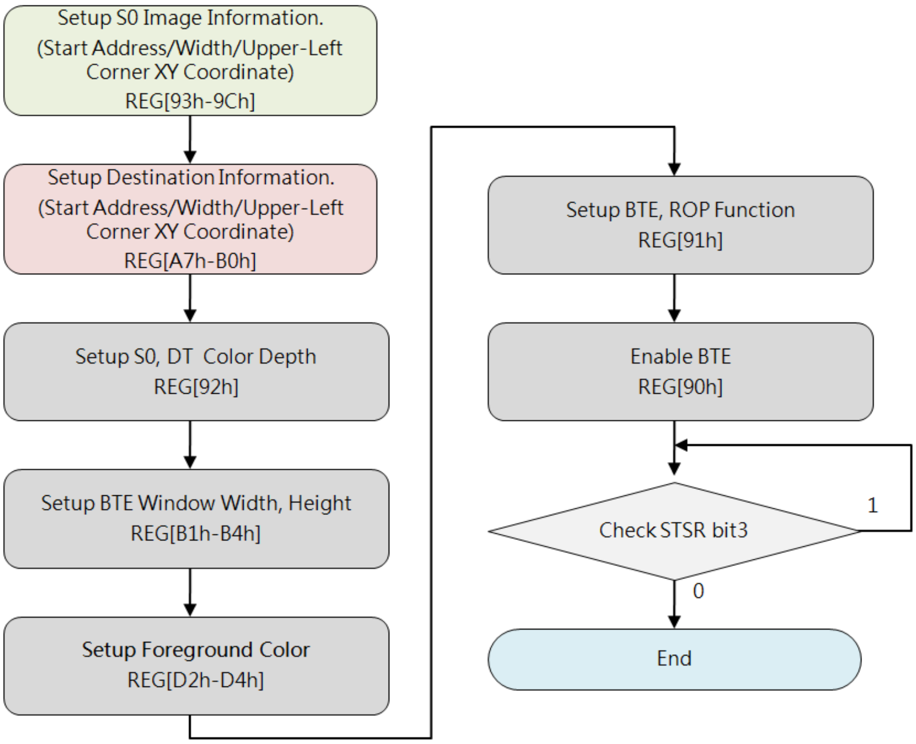


Figure 15-37: Flow Chart of Memory Copy with Color Expansion and Chroma Key

15.6.13 Solid Fill

This operation is to fill a specified Rectangle Area (BTE Window) of DT with a Solid Color whose data is defined in the Foreground Color Register.

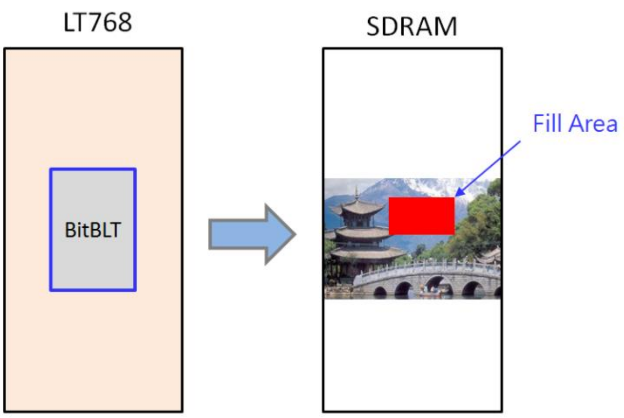


Figure 15-38: Example of Solid Fill

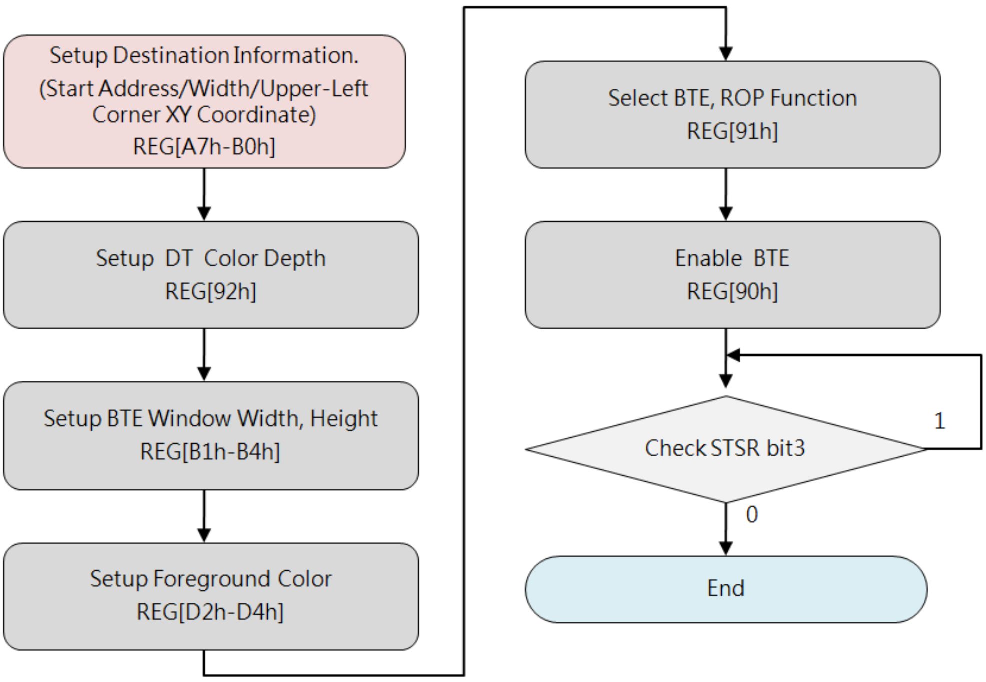


Figure 15-39: Flow Chart of Solid Fill

16. Display Text

LT7689 supports 2 sources of Text (Characters and Symbols):

- CGROM(Character Graphic ROM): Embedded ISO/IEC 8859 Character Sets
- CGRAM(Character Graphic RAM): User-defined Character Sets (UCG sets)

LT7689 internal CGROM supports four sets of embedded Characters and Symbols of ASCII code, and internal CGRAM supports users to create their own Characters and Symbols sets when needed. The registers REG[CCh] ~REG[DEh] are for the purpose of User-defined Characters. The Foreground Color registers REG[D2h] ~REG[D4h] and the Background Color registers REG[D5h] ~ REG[D7h] are also employable to define Color for characters from any source.

16.1 Internal CGROM

LT7689 builts in three ASCII font types with different resolutions (character sizes): 8 x 16, 12 x 24, 16 x 32. The MCU just simply writes the font code to make the ASCII word display on the LCD panel. The size of the displayed ASCII characters is set by REG[CCh] [5:4]. The ASCII code is corresponding to the standard ISO/IEC 8859-1/2/4/5 coding (table 16-1 to Table 16-4 below). The type of ISO/IEC 8859 font displayed on the TFT panel is set by REG [CCh] [1:0]. In addition, the user can select the color of the text by setting the front-view register REG (D2h to D4h) and the background color register (REG (D5h~D7h). Please refer to the following program flowchart:

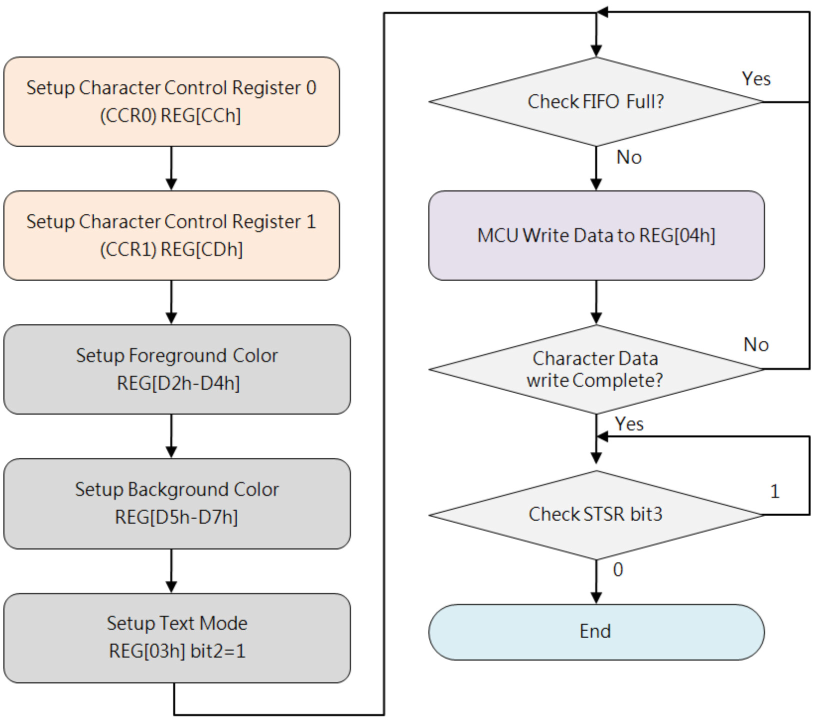


Figure 16-1: Flow Chart of CGROM Basic Programming

Table 16-1 shows the standard character encoding of ISO/IEC 8859-1. ISO stands for International Standardization Organization. The ISO/IEC 8859-1, generally known as “Latin-1”, it is the first sets of 8-bit coded character encoding developed by the ISO, and referred to ASCII that consists of 192 characters from the Latin script in the range of 0xA0-0xFF. This character coding is used throughout Western Europe, including Albanian, Afrikaans, Breton, Danish, Faroese, Frisian, Galician, German, Greenlandic, Icelandic, Irish, Italian, Latin, Luxembourgish, Norwegian, Portuguese, Rhaeto-Romanic, Scottish Gaelic, Spanish, Swedish. English letters without accent marks also can use ISO/IEC 8859-1. In addition, it is also commonly used in many languages outside Europe, such as Swahili, Indonesian, Malaysian and Tagalong.

In the Table 16-1, character codes 0x80~0x9F are defined by Microsoft Windows, also called CP1252 (WinLatin1).

Table 16-1: SO/IEC 8859-1

H\L	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		☺	☻	♥	♦	♣	♠	•	◘	○	◙	♂	♀	♪	♫	☼
1	►	◄	↕	‼	¶	§	▬	↨	↑	↓	→	←	∟	↔	▲	▼
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	⌂
8	€		‚	ƒ	„	…	†	‡	ˆ	‰	Š	‹	Œ		Ž	
9		'	'	"	"	•	–	—	˜	™	š	›	œ		ž	Ÿ
A		¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
B	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

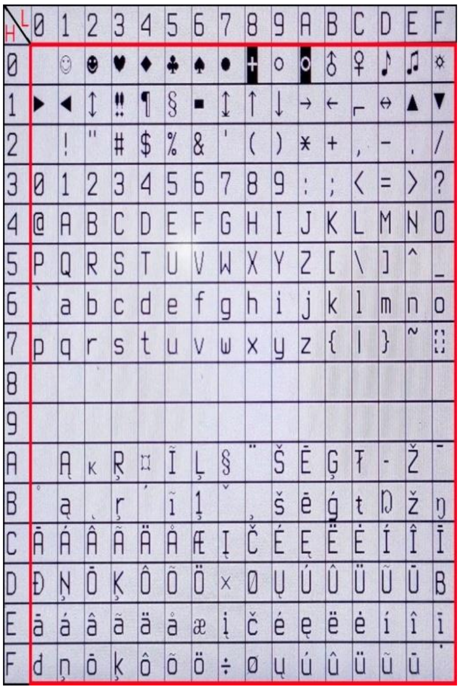
Table 16-2: ISO/IEC 8859-2

H\L	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		☺	☻	♥	♦	♣	♠	•	◘	○	◙	♂	♀	♪	♫	☼
1	►	◄	↕	‼	¶	§	▬	↨	↑	↓	→	←	∟	↔	▲	▼
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	⌂
8																
9																
A		Ą	˘	Ł	¤	Ľ	Ś	§	¨	Š	Ş	Ť	Ź	-	Ž	Ż
B	°	ą	˛	ł	´	ľ	ś	ˇ	¸	š	ş	ť	ź	˝	ž	ż
C	Ŕ	Á	Â	Ă	Ä	Ĺ	Ć	Ç	Č	É	Ę	Ë	Ě	Í	Î	Ď
D	Đ	Ń	Ň	Ó	Ô	Ő	Ö	×	Ř	Ů	Ú	Ű	Ü	Ý	Ţ	ß
E	ŕ	á	â	ă	ä	ĺ	ć	ç	č	é	ę	ë	ě	í	î	ď
F	đ	ń	ň	ó	ô	ő	ö	÷	ř	ů	ú	ű	ü	ý	ţ	˙

Table 16-2 shows the standard characters of ISO/IEC 8859-2, also known as Latin-2, it is the second sets of the 8-bit character encoding developed by ISO. This code sets can be used in almost any data interchange system to communicate in the following European languages: Croatian, Czech, Hungarian, Polish, Slovak, Slovenian, and Upper Sorbian. The Serbian, English, German, Latin can use ISO/IEC 8859-2 as well. Furthermore it is suitable to represent some western European languages like Finnish (with the exception of å used in Swedish and Finnish)

Table 16-3 shows the standard characters of ISO/IEC 8859-4, also known as Latin-4 or “North European”, it is the fourth sets of the ISO/IEC 8859 8-bit character encoding. It was designed originally to cover Estonian, Greenlandic, Latvian, Lithuanian, and Sami. This character set also supports Danish, English, Finnish, German, Latin, Norwegian, Slovenian, and Swedish.

Table 16-3: ISO/IEC 8859-4



H\L	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		☺	☻	♥	♦	♣	♠	•	◘	○	◙	♂	♀	♪	♫	☼
1	►	◄	↕	‼	¶	§	▬	↨	↑	↓	→	←	∟	↔	▲	▼
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	⌂
8																
9																
A		Ą	ĸ	Ŗ	¤	Ĩ	Ļ	§	¨	Š	Ē	Ģ	Ŧ	-	Ž	¯
B	°	ą	˛	ŗ	´	ĩ	ļ	ˇ	¸	š	ē	ģ	ŧ	Ŋ	ž	ŋ
C	Ā	Á	Â	Ã	Ä	Å	Æ	Į	Č	É	Ę	Ë	Ė	Í	Î	Ī
D	Đ	Ņ	Ō	Ķ	Ô	Õ	Ö	×	Ø	Ų	Ú	Û	Ü	Ũ	Ū	ß
E	ā	á	â	ã	ä	å	æ	į	č	é	ę	ë	ė	í	î	ī
F	đ	ņ	ō	ķ	ô	õ	ö	÷	ø	ų	ú	û	ü	ũ	ū	˙

Table 16-4: ISO/IEC 8859-5

H\L	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		☺	☻	♥	♦	♣	♠	•	◘	○	◙	♂	♀	♪	♫	☼
1	►	◄	↕	‼	¶	§	▬	↨	↑	↓	→	←	∟	↔	▲	▼
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	⌂
8																
9																
A		Ё	Ђ	Ѓ	Є	Ѕ	І	Ї	Ј	Љ	Њ	Ћ	Ќ	-	Ў	Џ
B	А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
C	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
D	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
E	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я
F	№	ё	ђ	ѓ	є	ѕ	і	ї	ј	љ	њ	ћ	ќ	§	ў	џ

Table 16-4 shows the standard characters of ISO/IEC 8859-5. also known as the fifth sets of the ISO/IEC 8859 8-bit character encoding. It was designed originally to cover Bulgarian ,Belarusian, Russian, Serbian and Macedonian.

8*16 12*24 16*32

Figure 16-2: Internal ASCII Font for 8*16, 12*24 and 16*32

16.2 User-defined Character Graphic (UCG)

User can create and utilize UCG when needed. LT7689 supports Half Width size (8x16, 12x24, 16x32 dot-matrix graphic) and Full Width size (16x16, 24x24, 32x32 dot-matrix graphic), and supports up to 32,768 UCGs with half width by encoding from 0000h up to 7FFFh, and up to 32,768 UCGs with full width by encoding from 8000h up to FFFFh.

To display a specific UCG, MCU just needs to write corresponding CODE of the UCG to LT7689, LT7689 can resolve the CODE (relative ADDRESS of CGRAM) to corresponding absolute ADDRESS of Memory where the UCG data is really saved, then transfer the graphic data to display Memory Buffer. Of course, users can define Foreground Color by setting the REG[D2h] ~REG[D4h] and Background Color by setting the REG[D5h]~REG[D7h] in advance.

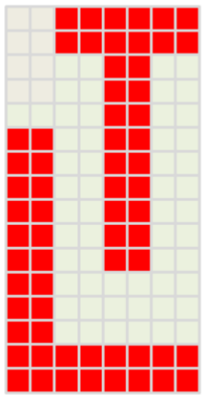
16.2.1 8*16 UCG Data Format

UCG with 8*16 size needs 16 bytes data. For example, CGRAM start address is 1000h and the first UCG encoding is 0000h, the data will be saved in 1000h~100Fh, then the second UCG encoding will be 0001h and its data will be saved in 1010h~101Fh. Below formula shows how the 8*16 UCG address in Memory is calculated, and Table 16-5 explains the data format and data byte sequence.

UCG_ADD = CGRAM_Start_ADD + (UCG_Code * 16)

Table 16-5: Data Format and Byte Sequenc of 8*16 UCG

UCG Code: 0000h		UCG Code: 0001h	
Address	Data	Address	Data
1000h	Byte0: 3Fh	1010h	Byte0
1001h	Byte1: 3Fh	1011h	Byte1
1002h	Byte2: 0Ch	1012h	Byte2
1003h	Byte3: 0Ch	1013h	Byte3
: : :	: : :	: : :	: : :
100Dh	Byte14: C0h	101Dh	Byte14
100Eh	Byte14: FFh	101Eh	Byte14
100Fh	Byte15: FFh	101Fh	Byte15



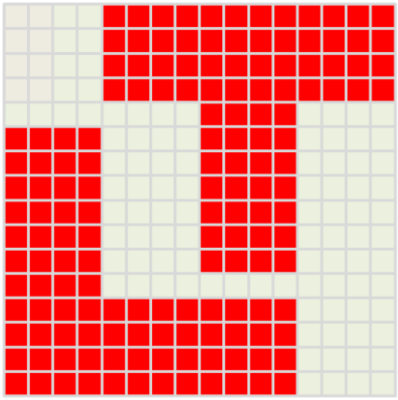
16.2.2 16*16 UCG Data Format

UCG with 16*16 size needs 32 bytes data. For example, CGRAM start address is 1000h and the first UCG encoding is 0000h, the data will be saved in 1000h~101Fh, then the second UCG encoding will be 0001h and its data will be saved in 1020h~103Fh. Below formula shows how the 16*16 UCG address in Memory is calculated, and Table 16-6 explains the data format and data byte sequence.

UCG_ADD = CGRAM_Start_ADD + (UCG_Code * 32)

Table 16-6: Data Format and Byte Sequenc of 16*16 UCG

UCG Code: 0000h			
Address	Data	Address	Data
1000h	Byte0: 0Fh	1001h	Byte1: FFh
1002h	Byte2: 0Fh	1003h	Byte3: FFh
1004h	Byte4: 0Fh	1005h	Byte5: FFh
1006h	Byte6: 0Fh	1007h	Byte7: FFh
: : :	: : :	: : :	: : :
101Ah	Byte26: FFh	101Bh	Byte27: F0h
101Ch	Byte28: FFh	101Dh	Byte29: F0h
101Eh	Byte30: FFh	101Fh	Byte31: F0h



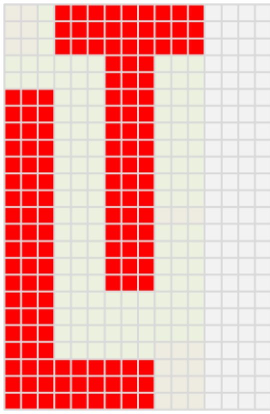
16.2.3 12*24 UCG Data Format

UCG with 12*24 size needs 48 bytes data, note that bit[3:0] of the byte with Odd Sequence Number is ignored. For example, CGRAM start address is 1000h and the first UCG encoding is 0000h, the data will be saved in 1000h~102Fh, then the second UCG encoding will be 0001h and its data will be saved in 1030h~105Fh. Below formula shows how the 12*24 UCG address in Memory is calculated, and Table 16-7 explains the data format and data byte sequence.

UCG_ADD = CGRAM_Start_ADD + (UCG_Code * 48)

Table 16-7: Data Format and Byte Sequenc of 12*24 UCG

UCG Code: 0000h			
Address	Data	Address	Data
1000h	Byte0: 1Fh	1001h	Byte1: F0h
1002h	Byte2: 1Fh	1003h	Byte3: F0h
1004h	Byte4: 1Fh	1005h	Byte5: F0h
1006h	Byte6: 03h	1007h	Byte7: 80h
: : :	: : :	: : :	: : :
102Ah	Byte42: FFh	102Bh	Byte43: 80h
102Ch	Byte44: FFh	102Dh	Byte45: 80h
102Eh	Byte46: FFh	102Fh	Byte47: 80h



16.2.4 24*24 UCG Data Format

UCG with 24*24 size needs 72 bytes data. For example, CGRAM start address is 1000h and the first UCG encoding is 0000h, the data will be saved in 1000h~1047h, then the second UCG encoding will be 0001h and its data will be saved in 1048h~108Fh. Below formula shows how the 24*24 UCG address in Memory is calculated, and Table 16-8 explains the data format and data byte sequence.

UCG_ADD = CGRAM_Start_ADD + (UCG_Code * 72)

Table 16-8: Data Format and Byte Sequenc of 24*24 UCG

UCG Code: 0000h					
Address	Data	Address	Data	Address	Data
1000h	Byte0	1001h	Byte1	1002h	Byte2
1003h	Byte3	1004h	Byte4	1005h	Byte5
1006h	Byte6	1007h	Byte7	1008h	Byte8
1009h	Byte9	100Ah	Byte10	100Bh	Byte11
:	:	:	:	:	:
103Fh	Byte63	1040h	Byte64	1041h	Byte65
1042h	Byte66	1043h	Byte67	1044h	Byte68
1045h	Byte69	1046h	Byte70	1047h	Byte71

16.2.5 16*32 UCG Data Format

UCG with 16*32 size needs 64 bytes data. For example, CGRAM start address is 1000h and the first UCG encoding is 0000h, the data will be saved in 1000h~103Fh, then the second UCG encoding will be 0001h and its data will be saved in 1040h~107Fh. Below formula shows how the 24*24 UCG address in Memory is calculated, and Table 16-9 explains the data format and data byte sequence.

UCG_ADD = CGRAM_Start_ADD + (UCG_Code * 64)

Table 16-9: Data Format and Byte Sequenc of UGC

UCG Code: 0000h			
Address	Data	Address	Data
1000h	Byte0	1001h	Byte1
1002h	Byte2	1003h	Byte3
1004h	Byte4	1005h	Byte5
1006h	Byte6	1007h	Byte7
:	:	:	:
103Ah	Byte58	103Bh	Byte59
103Ch	Byte60	103Dh	Byte61
103Eh	Byte62	103Fh	Byte63

16.2.6 32*32 UCG Data Format

UCG with 32*32 size needs 128 bytes data. For example, CGRAM start address is 1000h and the first UCG encoding is 0000h, the data will be saved in 1000h~107Fh, then the second UCG encoding will be 0001h and its data will be saved in 1080h~10FFh. Below formula shows how the 32*32 UCG address in Memory is calculated, and Table 16-10 explains the data format and data byte sequence.

UCG_ADD = CGRAM_Start_ADD + (UCG_Code * 128)

Table 16-10: Data Format and Byte Sequenc of 32*32 UCG

UCG Code: 0000h							
Address	Data	Address	Data	Address	Data	Address	Data
1000h	Byte0	1001h	Byte1	1002h	Byte2	1003h	Byte3
1004h	Byte4	1005h	Byte5	1006h	Byte6	1007h	Byte7
1008h	Byte8	1009h	Byte9	100Ah	Byte10	100Bh	Byte11
100Ch	Byte12	100Dh	Byte13	100Eh	Byte14	100Fh	Byte15
: : :	: : :	: : :	: : :	: : :	: : :	: : :	: : :
1074h	Byte116	1075h	Byte117	1076h	Byte118	1077h	Byte119
1078h	Byte120	1079h	Byte121	107Ah	Byte122	107Bh	Byte123
107Ch	Byte124	107Dh	Byte125	107Eh	Byte126	107Fh	Byte127

16.2.7 Initialize CGRAM from MCU

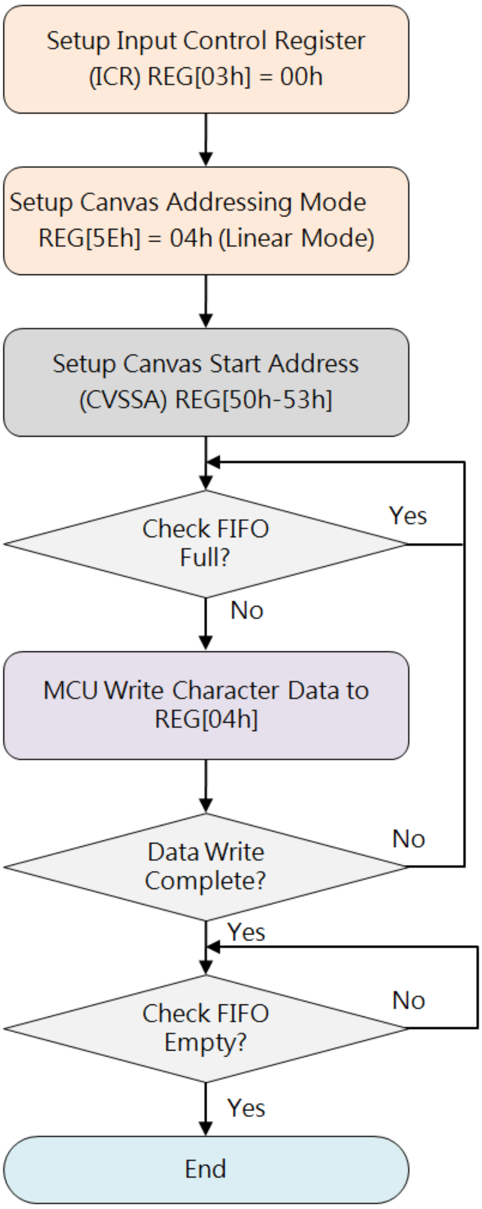


Figure 16-3: Flow Chart of CGRAM Initialization from MCU

16.2.8 Initialize CGRAM from Serial Flash

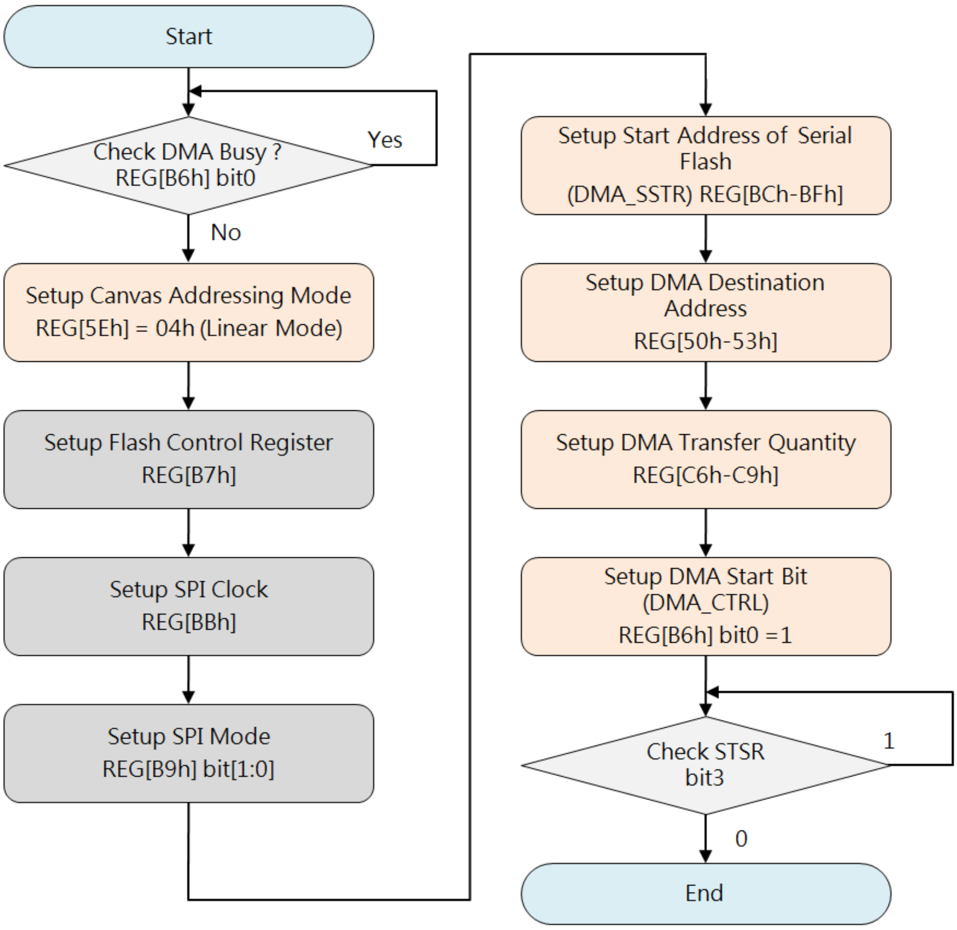


Figure 16-4: Flow Chart of CGRAM Initialization from Serial Flash

16.3 Character Rotation by 90 Degree

LT7689 supports character rotation by counterclockwise 90 degree. Normal (REG[CDh] bit4=0) text direction is from left to right then from top to bottom. If set REG[CDh] bit4=1, the character will be counterclockwise rotated by 90 degree, and flipped in vertical. Also, the text direction will be changed to from top to bottom then from left to right. However, to get the correct display result, the Display Scan Direction must be changed by setting VDIR Reg[12h] bit3=1 (Please note that Text Cursor, Graphic Cursor, and PIP will be disabled automatically under this setting). Below example shows a set of characters rotated by 90 degree:

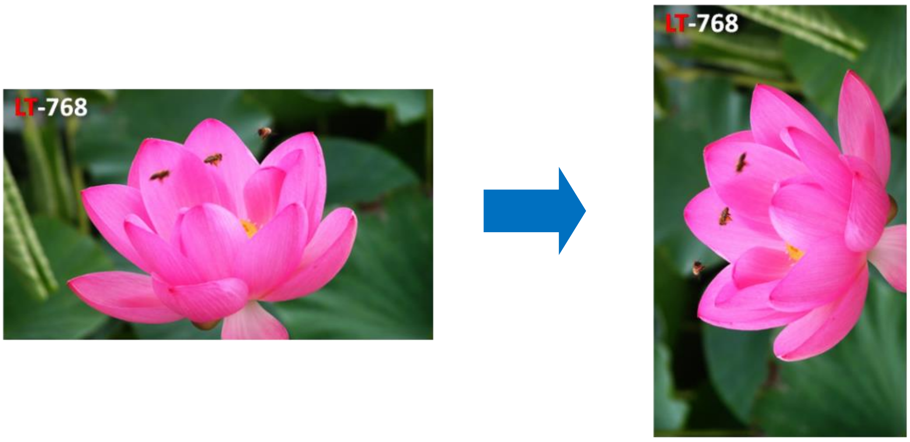


Figure 16-5: Example of Character Rotation

16.4 Size Enlargement

LT7689 supports linear *1, *2, *3, *4 character size enlargement for Height and/or Width, controlled by REG[CDh] bit[3:0].

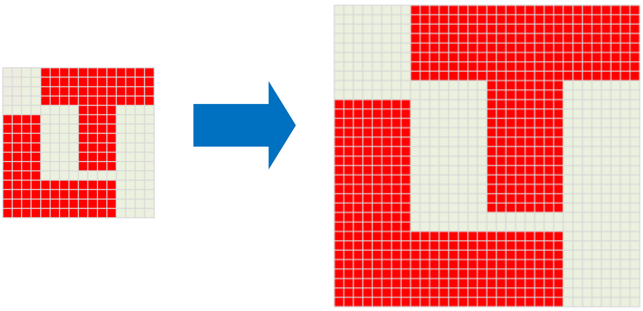


Figure 16-6: Example of Size Enlargement

16.5 Background Transparency

LT7689 supports character Background transparent, controlled by REG[CDh] bit6.

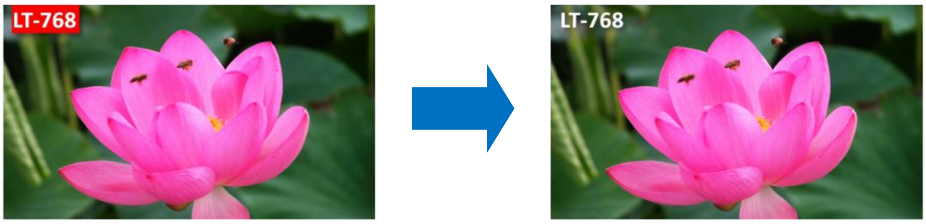


Figure 16-7: Example of Background Transparency

16.6 Character Full-Alignment

LT7689 supports character full-alignment that makes the character to align to each other. When inputting and displaying Half or Full size characters, set REG[CDh] bit7=1.

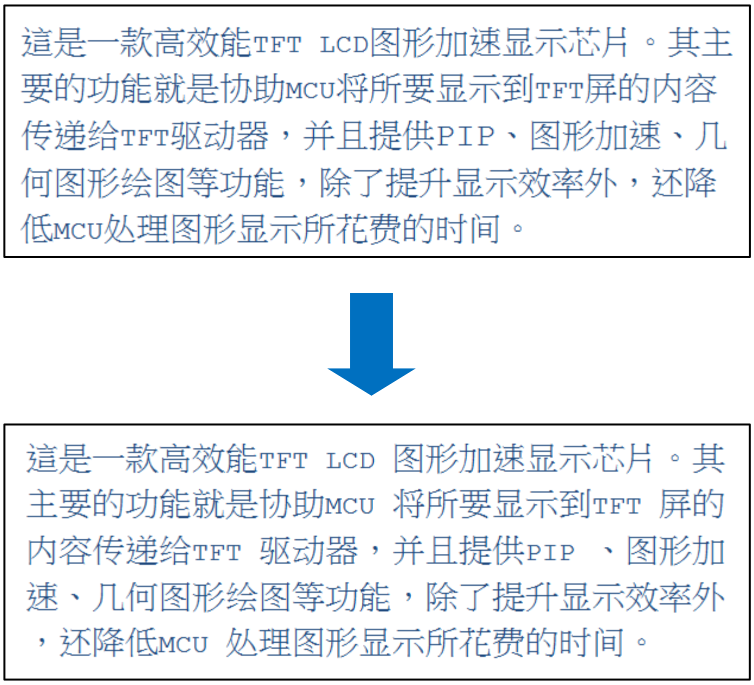


Figure 16-8: Example of Character Full-Alignment

16.7 Automatic Line Feed

LT7689 supports sequent text input and display, and can perform automatically Line Feed at active window boundary.

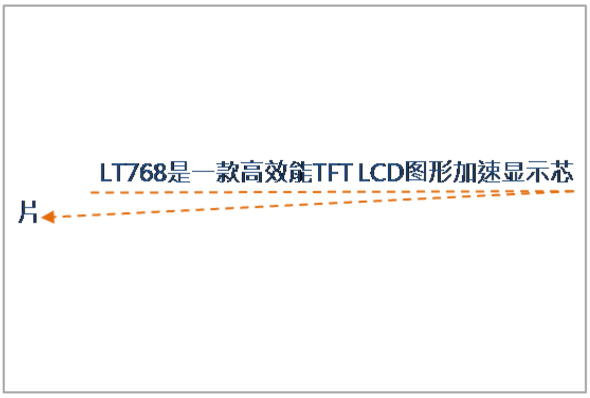


Figure 16-9: Example of Automatic Line Feed

16.8 Cursor

LT7689 supports 2 types of Cursor, Graphic Cursor and Text Cursor. The Graphic Cursor is 32*32 pixel graphic with 2-bits color index which can be displayed at an user-defined position. The Text Cursor is bit-wise graphic with 32*32 as maximum size, to point text input position. Note that when Vertical Scan Direction is set to “From bottom to Top” (VDIR REG[12h] bit3 VDIR=1), Text Cursor and Graphic Cursor as well as PIP will be disabled automatically.

16.8.1 Text Cursor

Text Cursor has Auto-move, Blinking, and Enlargement functions. Once enabled, the Text Cursor appears at the input position, and can automatically move to next input position after current input is completed. Auto-move also supports Auto Line Feed, but is dominated by Active Window. Therefore, Text Cursor must be positioned in active window and in Text Mode, moving distance and direction is the same as Character input settings.

Table 16-11: Regiaters Related with Text Cursor

Register Address	Register Name	Description
REG[03h]	ICR	bit2: Graphic/Text Mode Selection (Text Mode Enable)
REG[3Ch]	GTCCR	bit1: Text Cursor Enable
		bit0: Text Cursor Blinking Enable)
REG[64h:63h]	F_CURX	X_Position: Text Input X coordinate
REG[66h:65h]	F_CURY	Y_Position: Text Input Y coordinate
REG[D0h]	FLDR	Text Line Gap Setting

■ Text Cursor Blinking

Text Cursor Blinking can be set to enable by GTCCR (REG[3Ch]) bit[0]=1, and disable by bit[0]=0. Blinking interval can be obtained through below formula.

Blink_Time (sec) = BTCR[3Dh] * (1/Frame_Rate)

Figure 16-10: Example of Text Cursor Blinking

■ Text Cursor Height and Width

Text Cursor Height and Width are controlled by CURHS (REG[3Eh]) and CURVS (REG[3Fh]).

If Character enlargement is enabled, Text Cursor enlargement is also enabled automatically according to the same enlargement settings.

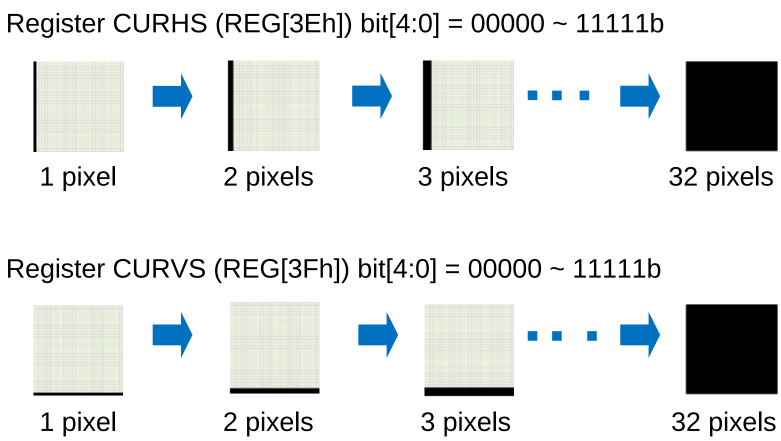


Figure 16-11: Example of Text Cursor Height and Width Settings

16.8.2 Graphic Cursor

LT7689 Graphic Cursor is 32*32 pixel graphic, and the color of each pixel is represented by 2 bits data. The color data are defined as shown in Table 16-2:

Table 16-12: Graphic Color Definition

2'b00	Color-0 (defined by REG[44h])
2'b01	Color-1 (defined by REG[45h])
2'b10	Background Color
2'b11	Inversed Background Color

To create a Graphic Cursor needs 256 bytes (32x32x2/8), Figure 16-12 shows the data format and byte sequence. LT7689 supports 4 sets of graphic cursor that users can choose from by setting GTCCR REG[3Ch] bit[3:2]. Display position of graphic cursor is controlled by register GCHP0 (REG[40h]), GCHP1(REG[41h]), GCVP0(REG[42h]) and GCVP1(REG[43h]).

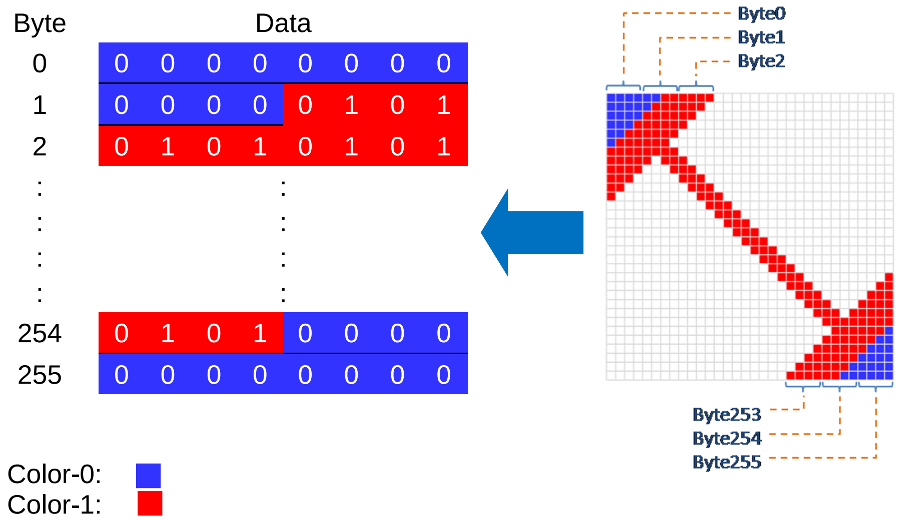


Figure 16-12: Graphic Cursor Data Format and Example

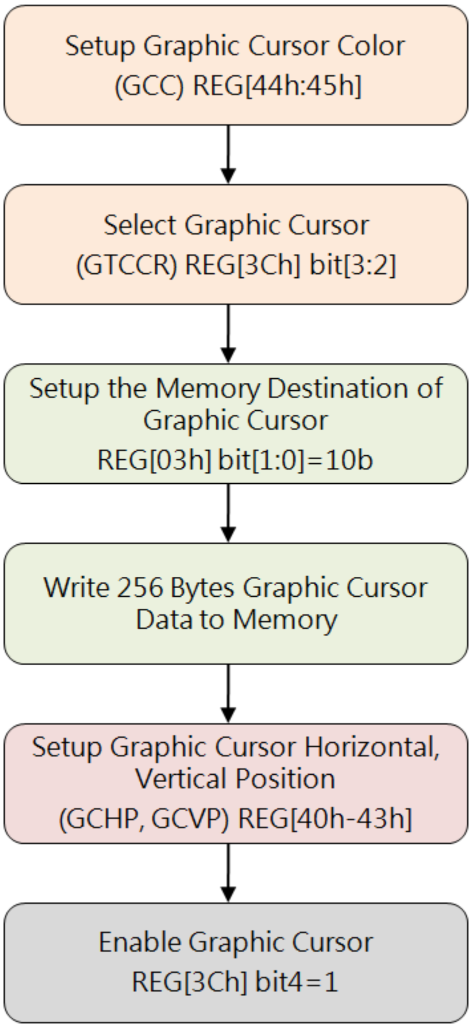


Figure 16-13: Flow Chart of Graphic Cursor Creation

17. Serial Bus Master

17.1 Power-on Display

LT7689's Power-on Display circuit has embedded a small microprocessor unit. The main function is to quickly display the screen at boot time by executing the program code stored in Flash memory in the absence of Host, or when the Host is still in its start-up phase. To use this function, PWM[0] pin must connect to a 10K pull-up resistor, then the "Power-On Display" function will be enabled(Refer to Figure17-1). When Power-on Display is enabled, LT7689 will automatically execute the program until the program code in the Flash memory is fully executed. After that, Host will retrieve the control of the system.

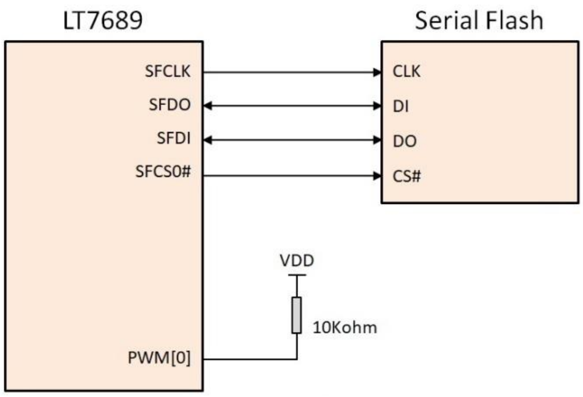


Figure 17-1: Enable the "Power-On Display" Function

The Power-on Display feature restricts that program code and display data must be stored in the same Flash memory. The Power-on Display unit supports 12 instructions as follows:

Table 17-1: 12 Power-on Display Instructions

No.	Instruction	Description	instruction Length (Byte)
1	EXIT	Exit instruction (00h/FFh)	1
2	NOP	NOP instruction (AAh)	1
3	EN4B	Enter 4-Byte mode instruction (B7h)	1
4	EX4B	Exit 4-Byte mode instruction (E9h)	1
5	STSR	Status read instruction (10h)	2
6	CMDW	Command write instruction (11h)	2
7	DATR	Data read instruction (12h)	2
8	DATW	Data write instruction (13h)	2
9	REPT	Load repeat counter instruction (20h)	2
10	ATTR	Fetch Attribute instruction (30h)	2
11	JUMP	Jump instruction (80h)	5
12	DJNZ	Decrement & Jump instruction (81h)	5

One Byte Instruction:

- **Exit instruction (EXIT) – 00h | FFh | Undefined instructions**

  EXIT instruction is executed to exit the Power-on Display function, and return control right to Host.

- **NOP instruction (NOP) – AAh**

  It will do nothing and then fetch next instruction.

- **Enter 4-Byte mode instruction (EN4B) – B7h**

  This instruction enables accessing the address length of 32-bit for the memory area of higher density (larger than 128Mb). The control unit default is in 24-bit address mode; after sending out the EN4B instruction, the address length becomes 32-bits instead of the default 24-bits. There are three ways to exit the 4-bytes mode: writing Exit 4-bytes Mode (EX4B) instruction, Hardware Reset or Power-off.

- **Exit 4-Byte mode instruction (EX4B) – E9h**

  The EX4B instruction is executed to exit the 4-bytes address mode and return to the default 3-bytes address mode. Once exiting the 4-bytes address mode, the address length will return to 24-bits.

Two Bytes Instruction:

- **Load repeat counter instruction (REPT) – 20h + param[0]**

  This instruction contains one byte parameter. This parameter is mainly for repeating counter value and used with DJNZ instruction.

- **Fetch attribute instruction (ATTR) – 30h + param[0]**

  This instruction contains one byte parameter. This instruction is used to configure controller to access serial Flash data. In Param[0]:

  _bit [3:0] is clock divisor for SPI clock. This is used to set proper SPI clock based on the system clock. Default is 0.

  Fsck = Fcore / [(Divisor + 1) * 2]

  _bit [4] is device mode selection for [CPOL, CPHA]. Value ‘0’ is mode 0, value ‘1’ is mode 3. Default is 1 for mode 3.

  _bit [5] is to define SFCS[1:0] deselect time which is also called chip select high time (tCSH) . Value ‘0’ is 4 core clocks; value ‘1’ is 8 clocks. Default is 8 core clocks.

  _bit [7:6] is dummy cycle number. These two bits set 4 kinds of dummy cycle. The Value 0, 1, 2 or 3 stands for 0, 8, 16 or 24 dummy cycles. Default is 0. If dummy cycle number is 0 then serial flash read command code is 03h, otherwise serial flash read command code is 0Bh.

- **Status read instruction (STSR) – 10h + param[0]**

  The parameter represents the expected value in Status read **instruction**. If the returned data does not match with expected value, this read instruction will be repeated.

- **Command write instruction (CMDW) – 11h + param[0]**

  The parameter represents the value that is to be written through CMDW instruction.

- **Data read instruction (DATR) – 12h + param[0]**

  The parameter represents an expected value. After issuing DATR, if the returned data does not match with the expected value, the instruction will be exceuted repeatedly.

- **Data write instruction (DATW) – 13h + param[0]**
  The parameter represents the written value by Data write instruction.

Five Bytes Instruction:

- **Jump instruction (JUMP) – 80h + param[3] + param[2] + param[1] + param[0]**
  It contains 4-bytes parameters. Parameter [3]~[0] are serial flash's 28-bits address information. For example, param[3] is address[27:24], param[2] is address[23:16], param[1] is address[15:8], and param[0] is address[7:0]. After execution, next instruction will be fetched from this specified address.

- **Decrement & Jump while not equal to zero (DJNZ) instruction – 81h + param[3] + param[2] + param[1] + param[0]**
  Parameter 3~0 are serial flash's 28-bits address information. i.e. param[3] is address[27:24], param[2] is address[23:16], param[1] is address[15:8] and param[0] is address[7:0]. If the repeat counter equals to zero, then the next instruction will be fetched from "current instruction address + 5", otherwise the repeat counter will decrement one and jump to specified address.

After the Power-on Reset, the Power-on Display function will serach the two SPI interfaces provided by LT7689. The first 8 bytes must be "61h, 72h, 77h, 63h, 77h, 62h, 78h, 67h". If the flash memory is identified then the subsequent processing address will be (0008h), otherwise the master control will be transferred to the Host. LT7689 internal microprocessor will start to execute instructions from the address 0008h of the flash memory. During the execution of Power-on Display, if an "Exit" or undefined instruction is executed, the processor will hand over the control to the host. Here is an example of using an external serial Flash and show a 1024*768 picture on an 1024*768 resolution TFT panel:

```
// Addr: 'h0000
61  72  77  63  77  62  78  67     // ID

//Initial PLL
11  05  13  8A      // REG_WR('h05, 'h8A), Write 0x8A to REG[05]
11  06  13  41      // REG_WR('h06, 'h41)
11  07  13  8A      // REG_WR('h07, 'h8A)
11  08  13  64      // REG_WR('h08 'h64)
11  09  13  8A      // REG_WR('h09,'h8A)
11  0A  13  64      // REG_WR('h0A, 'h64)
11  00  13  80      // REG_WR('h00, 'h80)
11  01  13  82      // REG_WR('h01, 'h82)
11  01  12  82      // REG_WR('h01, 'h82)
11  02  13  40      // REG_WR('h02, 'h40)
AA  AA  AA  AA      // NOP

//Initial Display RAM
11  E0  13  29      // REG_WR('hE0, 'h29)
11  E1  13  03      // REG_WR('hE1, 'h03)
11  E2  13  0B      // REG_WR('hE2, 'h0B)
11  E3  13  03      // REG_WR('hE3, 'h03)
11  E4  13  01      // REG_WR('hE4, 'h01)
AA  AA  AA  AA      // NOP

//Setup LCD Panel
11  10  13  04      // REG_WR('h10, 'h04)
11  12  13  85      // REG_WR('h12, 'h85)
11  13  13  03      // REG_WR('h13, 'h03)
11  14  13  7F      // REG_WR('h14, 'h7F)
11  15  13  00      // REG_WR('h15, 'h00)
11  1A  13  FF      // REG_WR('h1A, 'hFF)
11  1B  13  02      // REG_WR('h1B, 'h02)
```

```
// Setup Main Window
11   20   13   00          // REG_WR('h20, 'h00)
11   21   13   00          // REG_WR('h21, 'h00)
11   22   13   00          // REG_WR('h22, 'h00)
11   23   13   00          // REG_WR('h23, 'h00)
11   24   13   00          // REG_WR('h24, 'h00)
11   25   13   04          // REG_WR('h25, 'h04)
11   26   13   00          // REG_WR('h26, 'h00)
11   27   13   00          // REG_WR('h27, 'h00)
11   28   13   00          // REG_WR('h28, 'h00)
11   29   13   00          // REG_WR('h29, 'h00)
AA   AA   AA   AA          // NOP

//Setup Canvas Window
11   50   13   00          // REG_WR('h50, 'h00)
11   51   13   00          // REG_WR('h51, 'h00)
11   52   13   00          // REG_WR('h52, 'h00)
11   53   13   00          // REG_WR('h53, 'h00)
11   54   13   00          // REG_WR('h54, 'h00)
11   55   13   04          // REG_WR('h55, 'h04)

//Setup Active Window
11   56   13   00          // REG_WR('h56, 'h00)
11   57   13   00          // REG_WR('h57, 'h00)
11   58   13   00          // REG_WR('h58, 'h00)
11   59   13   00          // REG_WR('h59, 'h00)
11   5A   13   00          // REG_WR('h5A, 'h00)
11   5B   13   04          // REG_WR('h5B, 'h04)
11   5C   13   00          // REG_WR('h5C, 'h00)
11   5D   13   03          // REG_WR('h5D, 'h03)
11   5E   13   02          // REG_WR('h5E, 'h02)

//Setup DMA Transfer Data from Flash to Display RAM
11   BC   13   00          // REG_WR('hBC, 'h00)
11   BD   13   02          // REG_WR('hBD, 'h02)
11   BE   13   00          // REG_WR('hBE, 'h00)
11   BF   13   00          // REG_WR('hBF, 'h00)
11   C0   13   00          // REG_WR('hC0,'h00)
11   C1   13   00          // REG_WR('hC1 ,'h00)
11   C2   13   00          // REG_WR('hC2, 'h00)
11   C3   13   00          // REG_WR('hC3, 'h00)
11   C6   13   00          // REG_WR('hC6, 'h00)
11   C7   13   04          // REG_WR('hC7, 'h04)
11   C8   13   00          // REG_WR('hC8, 'h00)
11   C9   13   03          // REG_WR('hC9, 'h03)
11   CA   13   00          // REG_WR('hCA, 'h00)
11   CB   13   04          // REG_WR('hCB, 'h04)
11   B7   13   C0          // REG_WR('hB7, 'hC0)
11   B6   13   01          // REG_WR('hB6, 'h01)
AA   AA   AA   AA          // NOP
11   B6   12   00          // REG_WR('hB6, 'h00)
11   12   13   40          // REG_WR('h12, 'h40)
00                         // Exit
```

Note: Power-on display unit requests program codes & display data, fonts, and required contents for the program must be stored in the same serial Flash. If Host needs to switch to another serial Flash then all the codes & display data etc. will need to be stored to that serial Flash.

17.2 SPI Master

When transferring data through SPI master of LT7689, the two serial data lines will be sampled and synchronized with the serial clock signal SFCLK. Master will place the relevant information on the SFDO signal line for the slave device to catch data in the first half of the clock edge. There are four possible protocol modes of CPOL and CPHA that can be choosen by setting the bit[1:0] of Serial Master Control Register [SPIMCR2]. The master and slave device must operate under the same frequency.

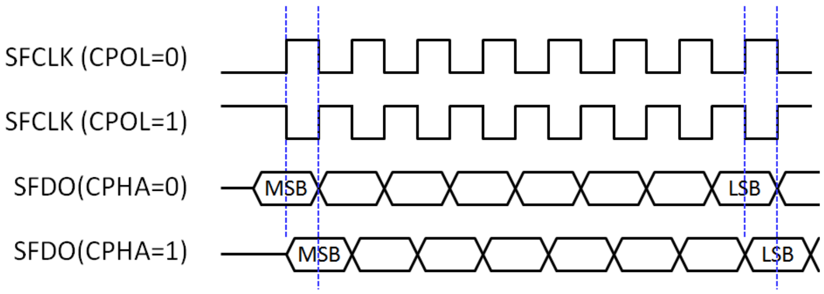


Figure 17-2: Master SPI Data Transfer

Transmitting Data Bytes

The SPI transmission can be initialized after the control register is setup. The method to initialize data is to write the data to the Register SPIDR (REG[B8h]). The data written to SPDIR is actually written to a FIFO which has 16 bytes depths, also known as "Write-FIFO". Each write access adds a data byte to the Write-FIFO. When the SS_Active is set to 1 and the FIFO is not empty, LT7689 will start to transfer the data that first written into "Write-FIFO" to Slave.

Receiving Data Bytes

Receiving data is done simultaneously with transmitting data; whenever a data byte is transmitted, a data byte is received. For each byte that needs to be read from a device, a dummy byte has to be written to the Write FIFO. This means using a dummy write cycle to activate the SPI while receiving the data. Whenever the transfer is done, the received data is written in the "Read-FIFO". The "Read-FIFO" is the counterpart of the Write-FIFO. It is an independent 16 bytes deep FIFO. The FIFO contents can be read by reading from the Register [SPIDR] (REG[B8h]).

FIFO Overrun

Both "Write-FIFO" and "Read-FIFO" are using Circular Memories to simulate a big RAM.
When FIFO is full, writing a new data into "Write-FIFO" will overwrite the oldest data. When writing data to "Write-FIFO" via [SPIDR] registers causes overflow, it will result in data errors. Then the data transmitted using the SPI interface will not be the first data entered, but the data that finally enters the FIFO. As shown in the example of Figure 17-3, WP is the write pointer, when "Write-FIFO" is full, and then next data written in FIFO will overwrite the first one. RP is the read pointer If "Read-FIFO" is not performed before "Write-FIFO" is overflow, RP will stay at the first place. Therefore, once the overflow occurs, "Read-FIFO" will get wrong data.

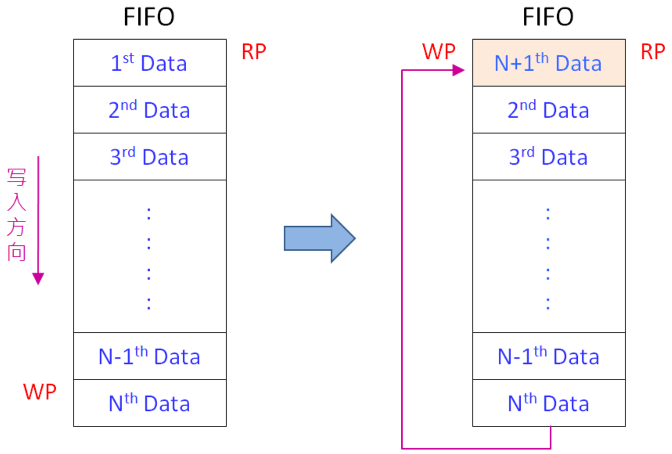


Figure 17-3: FIFO Overrun Example

The only way to recover from this situation is to reset the Write Buffer. Both the Read FIFO and the Write FIFO are reset when the Slave Select signal active [SS_ACTIVE] bit is cleared (‘0’). Read FIFO overruns might be less destructive. Especially when the SPI bus is used to transmit data only, and the received data is simply ignored. So the Read FIFO overruns is irrelevant. But, if the SPI bus is used to transmit and receive data, it is important to keep the Read FIFO aligned. The easiest way to do this is to perform a number of dummy reads. The number of dummy reads should be equal to the amount of bytes transmitted modulo 16.

$$N_{dummy\_reads} = N_{transmitted\_bytes} \bmod 16$$

Note: If the "Read-FIFO" is not empty, storing 16 bytes of data is bound to cause overwritten. Therefore, before receiving every 16 bytes of data, the host must confirm if “Read-FIFO” is empty or not.

Reference Code for SPI Master Loop Test (Connect SFDO to SFDI)

```
REG_WR ('hBB, 8'h1f) ;              //Divisor, configure SPI clock frequency
REG_WR ('hB9, 8'b0001_1111) ;       // {1'b0, mask, SS#_sel, ss_active, ovfirqen, emtirqen,
                                    // cpol, cpha}, SS# low
REG_WR ('hB8, 8'h55) ;              // TX
REG_WR ('hB8, 8'haa) ;              // TX
REG_WR ('hB8, 8'h87) ;              // TX
REG_WR ('hB8, 8'h78) ;              // TX
wait (INT#) ;
REG_RD ('hBA, acc) ;
while (acc != 8'h84) begin
    $display ("wait for FIFO empty ...") ;
    REG_RD ('hBA, acc) ;
end
REG_WR ('hBA, 8'h04) ;              // clear interrupt flag
REG_RD ('hB8, 8'h55) ;              // RX
REG_RD ('hB8, 8'haa) ;              // RX
REG_RD ('hB8, 8'h87) ;              // RX
REG_RD ('hB8, 8'h78) ;              // RX
REG_WR ('hB9, 8'b0000_1111) ;       // {1'b0, mask, SS#_sel, ss_active, ovfirqen, emtirqen,
                                    // cpol, cpha}, SS# high.
```


17.3 Serial Flash Controller

LT7689 builds in a SPI Master interface for Serial Flash/ROM, supporting for protocol of 4-BUS (Normal Read), 5-BUS (FAST Read), Dual mode 0, Dual mode 1 and Mode 0/Mode 3. Serial Flash/ROM function can be used for FONT mode and DMA mode. FONT mode means that the external serial Flash/ROM is treated as a source of character bitmap. DMA mode means that the external Flash/ROM is treated as the data source of DMA (Direct Memory Access). Host can speed up the data transfer to display memory, and does not need to intervene in this mode.

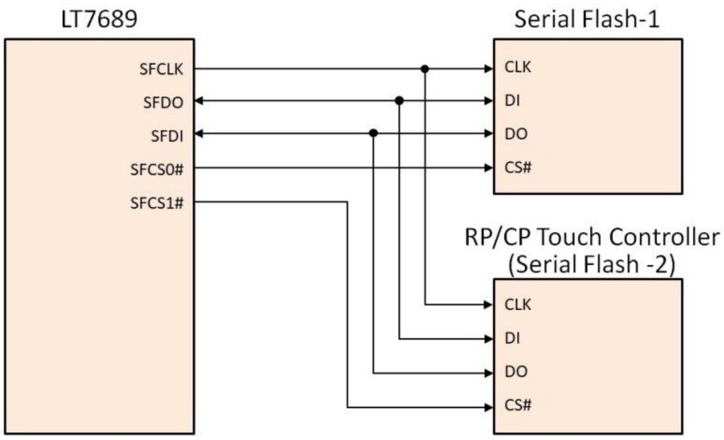


Figure 17-4: Application Circuit of Serial Flash/ROM

About Serial Flash/ROM read command protocol setting, please refer to Table 17-2 as below:

Table 17-2: Read Command Protocol of SPI Flash

REG [B7h] Bit[3:0]	Read Command Code
000xb	**1x Read Command Code – 03h** Normal read speed. Flash to LT7689 data input is SFDI pin. Without dummy cycle between address and data.
010xb	**1x Read Command Code – 0Bh** Fast read speed. Flash to LT7689 data input is SFDI pin. Eight dummy cycles inserted between address and data.
1x0xb	**1x Read Command Code – 1Bh** High speed read. Flash to LT7689 data input is SFDI pin. Sixteen dummy cycles inserted between address and data.
xx10b	**2x Read Command Code – 3Bh** Interleaved data input on SFDI and SFDO pins. Eight dummy cycles inserted between address and data phase. (Mode 0)
xx11b	**2x Read Command Code – BBh** Address output & data input interleaved on SFDI and SFDO pins. Four dummy cycles inserted between address and data phase. (Mode 1)

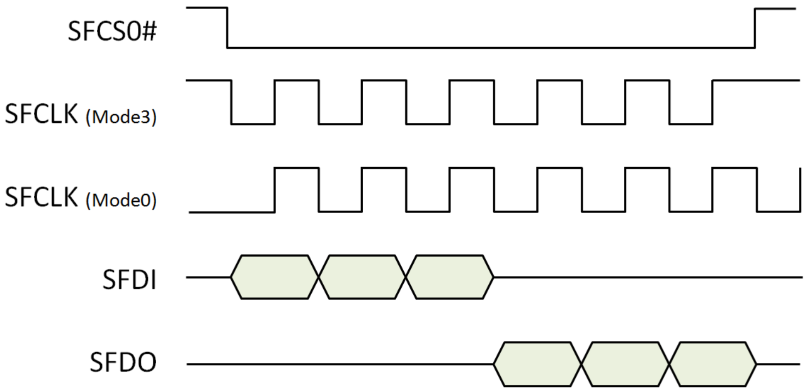


Figure 17-5: Mode 0 and Mode 3 Protocol

Figure 17-6 is a Timing diagram of Read Commands for SPI Flash Memory. If REG[B7h] bit5=0, the address line is 24bits and requires 24 clock. If REG[B7h] bit5=1, the address line is 32bits and requires 32 clock.

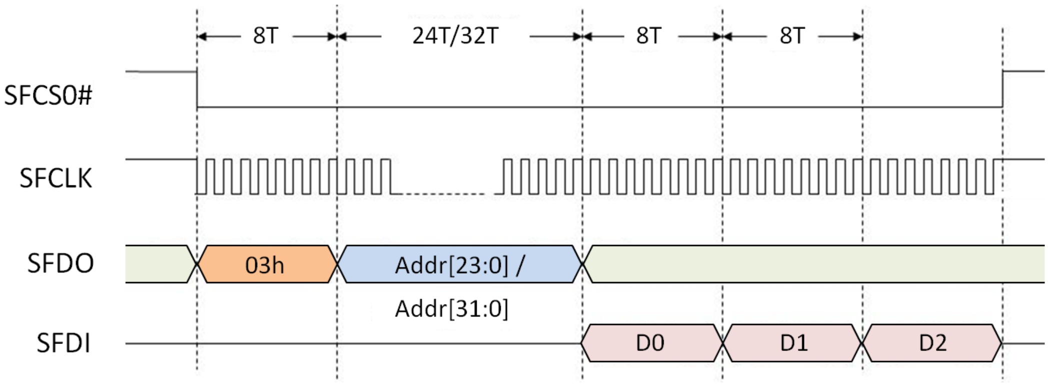


Figure 17-6: Normal Read Command of SPI Flash

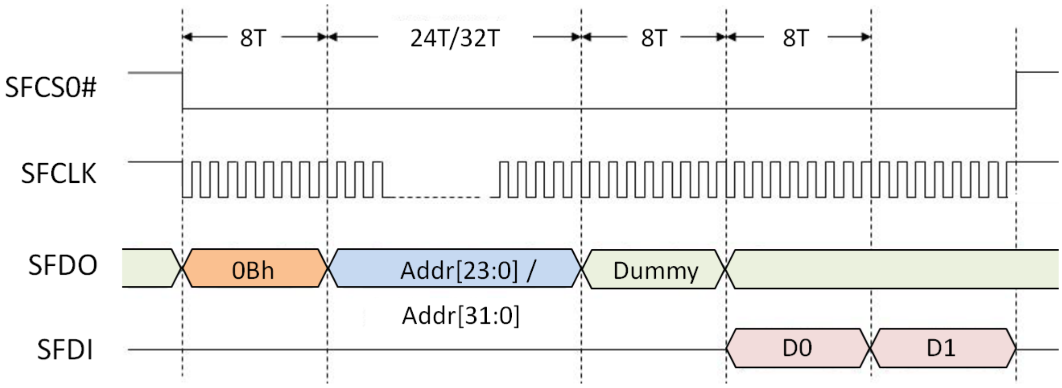


Figure 17-7: SPI Fast Read Command of SPI Flash

Figure 17-8 shows flash to LT7689 data input as interleaved input, and data input appears on SFDI and SFDO pins. If REG[B7h] bit5=0 represents an address line of 24bits, it requires 24 clocks. If REG[B7h] bit5=1 represents an address line of 32bits, it requires 32 clocks.

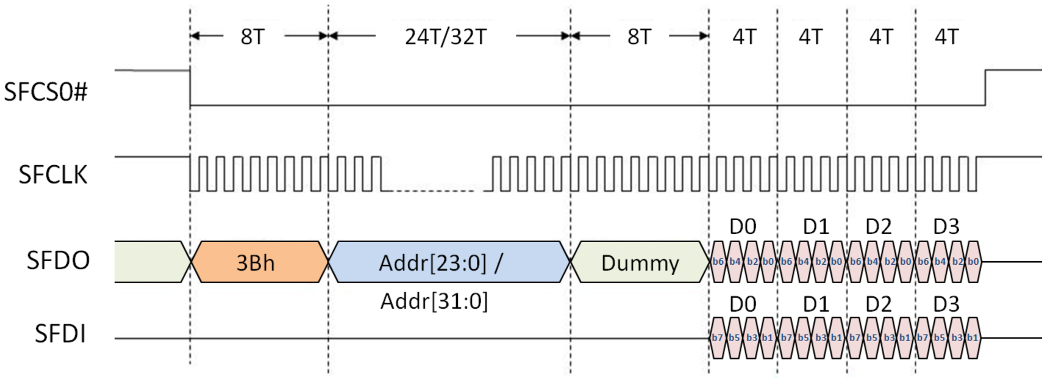


Figure 17-8: SPI Flash Read Command (Data and Address are Interleaved)

Figure 17-9 shows LT7689 address output and data input are interlaced, data and address are interleaved on SFDI and SFDO pins. If REG[B7h] bit5=0, it means the address line is 24bits, because the input is interleaved, only 12 clocks are required. If REG[B7h] bit5=1, it represents an address line of 32bits, only 16 clock are required.

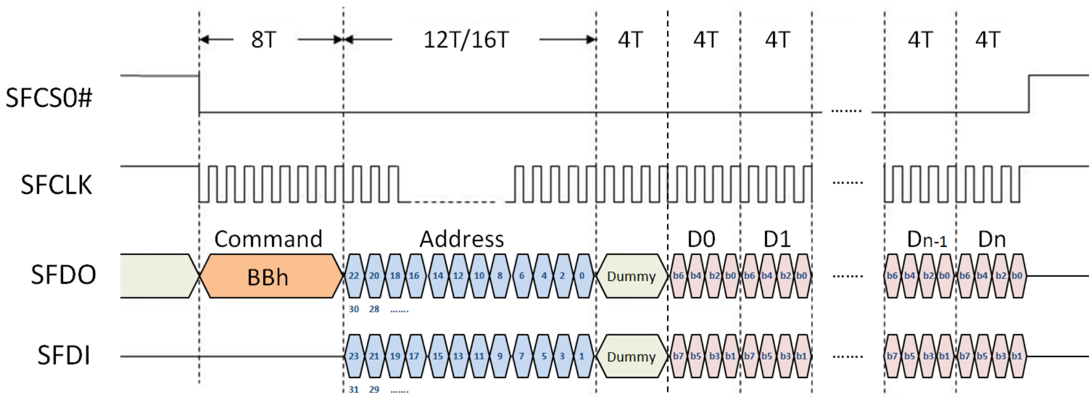


Figure 17-9: SPI Flash Read Command (Data and Address are Interleaved)

17.3.1 External Serial Flash

External Flash Memory can be used as an source of the image data. In Graphic Mode, Host can use DMA (Direct Memory access) mode to access data of External Flash Memory. It means the Flash Memory can be used as the source of DMA, and developer can store a large amount of display data in advance. Therefore Host does not need to take a lot of time to transfer large and commonly used graphic data to LT7689's Display RAM.

The data format of external Serial Flash Memory must be consistent with the format of the Display RAM. The graphic data format for Flash Memory are as follows:

Table 17-3: 8bpp Image Data Format of Serial Flash

Addr	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Addr	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0001h	R_1^7	R_1^6	R_1^5	G_1^7	G_1^6	G_1^5	B_1^7	B_1^6	0000h	R_0^7	R_0^6	R_0^5	G_0^7	G_0^6	G_0^5	B_0^7	B_0^6
0003h	R_3^7	R_3^6	R_3^5	G_3^7	G_3^6	G_3^5	B_3^7	B_3^6	0002h	R_2^7	R_2^6	R_2^5	G_2^7	G_2^6	G_2^5	B_2^7	B_2^6
0005h	R_5^7	R_5^6	R_5^5	G_5^7	G_5^6	G_5^5	B_5^7	B_5^6	0004h	R_4^7	R_4^6	R_4^5	G_4^7	G_4^6	G_4^5	B_4^7	B_4^6
0007h	R_7^7	R_7^6	R_7^5	G_7^7	G_7^6	G_7^5	B_7^7	B_7^6	0006h	R_6^7	R_6^6	R_6^5	G_6^7	G_6^6	G_6^5	B_6^7	B_6^6
0009h	R_9^7	R_9^6	R_9^5	G_9^7	G_9^6	G_9^5	B_9^7	B_9^6	0008h	R_8^7	R_8^6	R_8^5	G_8^7	G_8^6	G_8^5	B_8^7	B_8^6
000Bh	R_{11}^7	R_{11}^6	R_{11}^5	G_{11}^7	G_{11}^6	G_{11}^5	B_{11}^7	B_{11}^6	000Ah	R_{10}^7	R_{10}^6	R_{10}^5	G_{10}^7	G_{10}^6	G_{10}^5	B_{10}^7	B_{10}^6

Table 17-4: 16bpp Image Data Format of Serial Flash

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Addr	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	G_0^7	G_0^6	G_0^5	0000h	G_0^4	G_0^3	G_0^2	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3
2	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	G_1^7	G_1^6	G_1^5	0002h	G_1^4	G_1^3	G_1^2	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3
3	R_2^7	R_2^6	R_2^5	R_2^4	R_2^3	G_2^7	G_2^6	G_2^5	0004h	G_2^4	G_2^3	G_2^2	B_2^7	B_2^6	B_2^5	B_2^4	B_2^3
4	R_3^7	R_3^6	R_3^5	R_3^4	R_3^3	G_3^7	G_3^6	G_3^5	0006h	G_3^4	G_3^3	G_3^2	B_3^7	B_3^6	B_3^5	B_3^4	B_3^3
5	R_4^7	R_4^6	R_4^5	R_4^4	R_4^3	G_4^7	G_4^6	G_4^5	0008h	G_4^4	G_4^3	G_4^2	B_4^7	B_4^6	B_4^5	B_4^4	B_4^3
6	R_5^7	R_5^6	R_5^5	R_5^4	R_5^3	G_5^7	G_5^6	G_5^5	000Ah	G_5^4	G_5^3	G_5^2	B_5^7	B_5^6	B_5^5	B_5^4	B_5^3

Table 17-5: 24bpp Image Data Format of Serial Flash

Order	Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Addr	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1	G_0^7	G_0^6	G_0^5	G_0^4	G_0^3	G_0^2	G_0^1	G_0^0	0000h	B_0^7	B_0^6	B_0^5	B_0^4	B_0^3	B_0^2	B_0^1	B_0^0
2	B_1^7	B_1^6	B_1^5	B_1^4	B_1^3	B_1^2	B_1^1	B_1^0	0002h	R_0^7	R_0^6	R_0^5	R_0^4	R_0^3	R_0^2	R_0^1	R_0^0
3	R_1^7	R_1^6	R_1^5	R_1^4	R_1^3	R_1^2	R_1^1	R_1^0	0004h	G_1^7	G_1^6	G_1^5	G_1^4	G_1^3	G_1^2	G_1^1	G_1^0
4	G_2^7	G_2^6	G_2^5	G_2^4	G_2^3	G_2^2	G_2^1	G_2^0	0006h	B_2^7	B_2^6	B_2^5	B_2^4	B_2^3	B_2^2	B_2^1	B_2^0
5	B_3^7	B_3^6	B_3^5	B_3^4	B_3^3	B_3^2	B_3^1	B_3^0	0008h	R_2^7	R_2^6	R_2^5	R_2^4	R_2^3	R_2^2	R_2^1	R_2^0
6	R_3^7	R_3^6	R_3^5	R_3^4	R_3^3	R_3^2	R_3^1	R_3^0	000Ah	G_3^7	G_3^6	G_3^5	G_3^4	G_3^3	G_3^2	G_3^1	G_3^0

DMA function provides a faster method for users to update/transfer mass data to display memory. The only source of DMA function in LT7689 is external Serial Flash/ROM. There are two kinds of data type defined for the DMA. One is Linear Mode and the other is Block Mode. It provides a flexible selection for user applications. The destination of DMA function is dominated by active window in display memory. When DMA function is active, the specific data from Serial Flash/ROM will be transferred one by one to Display Memory by LT7689 automatically. After the DMA function is completed, LT7689 will generate an interrupt to notify the Host. Please refer to following sections for the detail operation.

17.3.2 DMA in Linear Mode for External Serial Flash

The DMA Linear Mode is used to send CGRAM data to Display RAM. The color depth of Active window must be set as 8bpp.

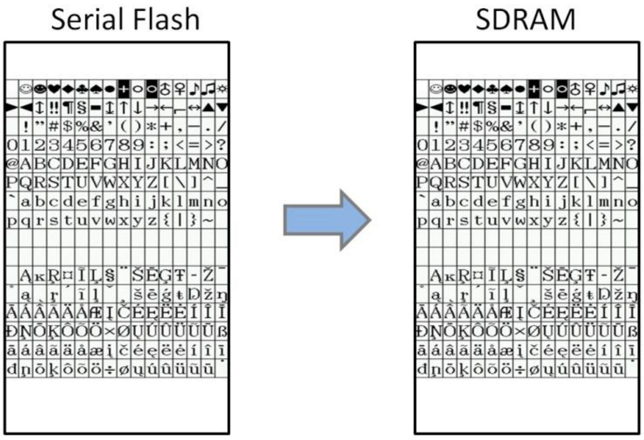


Figure 17-10: DMA in Linear Mode for External Serial Flash

17.3.3 DMA in Block Mode for External Serial Flash

DMA access in this block mode is primarily used to transfer graphic data. The process unit is pixel. Please refer to following diagram and flowchart.

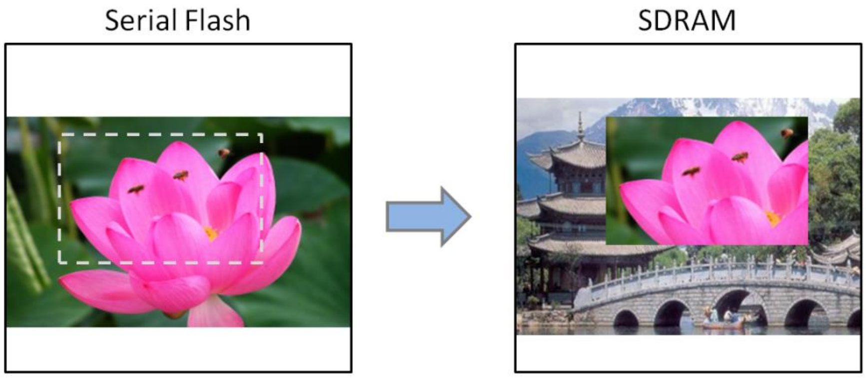


Figure 17-11: DMA in Block Mode for External Serial Flash

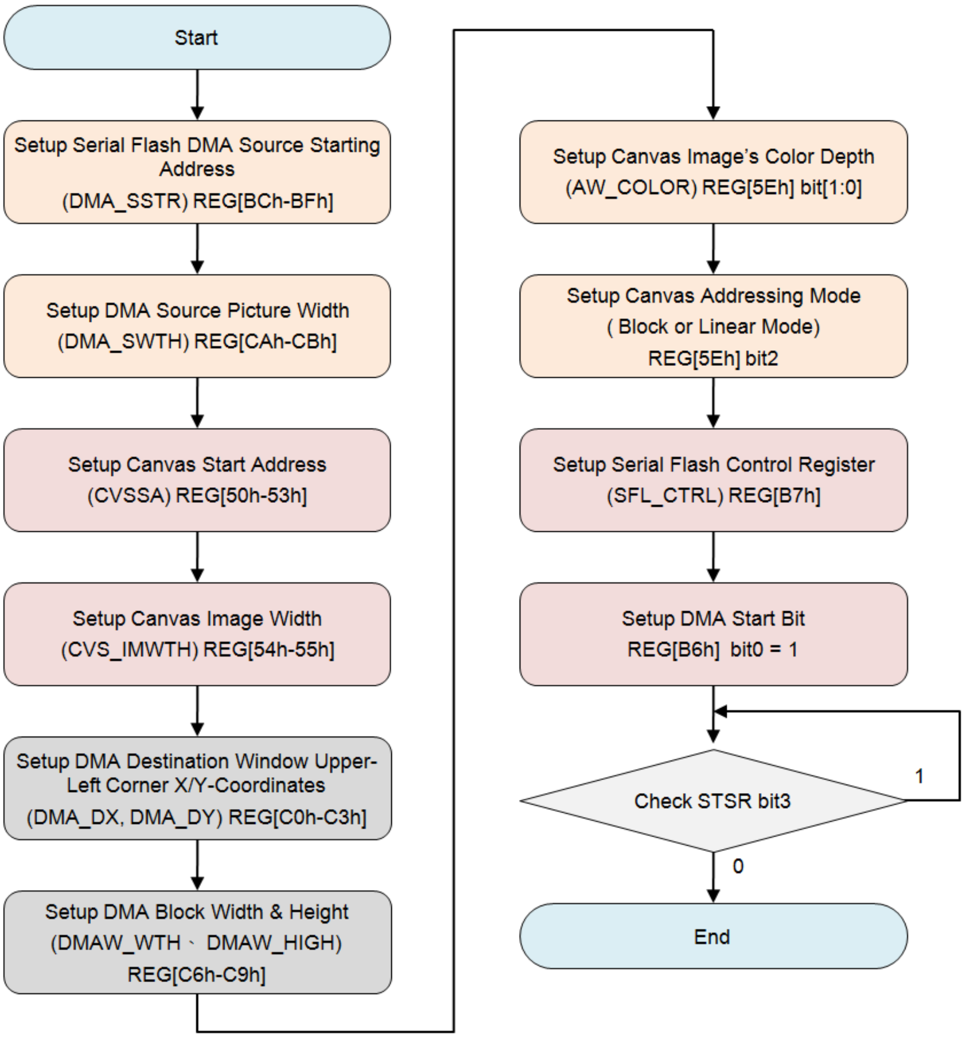


Figure 17-12: DMA Data Transfer Flowchart (Polling Mode)

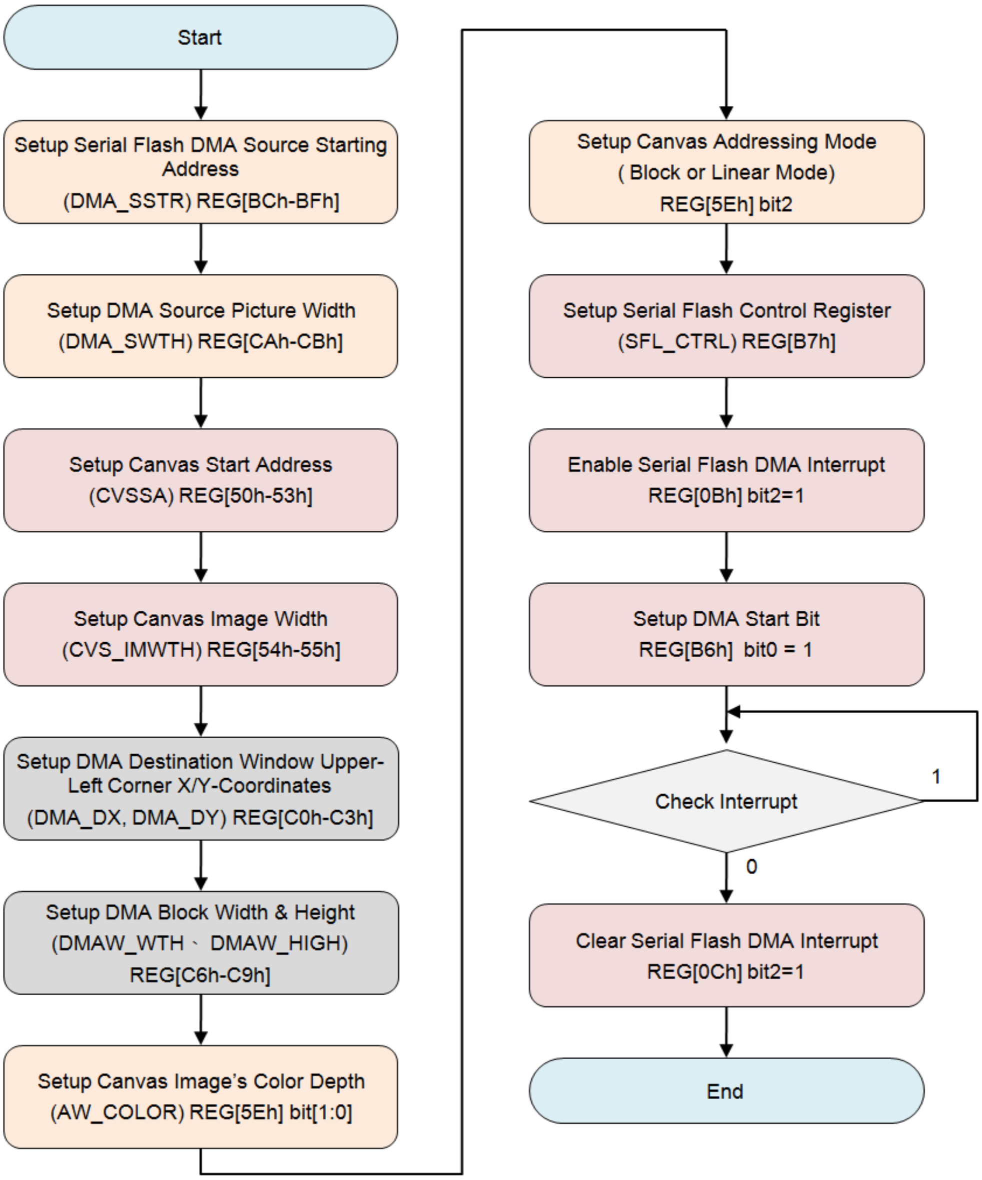


Figure 17-13: DMA Data Transfer Flowchart (Interrupt Mode)

18. Power Management

In the management mode of power supply, LT7689 has four operation modes, distinguished by the amount of power consumption, from high to Low: Normal mode, Standby mode, Suspend mode, and Sleep mode. The four modes of operation are set by the register REG[DFh]. Following is the comparison table for the related clock action of the four operation modes

Table 18-1: Power Management vs. Clock Active

Item	Normal Mode	Standby Mode	Suspend Mode	Sleep Mode
	PLL Enable	Serial MCU	Serial MCU	Serial MCU
MCLK	MPLL Clock	MPLL Clock	OSC	Stop
CCLK	CPLL Clock	OSC	OSC	OSC
PCLK	PPLL Clock	Stop	Stop	Stop
CPLL	ON	ON	OFF	OFF
MPLL	ON	ON	OFF	OFF
PPLL	ON	ON	OFF	OFF

Note:

1. When LT7689 enters the power saving mode, the LCD interface will not output the signal. Therefore, before entering the power-saving mode, Host needs to issue “display off” or “power off” to the LCD module to avoid LCD polarization damage.
2. The “OSC” means external X’tal oscillator.

18.1 Normal Mode

In this mode, three of the internal PLL functions are working normally. That is, Host setup Registers to control the three PLL to generate CCLK (Core Clock), MCLK (Display RAM Clock) and PCLK (LCD scan Clock) so that the whole system can work normally. Because it will take some time for the PLL to work steadily, Host must first check if the PLL frequency is in a stable state through register 01h bit7.

18.2 Standby Mode

If setup REG[DFh] bit[1:0] to 01b, LT7689 will enter Standby mode. The System Clock (CCLK) and LCD-Scan Clock (PCLK) will stop. The Display RAM Clock will remain active and still be provided by MCLK.

Enter Standby Mode:

- Select Standby Mode (REG[DFh] bit[1:0] = 01b)
- Setup REG[DFh] bit7 to 1 (Enter Standby Mode)
- Host may check the bit1 of Status Register (STSR), and wait for this bit to become 1 to make sure that LT7689 enters Standby mode.

Back to Normal Mode:

- Setup REG[DFh] bit7 to 0 (Quit from Standby Mode)
- Host may check the bit1 of Status Register (STSR), and wait for this bit to become 0 to make sure that LT7689 is back to Normal Mode.

18.3 Suspend Mode

If setup REG[DFh] bit[1:0] to 10b, LT7689 will enter Suspend mode. The System Clock (CCLK), Display RAM Clock(MCLK) and LCD-Scan Clock (PCLK) will stop. The clock of Display RAM will be provided by OSC Clock.

Enter Suspend Mode:

- Setup appropriate Display RAM(SDRAM) Refresh Clock based on the OSC frequency.
- Select Suspend Mode(REG[DFh] bit[1:0] = 10b)
- Setup REG[DFh] bit7 to 1 (Enter Suspend Mode)
- Host may check the bit1 of Status Register (STSR), and wait for this bit to become 1 to make sure LT7689 enters Suspend Mode.

Back to Normal Mode:

- Setup REG[DFh] bit7 to 0 (Quit from Suspend Mode)
- Host may check the bit1 of Status Register (STSR), and wait for this bit to become 0 to make sure LT7689 is back to Normal Mode.

18.4 Sleep Mode

If setup REG[DFh] bit[1:0] to 11b, LT7689 will enter Sleep Mode. And all of Clocks and PLL will stop operating.

Enter Sleep Mode:

- Select Sleep Mode (REG[DFh] bit[1:0] = 11b)
- Setup REG[DFh] bit7 to 1 (Enter Sleep Mode)
- If REG[E0h] bit7 is 0, LT7689 will enter Sleep mode, and the Display RAM will enter Power Down mode. If REG[E0h] bit7 is 1, the Display RAM will enter Refresh mode.
- If Host interface is Serial Mode, then OSC will not be stopped.
- Host may check the bit1 of Status Register (STSR), and wait for this bit to become 1 to make sure LT7689 enters Sleep Mode.

Back to Normal Mode:

- Setup REG[DFh] bit7 to 0 (Quit from Sleep Mode)
- If OSC is stopped in Sleep mode, the Host has to enable OSC.
- Host may check the bit1 of Status Register (STSR), and wait for this bit to become 0 to make sure LT7689 is back to Normal Mode.

19. Register Description

LT7689 provides a compatible 4 forms of Host interface, and the internal registers are read and written through these Host interface cycles to complete. LT7689 contains a Status Register and many Instruction Registers. Status Registers can read data through the state reading period, and it can only be read and cannot be written. Instruction Registers can control most of the functions through the "Command Write" cycle and the "Data Write" cycle. During “Command Write” cycle, the address of the register is specified, and then during the “Data Wirte” cycle, data can be wriiten to the specified register. When a specified register data is to be read, the master will need to send the "Command write" cycle first. Then use the "Data Read" cycle to read the data. In other words, "Command Write" is to set register address, "Data Read" is to read register data.

Table 19-1: Host Interface Cycle

Host Interface Cycle	CS#	A0	8080 Type MCU		6800 Type MCU		Action Description
			RD#	WR#	RD#	WR#	
			EN	RW#	EN	RW#	
Command Write	0	0	1	0	1	0	Write Address of Register
Status Read	0	0	0	1	1	1	Read Status Register Data
Data Write	0	1	1	0	1	0	Write Data to Register or Memory
Data Read	0	1	0	1	1	1	Read Data from Register or Memory

All of the Registers of LT7689 are described as below. Each register contains 8bits data. The register table includes the detail description, default value and access attribute (RO: Read Only, WO: Write Only, RW: Readable and Writeable).

19.1 Status Register

Table 19-2: Status Register (STSR)

Bit	Description	Default	Access
7	**Memory Write FIFO Full Indicate** 0: The FIFO of Memory Write not Full 1: The FIFO of Memory Write is Full Host can write data to memory only if the Memory Write FIFO is not full.	0	RO
6	**Memory Write FIFO Empty Indicate** 0: The FIFO of Memory Write not Empty 1: The FIFO of Memory Write is Empty When the Memory Write FIFO is empty, Host can continuous to write 8bpp data with 64 pixels, 16bpp data with 32 pixels or 24bpp data with 16 pixels.	1	RO

Bit	Description	Default	Access
5	**Memory Read FIFO Full Indicate** 0: The FIFO of Memory Read not Full 1: The FIFO of Memory Read is Full When the Memory Read FIFO was Full, Host can continuous to read 8bpp data with 64 pixels, 16bpp data with 32 pixels or 24bpp data with 16 pixels.	0	RO
4	**Memory Read FIFO Empty Indicate** 0: The FIFO of Memory Read not Empty 1: The FIFO of Memory Read is Empty When the Memory Read FIFO not Empty, Host can continuous read pixel data from Memory.	1	RO
3	**Core Task is Busy, Fontwr_Busy** This bit represents whether the on-going action is completed (BTE, Geometry engine, DMA, text writing, or graphic writing). 0: The action is complete or idle. 1: The action is not completed, in a busy state. When the Host program switches text and graphics mode, or changes the base diagram related settings, the program must first verify if the LT7689 is idle. **Note:** In text mode, before program changes text rotation, line spacing, character spacing, foreground color, background color, and text/graphics settings, Host must confirm this bit is 0.	0	RO
2	**Display RAM Ready for Access** 0: Display RAM is not ready for access 1: Display RAM is ready for access Before checking this bit, Host has to setup REG[E4h] bit0 “SDR_INIT” as 1.	0	RO
1	**Operation Mode Status** 0: Normal operation state 1: Inhibit operation state Inhibit operation state means internal reset event is running or initial display is running or chip enters power saving state. In power saving state, this bit becomes 1 until PLL clock stops.	0	RO
0	**Interrupt Pin State** 0: No interrupt activated 1: Interrupt activated	0	RO

19.2 Configuration Registers

REG[00h] Software Reset Register (SRR)

Bit	Description	Default	Access
7	**Reconfigure PLL frequency** Writing "1" to this bit will reconfigure PLL frequency. **Note:** 1. When users change PLL relative parameters, PLL clock will not be changed immediately, users must set this bit as "1" again. 2. Users may read (check) this bit to find out whether PLL clock is changed or not.	0	RW
6-1	Reserved	0	RO
0	**Software Reset (Reconfigure PLL Frequency)** 0: Normal Operation. 1: Software Reset. The bit will auto clear after reset. Software Reset only reset internal state machine. Configuration Registers value will not be reset. Therefore, all read-only flag in the register will return to its initial value. Users should actively make sure all the flag is in desired state.	0	WO
0	**Warning Condition Flag** 0: No warning operation occurred 1: Warning condition occurred. Please refer to the description of REG[E4h] bit3.	0	RO

REG[01h] Chip Configuration Register (CCR)

Bit	Description	Default	Access
7	**Check PLL Ready** Host may read (check) this bit to find out whether PLL clock is ready or not. Read "1" means PLL clock is ready and switched successfully.	0	RW
6	**Mask WAIT# on CS# De-assert** 0: No Mask, WAIT# keeps assert if internal state keeps busy and cannot accept next R/W cycle, no matter CS# assert/de-assert. If Host cycle cannot be extended while WAIT# keeps low, Host program should poll WAIT# and wait for it to become high, then start next access. 1: Mask, WAIT# de-assert when CS# de-assert. Used to automatically extend Host cycle by Wait#.	1	RW
5	**Keypad-scan Enable/Disable** 0: Disable 1: Enable	0	RW

Bit	Description	Default	Access
4-3	**TFT Panel I/F Setting** 00b: 24bits TFT Output 01b: 18bits TFT Output 10b: 16bits TFT Output 11b: Without TFT Output Other unused TFT output pins are set as GPIO or Key–Scan function.	01b	RW
2	NA	NA	NA
1	**Serial Flash or SPI Interface Enable/Disable** 0: Disable (GPIO Function) 1: Enable (SPI Master Function)	0	RW
0	NA	NA	NA

REG[02h] Memory Access Control Register (MACR)

Bit	Description	Default	Access
7-6	**Host Read/Write Image Data Format** 0xb: Direct Write, for below I/F format: 810b: Mask high byte of each data (ex. 16 bit MPU I/F with 8-bpp data mode 1) 11b: Mask high byte of even data (ex. 16 bit MPU I/F with 24-bpp data mode 2)	0	RW
5-4	**Host Read Memory Direction (Only for Graphic Mode)** 00b: Left→Right then Top→Bottom. 01b: Right→Left then Top→Bottom. 10b: Top→Bottom then Left→Right. 11b: Bottom→Top then Left→Right. Ignored if canvas is in linear addressing mode.	0	RW
3	NA	NA	NA
2-1	**Host Write Memory Direction (Only for Graphic Mode)** 00b: Left→Right then Top→ Bottom (Original) 01b: Right → Left then Top → Bottom (Horizontal Flip) 10b: Top → Bottom then Left → Right (Rotate right 90° & Horizontal flip) 11b: Bottom → Top then Left → Right (Rotate left 90°) Ignored if canvas is in linear addressing mode.	0	RW
0	NA (must keep it as 0)	0	RO

REG[03h] Input Control Register (ICR)

Bit	Description	Default	Access
7	**Interrupt Pin Active Level** 0: Low Active 1: High Active	0	RW
6	**External Interrupt Signal - PSM[0] Pin De-bounce** 0: Without De-bounce 1: Enable De-bounce (1,024 OSC Clock)	0	RW
5-4	**External Interrupt Signal - PSM[0] Pin Trigger Type** 00b: low level trigger 01b: falling edge trigger 10b: high level trigger 11b: rising edge trigger	00b	RW
3	NA	0	RW
2	**Text Mode Enable** 0: Graphic mode 1: Text mode Before toggle this bit, users must make sure core task busy bit in status register is busy or idle. This bit is always 0 if canvas' address mode is in linear mode.	0	RW
1-0	**Memory Port Read/Write Destination Selection** 00b: Image buffer (Display RAM) for image data, pattern, user-characters. Support Read-modify-Write. 01b: Gamma table for Color Red/Green/Blue. Each color's gamma table has 256 bytes. Users need to specify desired gamma table and continuously write 256 bytes. 10b: Graphic Cursor RAM (only accept low 8-bits MPU data, similar to normal register data r/w.), not support Graphic Cursor RAM read. It contains 4 graphic cursor sets. Each set has 128x16 bits. Users need to specify target graphic cursor set and continuously write 256 bytes. 11b: Color palette RAM. It is 64x12 bits SRAM. Since MCU writes 8 bits data at a time, when it writes every even byte, only the lower 4 bits will be stored to the RAM. Not support Color palette RAM read. Users need to continuously write 128 bytes.	0	RW

REG[04h] Memory Data Read/Write Port (MRWDP)

Bit	Description	Default	Access
7-0	**Write Function: Memory Write Data** Data to write in memory corresponding to the setting of REG[03h][1:0]. Continuous data write cycle can be accepted in case of writing great amount of data. **Note:** A. Image data in Display RAM: Set to 8/16-bits based on the MCU I/F bit width. Host R/W image data format can be set. Canvas color depth and related settings can also be set in block mode. B. Pattern data for BTE operation in Display RAM: Set to 8/16-bits based on the MCU I/F bit width. Host R/W image data format can be set. Canvas color depth and related settings can also be set in block mode. Active window's width and height should be set as 8x8 or 16x16 depending on users needs. C. User-characters in Display RAM: Set to 8/16-bits based on the MCU I/F bit width. Host R/W image data format can be set, and canvas is set to linear mode. D. Character code: only low 8-bits MCU data are accepted, similar to normal register R/W. For two bytes character code, input high byte first. For user defined Character, code < 8000h is half size, code >= 8000h is full size. E. Gamma table data: only low 8-bits MPU data are accepted. Users must set "Select Gamma table sets([3Ch] Bit6-5)" to clear internal Gamma table's address counter then start to write data. Users should program 256 bytes data to memory data port. F. Graphic Cursor RAM data: only low 8-bits MPU data are accepted. Users must set "Select Graphic Cursor sets" bits to clear internal Graphic Cursor RAM address counter then start to write data. G. Color palette RAM data: only low 8-bits MPU data are accepted. User must program full Color palette RAM in a continuous 128 byte data and write to memory data port, and cannot change register address during the writing process. **Read Function: Memory Read Data** To read data from memory, REG[03h][1:0] has to be set. Continuous data read cycle can be accepted in case of reading great amount of data. **Note1:** when reading different port address, a dummy read must be issued. The first data read cycle is dummy read and data should be ignored. Graphic Cursor RAM & Color palette RAM data do not support data read function. **Note2:** No matter what the color depth setting is, data reading is based on 4 bytes. **Note3:** If users want to change the address of the written registers, but there are data already written to Display RAM, users must make sure (1) write FIFO is empty before changing register number; (2) Core task busy status bit becomes idle.	--	RW

19.3 PLL Setting Register

REG[05h] PCLK PLL Control Register 1 (PPLLC1)

Bit	Description	Default	Access
7-6	**PCLK Output Divider Ratio, OD[1:0]** 00b: Divided by 1. 01b: Divided by 2. 10b: Divided by 3. 11b: Divided by 4.	0	RW
5-1	**PCLK Input Divider Ratio, R[4:0]** The value should be 2~31.	10	RW
0	**PCLK Feedback Divider Ratio of Loop, N[8]** Total 9 bits, the value should be 2~511..	0	RW

REG[06h] PCLK PLL Control Register 2 (PPLLC2)

Bit	Description	Default	Access
7-0	**PCLK PLLDIVN[7:0]** Total 9 bits, the value should be 2~511.	60	RW

Note: PCLK is used by TFT panel's scan clock and derived from CCLK.

REG[07h] MCLK PLL Control Register 1 (MPLLC1)

Bit	Description	Default	Access
7-6	**MCLK output divider Ratio, OD[1:0]** 00b: Divided by 1. 01b: Divided by 2. 10b: Divided by 3. 11b: Divided by 4.	0	RW
5-1	MCLK Input Divider Ratio, R[4:0] The value should be 2~31.	10	RW
0	MCLK Feedback Divider Ratio of Loop, N[8] Total 9 bits, the value should be 2~511.	0	RW

REG[08h] MCLK PLL Control Register 2 (MPLLC2)

Bit	Description	Default	Access
7-0	**MCLK PLLDIVN[7:0]** Total 9 bits, the value should be 2~511.	133	RW

Note: MCLK is used by internal Display RAM's clock.

REG[09h] CCLK PLL Control Register 1 (CPLLC1)

Bit	Description	Default	Access
7-6	**CCLK Output Divider Ratio, OD[1:0]** 00b: Divided by 1. 01b: Divided by 2. 10b: Divided by 3. 11b: Divided by 4.	0	RW
5-1	**CCLK Input Divider Ratio, R[4:0]** The value should be 2~31.	10	RW
0	**CCLK Feedback Divider Ratio of Loop, N[8]** Total 9 bits, the value should be 2~511.	0	RW

REG[0Ah] CCLK PLL Control Register 2 (CPLLC2)

Bit	Description	Default	Access
7-0	**CCLK PLLDIVN[7:0]** Total 9 bits, the value should be 2~511.	133	RW

Note1: CCLK is used by Core's Clock.

Note2: PLL output frequency is calculated from the following the equation:

FOUT = XI * (N/R)÷ OD

The input frequency XI/R is no less than 1MHz. For example, IN = 10MHz, R[4:0] = 01010, N[8:0] = 100000000, OD[1:0] = 11, then

FOUT = 10MHz * (256 / 10)÷ 4 = 64 MHz

19.4 Interrupt Control Register

REG[0Bh] Interrupt Enable Register (INTEN)

Bit	Description	Default	Access
7	**Wakeup/Resume Interrupt Enable** 0: Disable 1: Enable	0	RW
6	**External Interrupt Input - PSM[0] Enable)** 0: Disable 1: Enable	0	RW
5	**I2C Master Interrupt Enable** 0: Disable 1: Enable	0	RW
4	**VSYNC Time Base Interrupt Enable** 0: Disable Interrupt 1: Enable Interrupt This interrupt is to inform MCU that the VSYNC of the LCD has tearing effect.	0	RW
3	**Keypad-scan Interrupt Enable** 0: Disable Interrupt 1: Enable Interrupt	0	RW
2	**Serial Flash DMA Complete / Draw Task Finished / BTE Process Complete etc. Interrupt Enable** 0: Disable Interrupt 1: Enable Interrupt	0	RW
1	**PWM Timer-1 Interrupt Enable** 0: Disable Interrupt 1: Enable Interrupt	0	RW
0	**PWM Timer-0 Interrupt Enable** 0: Disable Interrupt 1: Enable Interrupt	0	RW

REG[0Ch] Interrupt Event Flag Register (INTF)

Bit	Description	Default	Access
7	**Wakeup/Resume Interrupt Flag** **Write:** Clear Interrupt Flag 0: No Operation 1: Clear Wakeup/Resume Interrupt Flag **Read:** Read Interrupt Flag 0: No Wakeup/Resume Interrupt Happens 1: Wakeup/Resume Interrupt Happens	0	RW
6	**External Interrupt Input - PSM[0] Flag** **Write:** Clear PSM[0] Pin Interrupt Flag 0: No Operation 1: Clear PSM[0] Interrupt Flag **Read:** Read PSM[0] Pin Interrupt Flag 0: No PSM[0] Interrupt Happens 1: PSM[0] Interrupt Happens	0	RW
5	NA	NA	NA
4	**VSYNC Time Base Interrupt Flag** **Write:** Clear VSYNC Interrupt Flag 0: No Operation 1: Clear VSYNC Interrupt Flag **Read:** Read VSYNC Interrupt Flag 0: No VSYNC Interrupt Happens 1: VSYNC Interrupt Happens	0	RW
3	**Keypad-scan Interrupt Flag** **Write:** Clear Keypad-scan Interrupt Flag 0: No Operation 1: Clear Keypad-scan Interrupt Flag **Read:** Read Keypad-scan Interrupt Flag 0: No Keypad-scan Interrupt Happens 1: Keypad-scan Interrupt Happens	0	RW

Bit	Description	Default	Access
2	**Serial Flash DMA Complete | Draw Task Finished | BTE Process Complete etc. Interrupt Flag** **Write:** Clear Process Complete Interrupt Flag 0: No Operation 1: Clear Interrupt Flag **Read:** Read Process Complete Interrupt Flag 0: No Interrupt Happens 1: Interrupt Happens	0	RW
1	**TFT_PWM[1] Timer Interrupt Flag** **Write:** Clear TFT_PWM[1] Interrupt Flag 0: No Operation 1: Clear TFT_PWM[1] Interrupt Flag **Read:** Read TFT_PWM[1] Interrupt Flag 0: No TFT_PWM[1] Interrupt Happens 1: TFT_PWM[1] Interrupt Happens	0	RW
0	**TFT_PWM[0] Timer Interrupt Flag** **Write:** Clear TFT_PWM[0] Interrupt Flag 0: No Operation 1: Clear TFT_PWM[0] Interrupt Flag **Read:** Read TFT_PWM[0] Interrupt Flag 0: No TFT_PWM[0] Interrupt Happens 1: TFT_PWM[0] Interrupt Happens	0	RW

Note: If the Host receives interruption, but the flag of this register shows no interrupt happened, then Host should confirm SPI Master State Register Interrupt Flag of REG[BAh].

REG[0Dh] Mask Interrupt Flag Register (MINTFR)

Bit	Description	Default	Access
7	**Mask Wakeup/Resume Interrupt Flag** 0: Unmask 1: Mask	0	RW
6	**External Interrupt Input - PSM[0] Flag** 0: Unmask 1: Mask	0	RW
5	NA	NA	NA
4	**VSYNC Time Base Interrupt Flag** 0: Unmask 1: Mask	0	RW
3	**Keypad-scan Interrupt Flag** 0: Unmask 1: Mask	0	RW

Bit	Description	Default	Access
2	**Serial Flash DMA Complete | Draw Task Finished | BTE Process Complete etc. Interrupt Flag** 0: Unmask 1: Mask	0	RW
1	**TFT_PWM[1] Timer Interrupt Flag** 0: Unmask 1: Mask	0	RW
0	**TFT_PWM[0] Timer Interrupt Flag** 0: Unmask 1: Mask	0	RW

Note: If the Host masks interrupt flag, then LT7689 will not send interruption to Host. If only some unmasked interrupt flags are used, Host can check interrupt flag to find out whether there is an interrupt.

REG[0Eh] Pull-High Control Register (PUENR)

Bit	Description	Default	Access
7-6	NA.	0	RO
5	**GPIO-F[7:0] Pull-High Enable** 0: without Pull-up Resistor 1: with Pull-up	0	RW
4	**GPIO-E[7:0] Pull-High Enable** 0: without Pull-up Resistor 1: with Pull-up	0	RW
3	**GPIO-D[7:0] Pull-High Enable** 0: without Pull-up Resistor 1: with Pull-up	0	RW
2	**GPIO-C[4:0] Pull-High Enable** 0: without Pull-up Resistor 1: with Pull-up	0	RW
1	**DB[15:8] Pull- High Enable** 0: without Pull-up Resistor 1: with Pull-up	0	RW
0	**DB[7:0] Pull- High Enable** 0: without Pull-up Resistor 1: with Pull-up	0	RW

Note: These bits are available only when GPIO function is enabled.

REG[0Fh] PD for GPIO/Key Function Select Register (PSFSR)

Bit	Description	Default	Access
7	**PD[18] – Function Select** 0: GPIO-D7 1: KO[4]	0	RW
6	**PD[17] – Function Select** 0: GPIO-D5 1: KO[2]	0	RW
5	**PD[16] – Function Select** 0: GPIO-D4 1: KO[1]	0	RW
4	**PD[9] – Function Select** 0: GPIO-D3 1: KO[3]	0	RW
3	**PD[8] – Function Select** 0: GPIO-D2 1: KI[3]	0	RW
2	**PD[2] – Function Select** 0: GPIO-D6 1: KI[4]	0	RW
1	**PD[1] – Function Select** 0: GPIO-D1 1: KI[2]	0	RW
0	**PD[0] – Function Select** 0: GPIO-D0 1: KI[1]	0	RW

19.5 LCD Display Control Registers

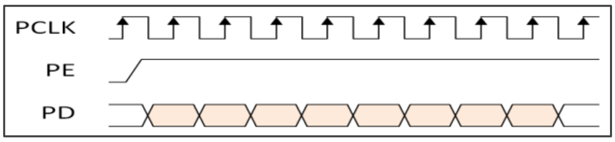
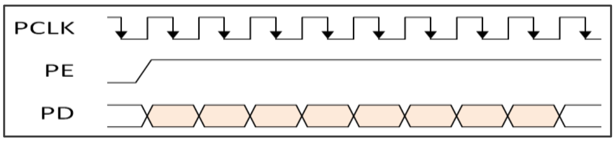
REG[10h] Main/PIP Window Control Register (MPWCTR)

Bit	Description	Default	Access
7	**PIP-1 Window Enable/Disable** 0: PIP-1 Window Disable 1: PIP-1 Window Enable PIP-1 window always on top of PIP-2 window.	0	RW
6	**PIP-2 Window Enable/Disable** 0: PIP-2 Window Disable 1: PIP-2 Window Enable PIP-1 window always on top of PIP-2 window	0	RW
5	NA	0	RO
4	**Select Configure PIP 1 or 2 Window's parameters** PIP window's parameter includes Color Depth, Starting Address, Image Width, Display Coordinates, Window Coordinates, Window Width and Window Height. 0: To configure PIP 1's parameters. 1: To configure PIP 2's parameters.	0	RW
3-2	**Main Image Color Depth Setting** 00b: 8bpp Generic TFT (256 color) 01b: 16bpp Generic TFT (65K color) 1xb: 24bpp Generic TFT (1.67M color)	1	RW
1	NA	0	RW
0	**To Control Panel's Synchronous Signals** 0: Sync Mode → Enable VSYNC, HSYNC, DE 1: DE Mode → Only DE enable, VSYNC & HSYNC in idle state.	0	RW

REG[11h] PIP Window Color Depth Setting (PIPCDEP)

Bit	Description	Default	Access
7-4	NA	0	RO
3-2	**PIP-1 Window Color Depth Setting** 00b: 8bpp Generic TFT (256 color) 01b: 16bpp Generic TFT (65K color) 1xb: 24bpp Generic TFT (1.67M color)	1	RW
1-0	**PIP-2 Window Color Depth Setting** 00b: 8bpp Generic TFT (256 color) 01b: 16bpp Generic TFT (65K color) 1xb: 24bpp Generic TFT (1.67M color)	1	RW

REG[12h] Display Configuration Register (DPCR)

Bit	Description	Default	Access
7	**PCLK Inversion** 0: TFT Panel fetches PD at PCLK rising edge.  1: TFT Panel fetches PD at PCLK falling edge.  	0	RW
6	**Display ON/OFF** 0b: Display Off 1b: Display On	0	RW
5	**Display Test Color Bar** 0b: Disable 1b: Enable	0	RW
4	**This bit must be 0.**	0	RW
3	**VDIR: Vertical Scan Direction** 0: From Top to Bottom 1: From bottom to Top	0	RW
2-0	**Parallel PD[23:0] Output Sequence** 000b: RGB 001b: RBG 010b: GRB 011b: GBR 100b: BRG 101b: BGR 110b: Gray 111b: Send out Idle State. All data are 0 (Black) or 1(White). This option has to setup with REG[13h].	0	RW

Note: When VDIR = 1, PIP window, Graphic Cursor and Text Cursor function will be disabled automatically.

REG[13h] Panel Scan Clock and Data Setting Register (PCSR)

Bit	Description	Default	Access
7	**HSYNC Polarity** 0: Low Active 1: High Active	0	RW
6	**VSYNC Polarity** 0: Low Active 1: High Active	0	RW
5	**PDE Polarity** 0: High Active 1: Low Active	0	RW
4	**PDE Idle State** 0: Pin “PDE” output Low 1: Pin “PDE” output High This is used to setup the PDE output status in Power Saving mode or Display Off.	0	RW
3	**PCLK Idle State** 0: Pin “PCLK” output Low 1: Pin “PCLK” output High This is used to setup the PCLK output status in Power Saving mode or Display Off.	0	RW
2	**PD Idle State** 0: Pins “PD[23:0]” output Low 1: Pins “PD[23:0]” output High This is used to setup the PD output status in Vertical/Horizontal Non-Display Period, Power Saving mode or Display Off.	0	RW
1	**HSYNC Idle State** 0: Pin “HSYNC” output Low 1: Pin “HSYNC” output High This is used to setup the HSYNC output status in Power Saving mode or Display Off.	1	RW
0	**VSYNC Idle State** 0: Pin “VSYNC” output Low 1: Pin “VSYNC” output High This is used to setup the VSYNC output status in Power Saving mode or Display Off.	1	RW

Note: The sum of (HST + HPW + HND) had better be larger than 64Pixels to prevent scanning FIFO empty. If both PIP1 and PIP2 are enabled and are very close to the left side of the window, there is only a small change in PIP1 and PIP2's UL-X. Please refer to Figure 12-2 TFT-LCD interface Timing diagram.

REG[14h] Horizontal Display Width Register (HDWR)

Bit	Description	Default	Access
7-0	**Horizontal Display Width Setting** This register is used to set a horizontal display width. Its specified LCD screen resolution is 8 pixels in one unit resolution. **Horizontal Display Width (pixels)** **= (HDWR + 1)* 8 + HDWFTR** HDWFTR (REG[15h]) is the fine-tuning value for Horizontal Display Width. Each fine-tuning resolution is 1 pixel, and the maximum horizontal width is 2,048 pixels.	4Fh	RW

REG[15h] Horizontal Display Width Fine Tune Register (HDWFTR)

Bit	Description	Default	Access
7-4	NA	0	RO
3-0	**Horizontal Display Width Fine Tuning** This Register is the fine-tuning value for Horizontal Display Width, and each fine-tuning resolution is 1 pixel.	0	RW

REG[16h] Horizontal Non-Display Period Register (HNDR)

Bit	Description	Default	Access
7-5	NA	0	RO
4-0	**Horizontal Non-Display Period** This register assigns the Period of Horizontal Non-Display. It's also called “Back Porch”. **Horizontal Non-Display Period (Pixels)** **= (HNDR + 1)* 8 + HNDFTR** HNDFTR (REG[17h]) is the fine-tuning value for Horizontal Non-display Period. Each fine-tuning resolution is 1 pixel, and the maximum horizontal width is 2,048 pixels.	03h	RW

REG[17h] Horizontal Non-Display Period Fine Tune Register (HNDFTR)

Bit	Description	Default	Access
7-4	NA	0	RO
3-0	**Horizontal Non-Display Period Fine Tuning** This Register is the fine-tuning value for Horizontal Non-display Period, and each fine-tuning resolution is 1 pixel. it is used to support the SYNC mode panel.	06h	RW

REG[18h] HSYNC Start Position Register (HSTR)

Bit	Description	Default	Access
7-5	NA	0	RO
4-0	**HSYNC Start Position** This register specifies the starting address of the HSYNC. The starting point for the calculation is the point at which the end of the display area is to start producing HSYNC. The basic unit of each adjustment is 8pixel. It's also called "Front Porch". **HSYNC Start Position = (HSTR + 1)* 8**	1Fh	RW

REG[19h] HSYNC Pulse Width Register (HPWR)

Bit	Description	Default	Access
7-5	NA	0	RO
4-0	**HSYNC Pulse Width** **HSYNC Pulse Width (Pixels)= (HPW + 1)* 8**	0	RW

REG[1Ah-1Bh] Vertical Display Height Register (VDHR)

Bit	Description	Default	Access
7-0	**Vertical Display Height** REG[1Ah] mapping to VDHR [7:0] REG[1Bh] bit[2:0] mapping to VDHR [10:8], bit[7:3] are not used. The height of the vertical display is in line, and the formula is as follows: **Vertical Display Height (Line)= VDHR + 1**	DFh	RW

REG[1Ch-1Dh] Vertical Non-Display Period Register (VNDR)

Bit	Description	Default	Access
7-0	**Vertical Non-Display Period** REG[1Ch] mapping to VNDR [7:0] REG[1Dh] bit[1:0] mapping to VNDR [9:8], REG[1Dh] bit[7:2] are not used. The formula is as follows: **Vertical Non-Display Period (Line)= (VNDR + 1)**	15h	RW

REG[1Eh] VSYNC Start Position Register (VSTR)

Bit	Description	Default	Access
7-0	**VSYNC Start Position** The starting position from the end of display area to the beginning of VSYNC. **VSYNC Start Position (Line)= (VSTR + 1)**	0Bh	RW

REG[1Fh] VSYNC Pulse Width Register (VPWR)

Bit	Description	Default	Access
7-6	NA	0	RO
5-0	**VSYNC Pulse Width** **VSYNC Pulse Width (Line)= (VPWR + 1)**	0	RW

REG[23h-20h] Main Image Start Address (MISA)

Bit	Description	Default	Access
7-0	**Main Image Start Address** REG[20h] mapping to MISA[7:0], bit[1:0] must be 0. REG[21h] mapping to MISA[15:8] REG[22h] mapping to MISA[23:16] REG[23h] mapping to MISA[31:24]	0	RW

REG[25h-24h] Main Image Width (MIW)

Bit	Description	Default	Access
7-0	**Main Image Width** REG[24h] mapping to MIW[7:0], bit[1:0] must be 0. REG[25h] bit[4:0] mapping to MIW[12:8], bit[7:5] are not used.. The unit is pixel, which is the pixel that represents the horizontal width of the actual LCD. The maximum setting is 8,192 pixels.	0	RW

REG[27h-26h] Main Window Upper-Left Corner X-Coordinates (MWULX)

Bit	Description	Default	Access
7-0	**Main Window Upper-Left Corner X-Coordinates** REG[26h] mapping to MWULX[7:0], bit[1:0] must be 0. REG[27h] bit[4:0] mapping to MWULX [12:8], bit[7:5] are not used. The unit is pixel. The sum of X-Coordinates plus Horizontal Display Width cannot be greater than 8,191.	0	RW

REG[29h-28h] Main Window Upper-Left corner Y-Coordinates (MWULY)

Bit	Description	Default	Access
7-0	**Main Window Upper-Left Corner Y-Coordinates** REG[28h] mapping to MWULY[7:0] REG[29h] bit[4:0] mapping to MWULY [12:8], bit[7:5] are not used. Unit: Pixel. Range is between 0 and 8,191.	0	RW

REG[2Bh-2Ah] PIP Window 1 or 2 Display Upper-Left Corner X-Coordinates (PWDULX)

Bit	Description	Default	Access
7-0	**PIP Window Display Upper-Left Corner X-Coordinates** REG[2Ah] mapping to PWDULX[7:0], bit[1:0] must be 0. REG[2Bh] bit[4:0] mapping to PWDULX[12:8], bit[7:5] are not used. Unit: Pixel. Y-axis coordinates should be less than vertical display height. This setting is relative to the PIP settings in the register of REG[10h].	0	RW

REG[2Dh-2Ch] PIP Window 1 or 2 Display Upper-Left corner Y-Coordinates (PWDULY)

Bit	Description	Default	Access
7-0	**PIP Window Display Upper-Left Corner Y-Coordinates** REG[2Ch] mapping to PWDULX[7:0] REG[2Dh] bit[4:0] mapping to PWDULX[12:8], bit[7:5] are not used. Y-axis coordinates should be less than vertical display height. This setting is relative to the PIP settings in the register of REG[10h].	0	RW

REG[31h-2Eh] PIP Image 1 or 2 Start Address (PISA)

Bit	Description	Default	Access
7-0	**PIP Image Start Address** REG[2Eh] mapping to PISA[7:0], bit[1:0] must be 0. REG[2Fh] mapping to PISA [15:8] REG[30h] mapping to PISA [23:16] REG[31h] mapping to PISA [31:24] This setting is relative to the PIP settings in the register of REG[10h].	0	RW

REG[33h-32h] PIP Image 1 or 2 Width (PIW)

Bit	Description	Default	Access
7-0	**PIP Image Width** REG[32h] mapping to PIW[7:0], bit[1:0] must be 0. REG[33h] bit[5:0] mapping to PIW[13:8], bit[7:6] are not used. Unit: Pixel. The value is physical width, and the maximum value is 8192 pixels. This width should be less than horizontal display width. This setting is relative to the PIP settings in the register of REG[10h].	0	RW

REG[35h-34h] PIP Window Image 1 or 2 Upper-Left Corner X-Coordinates (PWIULX)

Bit	Description	Default	Access
7-0	**PIP Window 1 or 2 Image Upper-Left Corner X-Coordinates** REG[34h] mapping to PWIULX[7:0], bit[1:0] must be 0. REG[35h] bit[4:0] mapping to PWIULX[12:8], bit[7:5] are not used. Unit: Pixel. The sum of X-axis coordinates plus PIP image width must be less than or equal to 8,191. This setting is relative to the PIP settings in the register of REG[10h].	0	RW

REG[37h-36h] PIP Window Image 1 or 2 Upper-Left Corner Y-Coordinates (PWIULY)

Bit	Description	Default	Access
7-0	**PIP Windows Display Upper-Left Corner Y-Coordinates** REG[36h] mapping to PWIULY[7:0], bit[1:0] must be 0. REG[37h] bit[4:0] mapping to PWIULY[12:8], bit[7:5] are not used. Unit: Pixel. The sum of Y-axis coordinates plus PIP window height should be less than or equal to 8,191. This setting is relative to the PIP settings in the register of REG[10h].	0	RW

REG[39h-38h] PIP Window 1 or 2 Width (PWW)

Bit	Description	Default	Access
7-0	**PIP Window Width** REG[38h] mapping to PWW[7:0], bit[1:0] must be 0. REG[39h] bit[5:0] mapping to PWW[13:8], bit[7:6] are not used. Unit: Pixel. Maximum value is 8,192 pixels. This setting is relative to the PIP settings in the register of REG[10h].	0	RW

REG[3Bh-3Ah] PIP Window 1 or 2 Height (PWH)

Bit	Description	Default	Access
7-0	**PIP Window Height** REG[3Ah] mapping to PWH[7:0], bit[1:0] must be 0. REG[3Bh] bit[5:0] mapping to PWH[13:8], bit[7:6] are not used. Unit: Pixel. The value is physical pixel number and the maximum value is 8,192 pixels. This setting is relative to the PIP settings in the register of REG[10h].	**0**	**RW**

Note 1: The resolution of Window Size and Starting Position in the horizontal direction is 8 pixels, the vertical resolution is 1 line.

Note 2: The above registers REG[20h] ~ REG[3Bh] need to be written from LSB to MSB in turn. Suppose we need to set the Main Image Start Address, the relative registers are REG[20h] to REG[23h]. Then Host must write Address data from LSB (REG[20h]) to MSB (REG[23h]) in turn. When REG[23h] is written, LT7689 will then write the complete Main Image Start Address to the internal register.

REG[3Ch] Graphic / Text Cursor Control Register (GTCCR)

Bit	Description	Default	Access
7	**Gamma Correction Enable** 0: Disable 1: Enable Gamma correction is the last output stage.	0	RW
6-5	**Gamma Table Select for Host Write Gamma Data** 00b: Gamma table for Blue 01b: Gamma table for Green 10b: Gamma table for Red 11b: NA	0	RW
4	**Graphic Cursor Enable** 0: Graphic Cursor Disable 1: Graphic Cursor Enable Graphic cursor will be disabled if VDIR (REG[12h] bit3) is set to “1”.	0	RW
3-2	**Graphic Cursor Selection** Select one from four graphic cursor types. (00b to 11b) 00b: Graphic Cursor Set 1 01b: Graphic Cursor Set 2 10b: Graphic Cursor Set 3 11b: Graphic Cursor Set 4	0	RW
1	**Text Cursor Enable** 0: Disable 1: Enable **Note:** Text cursor & Graphic cursor cannot be enabled simultaneously. Graphic cursor has higher priority than Text cursor if enabled simultaneously.	0	RW

Bit	Description	Default	Access
0	**Text Cursor Blinking Enable** 0: Disable 1: Enable	0	RW

REG[3Dh] Blink Time Control Register (BTCR)

Bit	Description	Default	Access
7-0	**Text Cursor Blink Time Setting** 00h: 1 Frame Cycle Time 01h: 2 Frames Cycle Time 02h: 3 Frames Cycle Time : : FFh: 256 frames Cycle Time	0	RW

REG[3Eh] Text Cursor Horizontal Size Register (CURHS)

Bit	Description	Default	Access
7-5	NA	0	RO
4-0	**Text Cursor Horizontal Size Setting** 00000b: 1 Pixel 00001b: 2 Pixels : : 11111b: 32 Pixels **Note:** When the character is enlarged, the cursor will be enlarged at the same time.	07h	RW

REG[3Fh] Text Cursor Vertical Size Register (CURVS)

Bit	Description	Default	Access
7-5	NA	0	RO
4-0	**Text Cursor Vertical Size Setting** Unit: Pixel, Zero-based number. Value “0” means 1 pixel. **Note:** When the character is enlarged, the cursor will be enlarged at the same time.	0	RW

REG[40h-41h] Graphic Cursor Horizontal Position Register (GCHP)

Bit	Description	Default	Access
7-0	**Graphic Cursor Horizontal Position** REG[40h] mapping to GCHP[7:0] REG[41h] bit[4:0] mapping to GCHP[12:8], bit[7:5] are not	0	RW

	used.		

REG[42h-43h] Graphic Cursor Vertical Position Register (GCVP)

Bit	Description	Default	Access
7-0	**Graphic Cursor Vertical Position** REG[42h] mapping to GCVP[7:0] REG[43h] bit[4:0] mapping to GCVP[12:8], bit[7:5] are not used.	0	RW

REG[44h] Graphic Cursor Color 0 (GCC0)

Bit	Description	Default	Access
7-0	**Graphic Cursor Color 0 with 256 Colors** RGB Format [7:0] = **RRRGGGBB**	0	RW

REG[45h] Graphic Cursor Color 1 (GCC1)

Bit	Description	Default	Access
7-0	**Graphic Cursor Color 1 with 256 Colors** RGB Format [7:0] = **RRRGGGBB**	0	RW

19.6 Geometric Engine Control Registers

REG[53h-50h] Canvas Start Address (CVSSA)

Bit	Description	Default	Access
7-0	**Start Address of Canvas** REG[50h] mapping to CVSSA[7:0], bit[1:0] must be 0. REG[51h] mapping to CVSSA[15:8] REG[52h] mapping to CVSSA[23:16] REG[53h] mapping to CVSSA[31:24] These Registers will be ignored if canvas is in linear Addressing mode.	0	RW

REG[55h-54h] Canvas Image Width (CVS_IMWTH)

Bit	Description	Default	Access
7-0	**Canvas Image Width** REG[54h] mapping to CVS_IMWTH[7:0], bit[1:0] must be 0. REG[55h] bit[5:0] mapping to CVS_IMWTH[13:8], bit[7:6] are not used. Width = Real Image Width These Registers will be ignored if canvas is in linear Addressing mode.	0	RW

Note: The unit of REG[54h] ~ REG[5Dh] is Pixel.

REG[57h-56h] Active Window Upper-Left Corner X-Coordinates (AWUL_X)

Bit	Description	Default	Access
7-0	**Active Window Upper-Left Corner X-Coordinates** REG[56h] mapping to AWUL_X[7:0] REG[57h] bit[4:0] mapping to AWUL_X[12:8], bit[7:5] are not used. Unit: Pixel. The sum of X-axis coordinates plus Active Window width cannot be larger than 8,191. These Registers will be ignored if canvas is in linear Addressing mode.	0	RW

REG[59h-58h] Active Window Upper-Left Corner Y-Coordinates (AWUL_Y)

Bit	Description	Default	Access
7-0	**Active Window Upper-Left Corner Y-Coordinates** REG[58h] mapping to AWUL_Y[7:0] REG[59h] bit[4:0] mapping to AWUL_Y[12:8], bit[7:5] are not used. Unit: Pixel. The sum of Y-axis coordinates plus Active Window height cannot be larger than 8,191. These Registers will be ignored if canvas is in linear Addressing mode.	0	RW

REG[5Bh-5Ah] Active Window Width (AW_WTH)

Bit	Description	Default	Access
7-0	**Active Window Width [13:8]** REG[5Ah] mapping to AW_WTH[7:0] REG[5Bh] bit[5:0] mapping to AW_WTH[13:8], bit[7:6] are not used. Unit: Pixel. The value is physical pixel width. The Maximum pixel width is 8,192. These Registers will be ignored if canvas is in linear Addressing mode.	0	RW

REG[5Dh-5Ch] Active Window Height (AW_HT)

Bit	Description	Default	Access
7-0	**Height of Active Window [13:8]** REG[5Ch] mapping to AW_HT[7:0] REG[5Dh] bit[5:0] mapping to AW_HT[13:8], bit[7:6] are not used. Unit: Pixel. The value is physical pixel width. The maximum pixel width is 8,192. These Registers will be ignored if canvas is in linear Addressing mode.	0	RW

REG[5Eh] Color Depth of Canvas & Active Window (AW_COLOR)

Bit	Description	Default	Access
7-4	NA	0	RO
3	**Select What will Be Read Back from Graphic Read/Write Position Register** 0: Read back Graphic Write position 1: Read back Graphic Read position (Pre-fetch Address)	0	RW
2	**Canvas Addressing Mode** 0: Block mode (X-Y coordinates addressing) 1: Linear mode	0	RW
1-0	**Canvas Image's Color Depth & Memory R/W Data Width** ***In Block Mode:*** 00b: 8bpp 01b: 16bpp 1xb: 24bpp **Note:** Monochrome data can be input under any one color depth and the proper image width. ***In Linear Mode:*** X0: 8-bits memory data read/write. X1: 16-bits memory data read/write	0	RW

Note: Please refer to Figure 13-2 Canvas and Active Windows.

REG[60h-5Fh] Graphic Read/Write X-Coordinate Register (CURH)

Bit	Description	Default	Access
7-0	**Write: Set Graphic Read/Write X-Coordinate position** **Read: Read Graphic Read/Write X-Coordinate position** To decide whether it is "Read position" or "Write position", it depends on the setting of REG[5Eh] bits. REG[5Fh] mapping to CURH[7:0] REG[60h] bit[4:0] mapping to CURH[12:8], bit[7:5] are not used. ***When DPRAM in Linear mode:*** Memory Read/Write address [15:0], Unit: Byte. **When DPRAM in Block mode:** Graphic Read/Write X-Coordinate [12:0], Unit: Pixel.	0	RW

Note: Host should program proper active window related parameters before configuring this register.

REG[62h-61h] Graphic Read/Write Y-Coordinate Register (CURV)

Bit	Description	Default	Access
7-0	***Write:*** **Set Graphic Read/Write X-Coordinate position** ***Read:*** **Read Graphic Read/Write X-Coordinate position** To decide whether it is "Read position" or "Write position", it depends on the setting of REG[5Eh] bits. REG[61h] mapping to CURV[7:0] REG[62h] bit[4:0] mapping to CURV[12:8], bit[7:5] are not used. ***When DPRAM In Linear Mode:*** Memory Read/Write address [31:16], Unit: Byte. ***When DPRAM In Block Mode:*** Graphic Read/Write Y-Coordinate [12:0], Unit: Pixel.	0	RW

Note: Host should program proper active window related parameters before configuring this register.

REG[64h-63h] Text Write X-Coordinates Register (F_CURX)

Bit	Description	Default	Access
7-0	**Text Write X-coordinates F_CURX[12:0]** REG[63h] mapping to F_CURX[7:0] REG[64h] bit[4:0] mapping to F_CURX[12:8], bit[7:5] are not used. ***Write:*** **Set Text Write X-Coordinates position** ***Read:*** **Current Text Write X-Coordinates position**	0	RW

REG[66h-65h] Text Write Y-Coordinates Register (F_CURY)

Bit	Description	Default	Access
7-0	**Text Write Y-coordinates F_CURY[12:0]** REG[65h] mapping to F_CURY[7:0] REG[66h] bit[4:0] mapping to F_CUR[12:8], bit[7:5] are not used. ***Write:*** **Set Text Write Y-Coordinates position** ***Read:*** **Current Text Write Y-Coordinates position**	0	RW

REG[67h] Draw Line/Triangle Control Register 0 (DCR0)

Bit	Description	Default	Access
7	**Draw Line / Triangle Start Control** *Write:* 0: Stop the drawing function. 1: Start the drawing function. *Read:* 0: Drawing function complete. 1: Drawing function is processing.	0	RW
6	NA	0	RO
5	**Fill Function for Triangle** 0: Non Fill 1: Fill	0	RW
4-1	**Draw Triangle or Line Select** 0000b: Draw Line 0001b: Draw Triangle 0010b: Rectangle 0011b: Quadrilateral 0100b: Pentagon 0101b: Polyline (3EP) 0110b: Polyline (4EP) 0111b: Polyline (5EP) 1000b: Ellipse 1001b: Rounded-Rectangle 1010b, 1011b: NA 1100b: Oval Arc on upper-right /1st Quadrant 1101b: Oval Arc on upper-left / 2nd Quadrant 1110b: Oval Arc on Lower-Left / 3rd Quadrant 1111b: Oval Arc on Lower-Right / 4th Quadrant	0	RW
0	**Polyline Style** 0: Open-end Polyline, 1: Closed Polyline (connect the last point to the start point)	0	RO

REG[69h-68h] Draw Line/Rectangle/Triangle Point 1 X-Coordinates Register (DLHSR)

Bit	Description	Default	Access
7-0	**Draw Line/Triangle Point 1 X-coordinates DLHSR[12:0]** REG[68h] mapping to DLHSR[7:0] REG[69h] bit[4:0] mapping to DLHSR[12:8], bit[7:5] are not used. Unit: Pixel. When draw a rectangle, start point & end point cannot locate at the same point or at same X- Coordinates or Y- Coordinates.	0	RW

REG[6Bh-6Ah] Draw Line/Rectangle/Triangle Point 1 Y-Coordinates Register (DLVSR)

Bit	Description	Default	Access
7-0	**Draw Line/Triangle Point 1 Y-coordinates DLVSR[12:0]** REG[6Ah] mapping to DLVSR[7:0] REG[6Bh] bit[4:0] mapping to DLVSR[12:8], bit[7:5] are not used.	0	RW

REG[6Dh-6Ch] Draw Line/Rectangle/Triangle Point 2 X-Coordinates Register (DLHER)

Bit	Description	Default	Access
7-0	**Draw Line/Triangle Point 2 X-coordinates DLHER[12:0]** REG[6Ch] mapping to DLHER[7:0] REG[6Dh] bit[4:0] mapping to DLHER[12:8], bit[7:5] are not used.	0	RW

REG[6Fh-6Eh] Draw Line/Rectangle/Triangle Point 2 Y-Coordinates Register (DLVER)

Bit	Description	Default	Access
7-0	**Draw Line/Triangle Point 2 Y-coordinates DLVER[12:0]** REG[6Eh] mapping to DLVER[7:0] REG[6Fh] bit[4:0] mapping to DLVER[12:8], bit[7:5] are not used.	0	RW

REG[71h-70h] Draw Triangle Point 3 X-Coordinates Register (DTPH)

Bit	Description	Default	Access
7-0	**Draw Line/Triangle Point 3 X-coordinates DTPH[12:0]** REG[70h] mapping to DTPH[7:0] REG[71h] bit[4:0] mapping to DTPH[12:8], bit[7:5] are not used.	0	RW

REG[73h-72h] Draw Triangle Point 3 Y-Coordinates Register (DTPV)

Bit	Description	Default	Access
7-0	**Draw Line/Triangle Point 3 Y-coordinates DTPV[12:0]** REG[72h] mapping to DTPV[7:0] REG[73h] bit[4:0] mapping to DTPV[12:8], bit[7:5] are not used.	0	RW

Note: When Drawing triangle: If any two endpoints are overlapped, it will draw a line. If three endpoints are overlapped, it will draw a dot.

REG[76h] Draw Circle/Ellipse/Ellipse Curve/Circle Square Control Register 1 (DCR1)

Bit	Description	Default	Access
7	**Draw Circle / Ellipse / Square /Circle Square Control** ***Write:*** 0: Stop the drawing function. 1: Start the drawing function. ***Read:*** 0: Drawing function complete. 1: Drawing function is processing.	0	RW
6	**Fill the Circle / Ellipse / Square / Rounded-rectangle Control** 0: Non Fill 1: Fill	0	RW
5-4	**Draw Circle / Ellipse / Square / Ellipse Curve / Rounded-rectangle Select** 00b: Draw Circle / Ellipse 01b: Draw Arc 10b: Draw Rectangle 11b: Draw Rounded-Rectangle	0	RW
3-2	NA	0	RO
1-0	**Draw Circle / Ellipse Curve Part Select (DECP)** 00b: bottom-left Ellipse Curve 01b: upper-left Ellipse Curve 10b: upper-right Ellipse Curve 11b: bottom-right Ellipse Curve	0	RW

REG[78h-77h] Draw Circle/Ellipse/Rounded-Rectangle Major-Radius Register (ELL_A)

Bit	Description	Default	Access
7-0	**Draw Circle/Ellipse/Rounded-rectangle Major-Radius** REG[77h] mapping to ELL_A[7:0] REG[78h] bit[4:0] mapping to ELL_A[12:8], bit[7:5] are not used. If drawing a circle, then the major radius must be set to equal to the minor radius (ELL_A = ELL_B).	0	RW

REG[7Ah-79h] Draw Circle/Ellipse/Rounded-rectangle Minor-Radius Register (ELL_B)

Bit	Description	Default	Access
7-0	**Draw Circle/Ellipse/Rounded-rectangle Minor-Radius** REG[79h] mapping to ELL_B[7:0] REG[7Ah] bit[4:0] mapping to ELL_B[12:8], bit[7:5] are not used.	0	RW

REG[7Ch-7Bh] Draw Circle/Ellipse/Rounded-Rectangle Center X-Coordinates Register (DEHR)

Bit	Description	Default	Access
7-0	**Draw Circle/Ellipse/Rounded-rectangle Center X-coordinates** REG[7Bh] mapping to DEHR[7:0] REG[7Ch] bit[4:0] mapping to DEHR[12:8], bit[7:5] are not used.	0	RW

REG[7Eh-7Dh] Draw Circle/Ellipse/Rounded-Rectangle Center Y-Coordinates Register (DEVR)

Bit	Description	Default	Access
7-0	**Draw Circle/Ellipse/Rounded-rectangle Center Y-coordinates** REG[7Dh] mapping to DEVR[7:0] REG[7Eh] bit[4:0] mapping to DEVR[12:8], bit[7:5] are not used.	0	RW

Note: The unit of REG[77h] ~ REG[7Eh] is Pixel.

REG[D2h] Foreground Color Register - Red (FGCR)

Bit	Description	Default	Access
7-0	**Foreground Color – Red (for Draw, Text or Color Expansion)** 256 Colors: Red mapping to bit[7:5] 65K Colors: Red mapping to bit[7:3] 16.7M Colors: Red mapping to bit[7:0]	FFh	RW

REG[D3h] Foreground Color Register - Green (FGCG)

Bit	Description	Default	Access
7-0	**Foreground Color – Green (for Draw, Text or Color Expansion)** ■ 256 Colors: Green mapping to bit[7:5] ■ 65K Colors: Green mapping to bit[7:2] ■ 16.7M Colors: Green mapping to bit[7:0]	FFh	RW

REG[D4h] Foreground Color Register - Blue (FGCB)

Bit	Description	Default	Access
7-0	**Foreground Color – Blue (for Draw, Text or Color Expansion)** ■ 256 Colors: Blue mapping to bit[7:6] ■ 65K Colors: Blue mapping to bit[7:3] ■ 16.7M Colors: Blue mapping to bit[7:0]	FFh	RW

Note: For Background color setting, please refer to the Text Engine Registers REG[D5h–D7h].

19.7 PWM Control Registers

REG[84h] PWM Prescaler Register (PSCLR)

Bit	Description	Default	Access
7-0	**PWM Pre-scaler Register** This register determines the pre-scaler value for Timer 0 and 1. The Base Frequency is: **Core_Freq / (Prescaler + 1)**	0	RW

REG[85h] PWM Clock Mux Register (PMUXR)

Bit	Description	Default	Access
7-6	**Setup PWM Timer-1 Divisor** (Select 2nd Clock Divider's MUX Input for PWM Timer-1) 00b = 1 01b = 1/2 10b = 1/4 11b = 1/8	0	RW
5-4	**Setup PWM Timer-0 Divisor** (Select 2nd Clock Divider's MUX Input for PWM Timer-0) 00b = 1 01b = 1/2 10b = 1/4 11b = 1/8	0	RW
3-2	**TFT_PWM[1] Function Control** 0xb: TFT_PWM[1] output system error flag (Scan FIFO POP error or Memory access out of range) 10b: TFT_PWM[1] output PWM timer 1 event or invert of PWM timer 0 11b: TFT_PWM[1] output Oscillator Clock	0	RW
1-0	**TFT_PWM[0] Function Control** 0xb: TFT_PWM[0] becomes GPIOC[7] 10b: TFT_PWM[0] output PWM Timer 0 11b: TFT_PWM[0] output Core Clock (CCLK).	0	RW

REG[86h] PWM Configuration Register (PCFGR)

Bit	Description	Default	Access
7	NA	0	RO
6	**PWM Timer-1 Output Inverter On/Off** Determine the output inverter on/off for Timer 1. 0 = Inverter off 1 = Inverter on for TFT_PWM[1]	0	RW
5	**PWM Timer-1 Auto Reload On/Off** Determine auto reload on/off for Timer 1. 0: One-Shot Mode 1: Interval Mode (Auto Reload)	1	RW
4	**PWM Timer-1 Start/Stop** 0: Stop 1: Start In Interval Mode, Host needs to program it as 0 to stop PWM timer. In One-shot Mode, this bit will be auto cleared. The Host may read this bit to find out if the current PWMx is running or stopped.	0	RW
3	**PWM Timer-0 Dead Zone Enable** 0: Disable 1: Enable	0	RW
2	**PWM Timer-0 Output Inverter On/Off** Determine the output inverter on/off for Timer 0. 0 = Inverter off 1 = Inverter on for TFT_PWM[0]	0	RW
1	**PWM Timer-0 Auto Reload On/Off** Determine auto reload on/off for Timer 0. 0: One-Shot Mode 1: Interval Mode (Auto Reload)	1	RW
0	**PWM Timer-0 Start/Stop** 0: Stop 1: Start In Interval Mode, Host needs program it as 0 to stop PWM timer. In One-shot Mode, this bit will be auto cleared. The Host may read this bit to find out if the current PWMx is running or stopped.	0	RW

REG[87h] Timer-0 Dead Zone Length Register [DZ_LENGTH]

Bit	Description	Default	Access
7-0	**Timer-0 Dead Zone Length Register** These 8 bits determine the dead zone length. The 1 unit time of the dead zone length is equal to Timer 0.	0	RW

REG[88h-89h] Timer-0 Compare Buffer Register [TCMPB0]

Bit	Description	Default	Access
7-0	**Timer-0 compare Buffer Register** REG[88h] mapping to TCMPB0 [7:0] REG[89h] mapping to TCMPB0 [15:8] The Timer-0 Compare Buffer Registers have a total of 16bits. When the counter is equal to or less than the value of this register, and if INV_ON bit is Off, then TFT_PWM[1] output is high level.	0	RW

REG[8Ah-8Bh] Timer-0 Count Buffer Register [TCNTB0]

Bit	Description	Default	Access
7-0	**Timer-0 Count Buffer Register [15:0]** REG[8Ah] mapping to TCNTB0 [7:0] REG[8Bh] mapping to TCNTB0 [15:8] The Timer-0 count registers have a total of 16bit. When the counter is equal to 0 and the Reload_EN is enabled, the PWM will reload the value of this register to the counter. When the PWM begins to count, the current count value can be read back through this register.	0	RW

REG[8Ch-8Dh] Timer-1 Compare Buffer Register [TCMPB1]

Bit	Description	Default	Access
7-0	**Timer-1 Compare Buffer Register** REG[8Ch] mapping to TCMPB1 [7:0] REG[8Dh] mapping to TCMPB1 [15:8] The Timer-1 Compare Buffer Registers have a total of 16bits. When the counter is equal to or less than the value of this register, and if INV_ON bit is Off, then TFT_PWM[1] output is high level.	0	RW

REG[8Eh-8Fh] Timer-1 Count Buffer Register [TCNTB1]

Bit	Description	Default	Access
7-0	**Timer-1 Count Buffer Register [15:0]** REG[8Eh] mapping to TCNTB1 [7:0] REG[8Fh] mapping to TCNTB1 [15:8] The Timer-1 count registers have a total of 16bit. When the counter is equal to 0 and the Reload_EN is enabled, the PWM will reload the value of this register to the counter. When the PWM begins to count, the current count value can be read back through this register.	0	RW

19.8 Bit Block Transfer Engine (BTE) Control Registers

REG[90h] BTE Function Control Register 0 (BLT_CTRL0)

Bit	Description	Default	Access
7-5	NA	0	RO
4	**BTE Function Enable / Status** ***Write:*** 0: No Action 1: BTE Enable ***Read:*** 0: BTE function is idle. 1: BTE function is busy. When BTE function is enabled, Host is not allowed to R/W memory through canvas[active window].	0	RW
3-1	NA	NA	NA
0	**Pattern Format** 0: 8*8 1: 16*16	0	RW

REG[91h] BTE Function Control Register1 (BLT_CTRL1)

<table>
<tr><th>Bit</th><th>Description</th><th>Default</th><th>Access</th></tr>
<tr><td>7-4</td><td>BTE ROP Code or Color Expansion Starting
ROP stands for Raster Operation. Some of BTE operation code has to collocate with ROP for advanced functions.

Table 19-3: BTE ROP Code
<table>
<tr><th>Bit[7:4]</th><th>Description</th></tr>
<tr><td>0000b</td><td>0 (Blackness)</td></tr>
<tr><td>0001b</td><td>~S0 · ~S1 or ~ (S0+S1)</td></tr>
<tr><td>0010b</td><td>~S0 · S1</td></tr>
<tr><td>0011b</td><td>~S0</td></tr>
<tr><td>0100b</td><td>S0 · ~S1</td></tr>
<tr><td>0101b</td><td>~S1</td></tr>
<tr><td>0110b</td><td>S0^S1</td></tr>
<tr><td>0111b</td><td>~S0+~S1 or ~ (S0 · S1)</td></tr>
<tr><td>1000b</td><td>S0 · S1</td></tr>
<tr><td>1001b</td><td>~ (S0^S1)</td></tr>
<tr><td>1010b</td><td>S1</td></tr>
<tr><td>1011b</td><td>~S0+S1</td></tr>
<tr><td>1100b</td><td>S0</td></tr>
<tr><td>1101b</td><td>S0+~S1</td></tr>
<tr><td>1110b</td><td>S0+S1</td></tr>
<tr><td>1111b</td><td>1 (Whiteness)</td></tr>
</table>
If BTE operation code function is color expansion with or without chroma key (08h / 09h / Eh / Fh), then these bits stand for starting bit on BTE window left boundary. MSB stands for left most pixel. For 8-bits Host, the value should be within 0 to 7. For 16-bits Host, the value should be within 0 to 15.</td><td>0</td><td>RW</td></tr>
<tr><td>3-0</td><td>BTE Operation Code bit[3:0]
LT7689 has an embedded a 2D BTE Engine, it can execute 13 BTE functions. Some of BTE Operation Code has to accommodate with the ROP code for the advanced function.</td><td>0</td><td>RW</td></tr>
</table>

Table 19-4: BTE Operation Code

REG[91h] Bit[3:0]	Description
0000b	**MCU Write with ROP** S0: Data comes from Host S1: Data comes from Memory D: According to ROP and write to Memory
0001b	Reserved
0010b	**Memory Copy with ROP** S0: Data comes from Memory S1: Data comes from Memory D: According to ROP and Write to memory
0011b	Reserved
0100b	**MCU Write with Chroma Keying (without ROP)** S0: Data comes from Host If the Host data is not the same color as the Chroma Key (Background Color Register), then the data will be written to the destination memory.
0101b	**Memory Copy (move) with Chroma keying (without ROP)** S0: Data comes from Memory If the S0 data is not the same color as the chroma key Background Color Register), then the data will be written to the destination.
0110b	**Pattern Fill with ROP** Pattern is specified by S0.
0111b	**Pattern Fill with Chroma Keying** Pattern is specified by S0. If the data for S0 is not the same as the Chroma Key (Background Color) color, then it will be written to the destination memory.
1000b	**MCU Write with Color Expansion** S0: Data comes from Host BTE converts it to the specified color and color depth, and writes it to the destination memory.
1001b	**MCU Write with Color Expansion and Chroma Keying** The monochrome data required by S0 is written by MCU if the bit of monochrome data is 1. The processed data is a foreground color, and if the monochrome data is 0, it is not written. The data is also referenced to the destination memory setting.
1010b	**Memory Copy with Opacity** S0, S1 & D: Data come from Memory

REG[91h] Bit[3:0]	Description
1011b	**MCU Write with Opacity** S0: Data comes from Host S1: Data comes from Memory D: Refer to the alpha blending operation and write to the destination memory.
1100b	**Solid Fill** The written value is the register setting value, and the written target is the destination memory.
1101b	Reserved
1110b	**Memory Copy with Color Expansion** S0 and D are in memory and S1 is not in use. S0 must be loaded through the microprocessor's write or DMA to preload 8bpp or 16bpp color depth into memory, so the S0 color depth should follow that color depth.
1111b	**Memory Copy with Color Expansion and Chroma Keying** S0 and D are in memory and S1 is not in use. S0 must be loaded through the microprocessor's write or DMA preload 8bpp or 16bpp color depth into memory, so the S0 color depth should follow that color depth. If the S0 data bit = 0, then no data will be written to D. If the S0 data bit = 1, the Foreground Color data will be written to D.

REG[92h] Source 0/1 & Destination Color Depth (BLT_COLR)

Bit	Description	Default	Access
7	NA	0	RO
6-5	**S0 Color Depth** 00b: 256 Color (8bpp) 01b: 64k Color (16bpp) 1xb: 16M Color (24bpp)	0	RW
4-2	**S1 Color Depth** 000b: 256 Color (8bpp) 001b: 64k Color (16bpp) 010b: 16M Color (24bpp) 011b: Constant color (S1 memory start address' setting must be defined as S1 constant color definition) 100b: 8 bit pixel alpha blending 101b: 16 bit pixel alpha blending	0	RW
1-0	**Destination Color Depth** 00b: 256 Color (8bpp) 01b: 64k Color (16bpp) 1xb: 16M Color (24bpp)	0	RW

REG[93h-96h] Source 0 Memory Start Address (S0_STR)

Bit	Description	Default	Access
7-0	**Source 0 Memory Start Address [31:2]** REG[93h] mapping to S0_STR [7:2] REG[94h] mapping to S0_STR [15:8] REG[95h] mapping to S0_STR [23:16] REG[96h] mapping to S0_STR [31:24] **Note:** REG[93h] bit[1:0] is fixed to 0.	0	RW

REG[97h-98h] Source 0 Image Width (S0_WTH)

Bit	Description	Default	Access
7-0	**Source 0 Image Width [12:2]** REG[97h] mapping to S0_WTH [7:2] REG[98h] bit[4:0] mapping to S0_WTH [12:8], bit[7-5] are not used. **Note:** It must be divisible by 4, and REG[97h] bit[1:0] must be fixed to 0. Unit: Pixel.	0	RW

REG[99h-9Ah] Source 0 Window Upper-Left Corner X-Coordinates (S0_X)

Bit	Description	Default	Access
7-0	**Source 0 Window Upper-Left Corner X-Coordinates [12:0]** REG[99h] mapping to S0_X [7:0] REG[9Ah] bit[4:0] mapping to S0_X [12:8], bit[7-5] are not used.	0	RW

REG[9Bh-9Ch] Source 0 Window Upper-Left corner Y-Coordinates (S0_Y)

Bit	Description	Default	Access
7-0	**Source 0 Window Upper-Left Corner Y-Coordinates [12:0]** REG[9Bh] mapping to S0_Y [7:0] REG[9Ch] bit[4:0] mapping to S0_Y[12:8], bit[7-5] are not used.	0	RW

REG[9Dh-A0h] Source 1 Memory Start Address 0 (S1_STR)

Bit	Description	Default	Access
7-0	**Source 1 Memory Start Address [31:2]** REG[9Dh] bit[7:2] mapping to S1_STR[7:2] REG[9Eh] mapping to S1_STR[15:8] REG[9Fh] mapping to S1_STR[23:16] REG[A0h] mapping to S1_STR[31:24] **Note1:** REG[9Dh] bit[1:0] must be fixed to 0. **Note2:** If source 1(S0) is set as Constant Color then S1 memory start address' setting will be defined as S1 constant color definition: REG[9Dh] is RED element (S1_RED), REG[9Eh] is Green element (S1_GREEN), REG[9Fh] is Blue Element (S1_BLUE), REG[A0h] is not used.	0	RW

REG[A1h-A2h] Source 1 Image Width (S1_WTH)

Bit	Description	Default	Access
7-0	**Source 1 Image Width [12:2]** REG[A1h] [7:2] mapping to S1_WTH [7:2] REG[A2h] bit[4:0] mapping to S1_WTH[12:8], bit[7:5] are not used. **Note:** It must be divisible by 4, and REG[A1h] bit[1:0] must be fixed to 0. Unit: Pixel.	0	RW

REG[A3h-A4h] Source 1 Window Upper-Left Corner X-Coordinates (S1_X)

Bit	Description	Default	Access
7-0	Source 1 Window Upper-Left Corner X-Coordinates [12:0] REG[A3h] mapping to S1_X [7:0] REG[A4h] bit[4:0] mapping to S1_X [12:8], bit[7:5] are not used.	0	RW

REG[A5h-A6h] Source 1 Window Upper-Left corner Y-Coordinates (S1_Y)

Bit	Description	Default	Access
7-0	**Source 1 Window Upper-Left Corner Y-Coordinates [12:0]** REG[A5h] mapping to S1_Y [7:0] REG[A6h] bit[4:0] mapping to S1_Y [12:8], bit[7:5] are not used.	0	RW

REG[A7h-AAh] Destination Memory Start Address (DT_STR)

Bit	Description	Default	Access
7-2	**Destination Memory Start Address [31:2]** REG[A7h] bit[7:2] mapping to DT_STR [7:2] REG[A8h] mapping to DT_STR [15:8] REG[A9h] mapping to DT_STR [23:16] REG[AAh] mapping to DT_STR [31:24] **Note:** REG[A7h] bit[1:0] must be fixed to 0.	0	RW

Note: The destination memory start address cannot be within the source 0 or Source 1 block, otherwise there will be an incorrect output result.
Start Address ~ S0/1's (Image_Width)* (Image_Height)* ([1|2|3]Color Depth)

REG[ABh-ACh] Destination Image Width (DT_WTH)

Bit	Description	Default	Access
7-0	**Destination Image Width [12:2]** REG[ABh] mapping to DT _WTH [7:2] REG[ACh] bit[4:0] mapping to DT _WTH [12:8], REG[ACh] bit[7-5] are not used. **Note:** It must be divisible by 4. And REG[ABh] bit[1:0] must be fixed to 0. Unit: Pixel.	0	RW

REG[ADh-AEh] Destination Window Upper-Left Corner X-Coordinates (DT_X)

Bit	Description	Default	Access
7-0	**Destination Window Upper-Left Corner X-Coordinates [12:0]** REG[ADh] mapping to DT_X [7:0] REG[AEh] bit[4:0] mapping to DT_X [12:8], bit[7-5] are not used.	0	RW

REG[AFh-B0h] Destination Window Upper-Left Corner Y-Coordinates (DT_Y)

Bit	Description	Default	Access
7-0	**Destination Window Upper-Left Corner X-Coordinates [12:0]** REG[AFh] mapping to DT_Y [7:0] REG[B0h] bit[4:0] mapping to DT_Y [12:8], bit[7-5] are not used.	0	RW

REG[B1h-B2h] BTE Window Width (BLT _WTH)

Bit	Description	Default	Access
7-0	**BTE Window Width [12:0]** REG[B1h] mapping to BLT_WTH [7:0] REG[B2h] bit[4:0] mapping to BLT_WTH [12:8], bit[7-5] are not used. When all Image Fill (Pattern Fill) functions for BTE are enabled, the BTE window Width is ignored and automatically set to 8 or 16. Unit: Pixel.	0	RW

REG[B3h-B4h] BTE Window Height (BLT_HIG)

Bit	Description	Default	Access
7-0	**Destination Image Height [12:0]** REG[B3h] mapping to BLT_HIG [7:0] REG[B4h] bit[4:0] mapping to BLT_HIG [12:8], bit[7-5] are not used. When all Image Fill (Pattern Fill) functions for BTE are enabled, the BTE window Height is ignored and automatically set to 8 or 16. Unit: Pixel.	0	RW

REG[B5h] Alpha Blending (APB_CTRL)

Bit	Description	Default	Access
7-4	NA	0	RO
5-0	**Window Alpha Blending Effect for S0 & S1** The value of alpha in the color code ranges from 1.0 down to 0.0, where 1.0 represents a fully opaque color, and 0.0 represents a fully transparent color. 00h: 0 01h: 1/32 02h: 2/32 : : 1Eh: 30/32 1Fh: 31/32 2Xh: 1 **Output Effect** **=〔S0 image * (1 - Alpha Setting Value)〕+** **(S1 Image * Alpha Setting Value)**	0	RW

19.9 Serial Flash & SPI Master Control Registers

REG[B6h] Serial Flash DMA Controller REG (DMA_CTRL)

Bit	Description	Default	Access
7-1	NA	0	RO
0	***Write:*** **DMA Start Bit** This bit can be written to 1 via Host, and the circuit will be automatically cleared to 0 right away. This bit cannot be used with Text Write, so if DMA is enabled, it cannot be set to text mode to input character code. ***Read:*** **DMA Busy Check Bit** 0: Idle 1: Busy The DMA transmission of a Serial Flash must be in Graphic Mode, and it must first set the canvas destination location, destination width, color depth, and addressing mode in the Display RAM.	0	RW

REG[B7h] Serial Flash/ROM Controller Register (SFL_CTRL)

Bit	Description	Default	Access
7	**Serial Flash/ROM Select** 0: Serial Flash/ROM 0 I/F is selected 1: Serial Flash/ROM 1 I/F is selected	0	RW
6	**Serial Flash /ROM Access Mode** 0: Text Mode, used for CGRAM 1: DMA Mode, used for CGRAM, Pattern, Boot Start Image or OSD.	0	RW
5	**Serial Flash/ROM Address Mode** 0: 24bits address mode 1: 32bits address mode If users want to use 32 bits address mode, Host must send EX4B command (B7h) to serial flash and set this bit to 1. Host also may check this bit to find out whether serial flash entered 32 bits address mode during initial display operation.	0	RW
4	NA	0	RW
3-0	**Read Command Code & Behavior Selection** 000xb: 1x Read Command - 03h. Normal Read Speed. Data is read from SFDI. Without dummy cycle between address and data 010xb: 1x Read Command - 0Bh. Faster Read speed. Data is read from SFDI. LT7689 inserts 8 dummy cycles between address and data. 1x0xb: 1x Read Command - 1Bh. Fastest Read Speed. Data read from SFDI. LT7689 inserts 16 dummy cycles between address and data.	0	R/W

Bit	Description	Default	Access
	xx10b: 2x Read Command - 3Bh. Interleaved data input from SFDI and SFDO. LT7689 inserts 8 dummy cycles (Dual Mode 0) between address and data. Refer to Figure 17-8. xx11b: 2x Read Command – BBh. Interleaved data input from SFDI and SFDO. LT7689 inserts 4 dummy cycles (Dual Mode 1) between address and data. Refer to Figure 17-9. **Note:** Not all of Serial Flash support above read command. Please refer to Serial Flash's datasheet to select proper read command.		

REG[B8h] SPI Master Tx /Rx FIFO Data Register (SPIDR)

Bit	Description	Default	Access
7-0	**SPI Master Tx /Rx FIFO Data Register** After Host programming the LT7689's control register, SPI transfers can be initiated. A transfer is initiated by writing to the Serial Peripheral Data Register [SPIDR]. Writing to the Serial Peripheral Data Register is actually writing to a 16 entries deep FIFO called the Write FIFO. Each write access adds a data byte to the Write FIFO. When the core is enabled – SS_ACTIVE is set to '1' – and the Write FIFO is not full, the core automatically transfers the oldest data byte. Receiving data is done simultaneously with transmitting data; whenever a data byte is transmitted a data byte is received. For each byte that needs to be read from a device, a dummy byte needs to be written to the Write FIFO. This instructs the core to initiate an SPI transfer, simultaneously transmitting the dummy byte and receiving the desired data. Whenever a transfer is finished, the received data byte is added to the Read FIFO. The Read FIFO is the counterpart of the Write FIFO. It is an independent 16 entries deep FIFO. The FIFO contents can be read by reading from the Serial Peripheral Data Register [SPIDR]	NA	RW

REG[B9h] SPI Master Control Register (SPIMCR2)

<table>
<tr><th>Bit</th><th>Description</th><th>Default</th><th>Access</th></tr>
<tr><td>7</td><td>NA</td><td>0</td><td>RO</td></tr>
<tr><td>6</td><td>SPI Master Interrupt Enable
0: Disable interrupt.
1: Enable interrupt.
If Host disables SPI Master interrupt flag, then LT7689 will not assert interrupt event to inform Host, but Host still may check interrupt event on SPIMSR.</td><td>0</td><td>RW</td></tr>
<tr><td>5</td><td>Control Slave Select Drive on Which SFCS0# / SFCS1#
0: Slave Select Signal (SS#) drive on SFCS0#
1: Slave Select Signal (SS#) drive on SFCS1#</td><td>0</td><td>RW</td></tr>
<tr><td>4</td><td>Slave Select Signal Active [SS_ACTIVE]
0: Inactive (SS# drive High)
1: Active (SS# drive Low)
While Slave Select signal is inactive, FIFO will be cleared, and engine will stay in idle state.
Note: It is suggested when Slave Select Signal is active, the settings of CPOL/CPHA should not be changed.</td><td>0</td><td>RW</td></tr>
<tr><td>3</td><td>Mask Interrupt for FIFO Overflow Error [OVFIRQMSK]
0: Unmask.
1: Mask.</td><td>1</td><td>RW</td></tr>
<tr><td>2</td><td>Mask Interrupt for While Tx FIFO Empty & SPI Engine/FSM Idle [EMTIRQMSK]
0: Unmask.
1: Mask.</td><td>1</td><td>RW</td></tr>
<tr><td>1:0</td><td>SPI Operation Mode
When DMA or external CGROM is enabled, only mode 0 & mode will be supported.

Table 19-5: SPI Operation Mode
<table>
<tr><th>Mode</th><th>CPOL:
Clock Polarity Bit</th><th>CPHA:
Clock Phase Bit</th></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>2</td><td>1</td><td>0</td></tr>
<tr><td>3</td><td>1</td><td>1</td></tr>
</table></td><td>0</td><td>RW</td></tr>
</table>

- Please refer to the Section 10.2 for SPI Master description.
- When CPOL = 0, SCK is 0 when it is inactive.

  _ CPHA = 0: data is read on rising edge (low->high). And data are changed on a falling edge (high->low).

_ CPHA = 1: data is read on falling edge (high->low) . And data are changed on a rising edge (low->high).

- When CPOL = 1, SCK is 1 when it is inactive (inversion of CPOL = 0)

_ CPHA = 0: data is read on falling edge (high->low). And data are changed on a rising edge (low->high).

_ CPHA = 1: data is read on rising edge (low->high). And data are changed on a falling edge (high->low).

REG[BAh] SPI Master Status Register (SPIMSR)

Bit	Description	Default	Access
7	**Tx FIFO Empty Flag** 0: Not Empty 1: Empty	1	RO
6	**Tx FIFO Full Flag** 0: Not Full 1: Full	0	RO
5	**Rx FIFO Empty Flag** 0: Not empty 1: Empty	1	RO
4	**Rx FIFO Full Flag** 0: Not Full 1: Full	0	RO
3	**Overflow Interrupt Flag** Write 1 will clear this flag	0	RW
2	**Tx FIFO Empty & SPI Engine/FSM Idle Interrupt Flag** Write 1 will clear this flag	0	RW
1-0	NA	0	RO

REG[BBh] SPI Clock period (SPI_DIVSOR)

Bit	Description	Default	Access
7-0	**SPI Clock Period** Set low & high period for SPI clock based on the system clock. **$F_{SCK} = F_{CORE}$ / (Divisor + 1)*2**	3	RW

REG[BCh-BFh] Serial Flash DMA Source Starting Address (DMA_SSTR)

Bit	Description	Default	Access
7-0	**Serial Flash DMA Source Start Address[31:0]** REG[BCh] mapping to DMA_SSTR[7:0] REG[BDh] mapping to DMA_SSTR[15:8] REG[BEh] mapping to DMA_SSTR[23:16] REG[BFh] mapping to DMA_SSTR[31:24] These registers index Serial Flash Address [31:0]. Directly point to the starting position of the source images.	0	RW

REG[C0h-C1h] DMA Destination Window Upper-Left Corner X-Coordinates (DMA_DX)

Bit	Description	Default	Access
7-0	**When REG 5Eh (AW_COLOR) bit 2 = 0 (Block Mode):** This register defines DMA Destination Window Upper-Left corner X-coordinates [12:0] on Canvas area. REG[C0h] mapping to DMA_DX[7:0] REG[C1h] bit[4:0] mapping to DMA_DX[12:8], bit[7:5] are not used. **When REG 5Eh (AW_COLOR) bit 2 = 1 (Linear Mode):** This register defines Destination address [15:2] in Display RAM. REG[C0h] bit[7:2] mapping to DMA_DX[7:2] REG[C1h] mapping to DMA_DX[15:8]	0	RW

REG[C2h-C3h] DMA Destination Window Upper-Left Corner Y-Coordinates (DMA_DY)

Bit	Description	Default	Access
7-0	**When REG 5Eh (AW_COLOR) bit 2 = 0 (Block Mode):** This register defines DMA Destination Window Upper-Left corner Y-coordinates [12:0] on Canvas area. REG[C2h] mapping to DMA_DY[7:0] REG[C3h] bit[4:0] mapping to DMA_DY[12:8], bit[7:5] are not used. **When REG 5Eh (AW_COLOR) bit 2 = 1 (Linear Mode):** This register defines Destination address [31:16] in Display RAM. REG[C2h] mapping to DMA_DY[23:16] REG[C3h] mapping to DMA_DY[31:24]	0	RW

REG[C4h] – REG[C5h]: Reserved

Bit	Description	Default	Access
7-0	NA	0	RO

REG[C6h-C7h] DMA Block Width (DMAW_WTH)

Bit	Description	Default	Access
7-0	**When REG 5Eh (AW_COLOR) bit 2 = 0 (Block Mode):** **DMA Block Width [15:0]** REG[C6h] mapping to DMAW_WTH[7:0] REG[C7h] mapping to DMAW_WTH[15:8] **When REG 5Eh (AW_COLOR) bit 2 = 1 (Linear Mode):** **DMA Transfer Number [15:0]** REG[C6h] mapping to DMAW_WTH[7:0] REG[C7h] mapping to DMAW_WTH[15:8]	0	RW

REG[C8h-C9h] DMA Block Height (DMAW_HIGH)

Bit	Description	Default	Access
7-0	**When REG 5Eh (AW_COLOR) bit 2 = 0 (Block Mode):** **DMA Block Height [15:0]** REG[C8h] mapping to DMAW_HIGH[7:0] REG[C9h] mapping to DMAW_HIGH[15:8] **When REG 5Eh (AW_COLOR) bit 2 = 1 (Linear Mode):** **DMA Transfer Number [31:16]** REG[C8h] mapping to DMAW_HIGH [23:16] REG[C9h] mapping to DMAW_HIGH [31:24]	0	RW

REG[CAh-CBh] DMA Source Picture Width (DMA_SWTH)

Bit	Description	Default	Access
7-0	**DMA Source Picture Width [12:0]** REG[CAh] mapping to DMA_SWTH[7:0] REG[CBh] bit[4:0] mapping to DMA_SWTH[12:8], bit[7:5] are not used. Unit: Pixel.	0	RW

Please refer to Section 10.3 description of SPI Flash Controller.

19.10 Text Engine Registers

REG[CCh] Character Control Register 0 (CCR0)

Bit	Description	Default	Access
7:6	**Character Source Selection** 00b: Select internal CGROM Character 01b: Select external CGROM Character 10b: Select user-defined Character. 11b: NA	0	RW
5-4	**Character Height Setting for user-defined Character** 00b: 16 dots, ex. 8*16 / 16*16 01b: 24 dots, ex. 12*24 / 24*24 10b: 32 dots, ex. 16*32 / 32*32 **Note**: 1. User-defined character width is decided by character code; width for code < 8000h is 8/12/16. width for code >=8000h is 16/24/32. 2.Internal CGROM supports sizes of 8*16 / 12*24 / 16*32.	0	RW
3-2	N	0	RO
1-0	**Character Selection for Internal CGROM** When bit[7:6] = 00b, Internal CGROM supports character sets with the standard coding of ISO/IEC 8859-1,2,4,5 which supports English and most of European country languages 00b: ISO/IEC 8859-1 01b: ISO/IEC 8859-2 10b: ISO/IEC 8859-4 11b: ISO/IEC 8859-5	0	RW

REG[CDh] Character Control Register 1 (CCR1)

Bit	Description	Default	Access
7	**Full Alignment Selection** 0: Full alignment disable 1: Full alignment enable. When Full alignment enable, displayed character width is equal to (Character Height)/2 if character width is equal to or smaller than (Character Height)/2, otherwise displayed font width is equal to Character Height.	0	RW
6	**Chroma Keying Enable on Text Input** 0: Character's background displayed with specified color. 1: Character's background displayed with original canvas' background.	0	RW
5	NA	0	RO

Bit	Description	Default	Access
4	**Character Rotation** 0: Normal. Text direction from left to right then from top to bottom 1: Counterclockwise 90 degree & vertical flip. Text direction from top to bottom then from left to right (it should accommodate with setting VDIR as 1). This attribute can be changed only when previous Text write is finished (Core_Busy = 0)	0	RW
3-2	**Character Width Enlargement Factor** 00b: X1. 01b: X2. 10b: X3. 11b: X4.	0	RW
1-0	**Character Height Enlargement Factor** 00b: X1. 01b: X2. 10b: X3. 11b: X4.	0	RW

REG[CEh-CFh] RESERVED

Bit	Description	Default	Access
7-0	NA	0	RO

REG[D0h] Character Line gap Setting Register (FLDR)

Bit	Description	Default	Access
7-5	NA	0	RO
4-0	**Character Line Gap Setting** Setting the character line gap. When entered character reaches the boundary of the active window, jump to the next line. (Unit: pixel) Color of gap will be the same as the background color. It will not be enlarged by character enlargement function.	0	RW

REG[D1h] Character to Character Space Setting Register (F2FSSR)

Bit	Description	Default	Access
7-6	NA	0	RW
5-0	**Character to Character Space Setting** 00h: 0 pixel 01h: 1 pixel 02h: 2 pixels : : 3Fh: 63 pixels Color of space will be the same as the background color. It will not be enlarged by character enlargement function.	0	RW

REG[D2h] Foreground Color Register - Red (FGCR)

Bit	Description	Default	Access
7-0	**Foreground Color – Red (for Draw, Text or Color Expansion)** ■ 256 Colors: Red mapping to bit[7:5] ■ 65K Colors: Red mapping to bit[7:3] ■ 16.7M Colors: Red mapping to bit[7:0]	FFh	RW

REG[D3h] Foreground Color Register - Green (FGCG)

Bit	Description	Default	Access
7-0	**Foreground Color – Green (for Draw, Text or Color Expansion)** ■ 256 Colors: Green mapping to bit[7:5] ■ 65K Colors: Green mapping to bit[7:2] ■ 16.7M Colors: Green mapping to bit[7:0]	FFh	RW

REG[D4h] Foreground Color Register - Blue (FGCB)

Bit	Description	Default	Access
7-0	**Foreground Color – Blue (for Draw, Text or Color Expansion)** ■ 256 Colors: Blue mapping to bit[7:6] ■ 65K Colors: Blue mapping to bit[7:3] ■ 16.7M Colors: Blue mapping to bit[7:0]	FFh	RW

REG[D5h] Background Color Register - Red (BGCR)

Bit	Description	Default	Access
7-0	**Background Color – Red (for Text or Color Expansion)** ■ 256 Colors: Red mapping to bit[7:5] ■ 65K Colors: Red mapping to bit[7:3] ■ 16.7M Colors: Red mapping to bit[7:0] **Note:** No matter background transparency is enabled or not, please do not set to the same value as Foreground Color, otherwise, image and text will become a square with Foreground Color. When using BTE function, these two values should not be set to the same either.	00h	RW

REG[D6h] Background Color Register - Green (BGCG)

Bit	Description	Default	Access
7-0	**Background Color –Green (for Text or Color Expansion)** ■ 256 Colors: Green mapping to bit[7:5] ■ 65K Colors: Green mapping to bit[7:2] ■ 16.7M Colors: Green mapping to bit[7:0] **Note:** No matter background transparency is enabled or not, please do not set to the same value as Foreground Color, otherwise, image and text will become a square with Foreground Color. When using BTE function, these two values should not be set to the same either.	00h	RW

REG[D7h] Background Color Register - Blue (BGCB)

Bit	Description	Default	Access
7-0	**Background Color –Blue (for Text or Color Expansion)** ■ 256 Colors: Blue mapping to bit[7:6] ■ 65K Colors: Blue mapping to bit[7:3] ■ 16.7M Colors: Blue mapping to bit[7:0] **Note:** No matter background transparency is enabled or not, please do not set to the same value as Foreground Color, otherwise, image and text will become a square with Foreground Color. When using BTE function, these two values should not be set to the same either.	00h	RW

REG[D8h] – REG[DAh]: Reserved

Bit	Description	Default	Access
7-0	NA	0	RO

REG[DBh] CGRAM Start Address 0 (CGRAM_STR0)

Bit	Description	Default	Access
7-0	**CGRAM Start ADDRESS [7:0]** User-defined Characters space REG[DBh] mapping to CGRAM_STR [7:0] REG[DCh] mapping to CGRAM_STR [15:8] REG[DEh] mapping to CGRAM_STR [23:16] REG[DEh] mapping to CGRAM_STR [31:24] Host must save CGRAM data based on the setting of canvas, and set CGRAM address to tell engine where to fetch CGRAM data.	0	RW

If users want to change rotate attribute, character line gap, character-to-character space, foreground color, background color and Text/graphic mode settings, please make sure Task_Busy (Status Register bit3) status bit is low.

19.11 Power Management Control Register

REG[DFh]: Power Management Register (PMU)

Bit	Description	Default	Access
7	**Enter Power Saving State** 0: Normal state or wake up from power saving state 1: Enter power saving state. **Note:** There are 3 ways to wake up from power saving state: External interrupt event, Key Scan wakeup, and Software wakeup. Write this bit to 0 will generate Software wakeup. It will not be cleared until chip resume. MCU must wait until system is awekened before writting to other registers. Users may check this bit or check status bit1 (power saving) to find out if the system is back to the normal state.	0	RW
6-2	NA	0	RO
1-0	**Power Saving Mode Definition** 00b: NA 01b: Standby Mode, CCLK & PCLK will Stop. MCLK is provided by MPLL. 10b: Suspend Mode, CCLK & PCLK will Stop. MCLK is provided by OSC. 11b: Sleep Mode, All of Clocks and PLL will Stop.	3	RW

19.12 Display RAM Control Register

REG[E0h] SDRAM Attribute Register (SDRAR)

Bit	Description	Default	Access
7	**SDRAM Power Saving** 0: Execute power down command to enter power saving mode 1: Execute self refresh command to enter power saving mode	0	RW
6	Must keep 0.	0	RW
5	**SDRAM Bank Number, SDR_BANK** 0: 2 Banks (column addressing size only support 256 words) 1: 4 Banks	1	RW
4-3	**SDRAM Row Addressing, SDR_ROW** 00b: 2K (A0-A10) 01b: 4K (A0-A11) 1xb: 8K (A0-A12)	1	RW
2-0	**SDRAM Column Addressing, SDR_COL** 000b: 256 (A0-A7) 001b: 512 (A0-A8) 010b: 1,024 (A0-A9) 011b: 2,048 (A0-A9, A11) 1xxb: 4,096 (A0-A9, A11-A12)	0	RW

Table 19-6: The initialize of REG[E0h]

Model	Embedded Display RAM Type	REG[E0h]	Description
LT7689	128Mb, 16MB, 8M*16	**0x29**	Bank no: 4, Row Size: 4096, Column Size: 512

Note: The value of register REG[E0h] must be set according to the LT7689 model name. Otherwise, the display of TFT panel will be abnormal and the image will be garbled.

REG[E1h] SDRAM Mode Register & Extended Mode Register (SDRMD)

Bit	Description	Default	Access
7-3	Must keep 0.	0	RW
2-0	**SDRAM CAS latency, SDR_CASLAT** 010b: 2 SDRAM clock 011b: 3 SDRAM clock Others: Reserved **Note:** The suggest setting value of this register is **03h.** This register is locked after SDR_INITDONE (REG[E4h] bit0) is set as 1.	011b	RW

REG[E2h-E3h] SDRAM Auto Refresh Interval (SDR_REF)

Bit	Description	Default	Access
7-0	**SDRAM Auto Refresh Interval** REG[E2h] mapping to SDR_REF [7:0]. REG[E3h] mapping to SDR_REF [15:8] The internal refresh time is determined by the refresh cycle of the SDRAM and the row size. For example, if the SDRAM frequency is 100MHz, SDRAM's refresh cycle Tref is 64ms, and the row size is 4,096, then the internal refresh time should be less than $64*10^{-3} / 4096 * 100*10^{6}$ ~= 1562 = 61Ah.. Therefore the REG[E3h][E2h] is set to 061Ah. **Note:** If this register is set to 0000h, SDRAM automatic refresh will be prohibited.	00h	RW

Table 19-7: The Reference Setting of REG[E3h-E2h]

Model	REG[E3h]	REG[E2h]
LT7689	06h	1Ah

REG[E4h] SDRAM Control Register (SDRCR)

Bit	Description	Default	Access
7-4	Must keep 0.	0	RW
3	**Report Warning Condition** 0: Disable or Clear warning flag 1: Enable warning flag Warning conditions include (1) memory read cycle is close to the maximum address boundry of SDRAM (exceeding maximum address minus 512 bytes); (2) out of memory access range; (3) insufficient SDRAM bandwidth to fulfill panel's frame rate. Under the above situation, the warning event will be latched. The warning flag could be cleared by setting this bit to 0.	0	RW
2	**SDRAM Timing Parameter Register Enable, SDR_PARAMEN** 0: Disable Display RAM timing parameter registers 1: Enable Display RAM timing parameter registers	0	RW
1	**Enter Power Saving Mode, SDR_PSAVING** 0 to 1 transition will enter power saving mode 1 to 0 transition will exit power saving mode	0	RW
0	**Start SDRAM Initialization Procedure, SDR_INITDONE** 0 to 1 transition will execute Display RAM initialization procedure. Read value '1' means Display RAM is initialized and ready for access. Once it is written to 1, it cannot be rewritten to 0. 1 to 0 transition: No operation. "Write 1" will execute Display RAM initialization procedure.	0	RW

Note: The following Display RAM Timing Registers (REG[E0h-E3h]) are available only when SDR_PARAMEN (REG[E4] bit2) is set to 1.

REG[E0h] SDRAM Timing Parameter 1

Bit	Description	Default	Access
7-4	NA	0	RO
3-0	**Time from Load Mode Command to Active/Refresh Command (T_{MRD})** 0000b: 1 SDRAM Clock 0001b: 2 SDRAM Clock 0010b: 3 SDRAM Clock : : 1111b: 16 SDRAM Clock	2	RW

REG[E1h] SDRAM Timing Parameter 2

Bit	Description	Default	Access
7-4	**Auto Refresh Period, T_{RFC}** 0h – Fh: 1 ~ 16 SDRAM Clock (As REG[E0h] bit[3:0])	8	RW
3-0	**Time of Exit SELF Refresh-to-ACTIVE Command (T_{XSR})** 0h – Fh: 1 ~ 16 SDRAM Clock	7	RW

REG[E2h] SDRAM Timing Parameter 3

Bit	Description	Default	Access
7-4	**Time of Pre-charge Command Period (T_{RP}, 15/20ns)** 0h – Fh: 1 ~ 16 SDRAM Clock	2	RW
3-0	**Time of WRITE Recovery Time (T_{WR})** 0h – Fh: 1 ~ 16 SDRAM Clock	0	RW

REG[E3h] SDRAM Timing Parameter 4

Bit	Description	Default	Access
7-4	**Delay Time of Active-to-Read/Write (T_{RCD})** 0h – Fh: 1 ~ 16 SDRAM Clock	2	RW
3-0	**Time of Active-to-Precharge (T_{RAS})** 0h – Fh: 1 ~ 16 SDRAM Clock	6	RW

19.13 GPIO Register

REG[F0h] GPIO-A Direction (GPIOAD)

Bit	Description	Default	Access
7	**GPIOA[7] In/Out Control** 0: Output. 1: Input.	FFh	RW
6-0	NA	NA	NA

REG[F1h] GPIO-A (GPIOA)

Bit	Description	Default	Access
7	**GPIOA[7] Data** Write: Output Data to GPIOA Read: Read Data from GPIOA GPIOA[7] is a General Purpose I/O. These signals are shared with DB[15:8].	NA	RW
6-0	NA	NA	NA

REG[F2h] GPIO-B (GPIOB)

Bit	Description	Default	Access
7-0	NA	NA	NA

REG[F3h] GPIO-C Direction (GPIOCD)

Bit	Description	Default	Access
7-0	**GPIOC In/Out Control** 0: Output 1: Input.	FFh	RW

REG[F4h] GPIO-C (GPIOC)

Bit	Description	Default	Access
7	**GPIOC[7] Data** Write: Output Data to GPIOC[7] Read: Read Data from GPIOC[7] GPIOC[7] is shared with PWM[0]. GPIOC is available only when PWM and SPI Master functions disabled.	NA	RW
6-5	NA	NA	RW
4-0	**GPIOC[4:0] Data** Write: Output Data to GPIOC[4:0] Read: Read Data from GPIOC[4:0] GPIOC[4:0] are shared with { SFCS1#, SFCS0#, SFDI, SFDO, SFCLK }. They are available when PWM and SPI Master functions disabled.	NA	RW

REG[F5h] GPIO-D Direction (GPIODD)

Bit	Description	Default	Access
7-0	**GPIOD In/Out Control** 0: Output 1: Input	FFh	RW

REG[F6h] GPIO-D (GPIOD)

Bit	Description	Default	Access
7-6	**GPIOD Data** Write: Output Data to GPIOD[7:0] Read: Read Data from GPIOD[7:0] GPIOD[7:6] are shared with PD[18, 2]. GPIOD[7,6] are available when LCD panel data bus is 16bits.	NA	RW
5-0	NA	NA	NA

REG[F7h-FFh]: Reserved.

20. Package Information

- **LT7689 (QFN-96pin)**

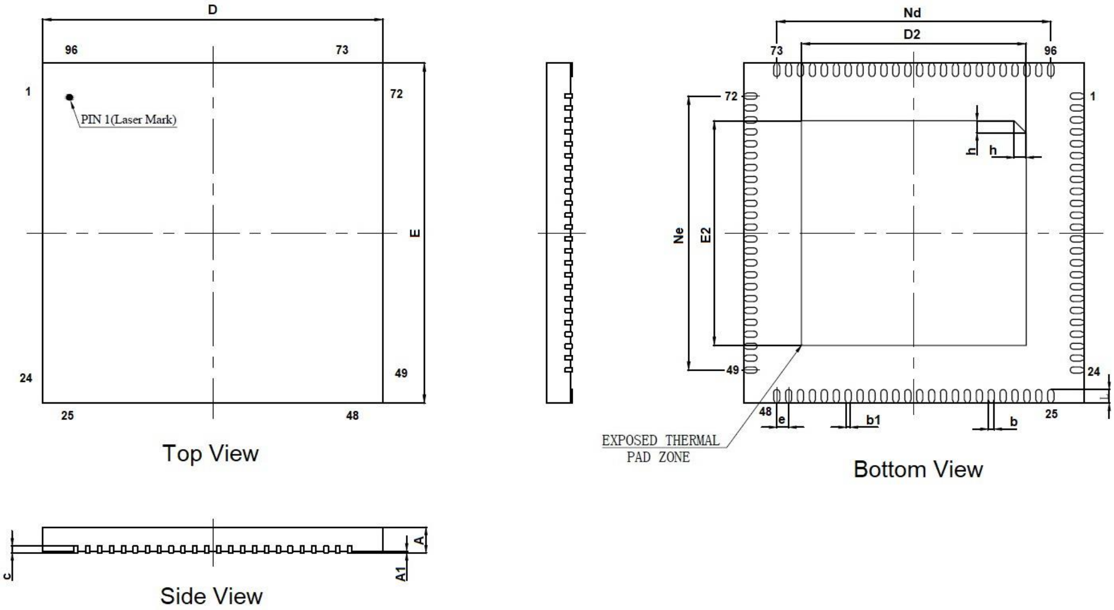


Figure 20-1: QFN-96Pin Outline

Note: When PCB layout, the heat pad of LT7689's back (thermal Pad Zone) must be directly grounded.

Table 20-1: QFN-96Pin Dimenstion

Symbol	Millimeter			Symbol	Millimeter		
	Min.	Nom.	Max		Min.	Nom.	Max
A	0.80	0.85	0.9	E	9.90	10.00	10.10
A1	-	0.02	0.05	Ne	8.05BSC		
b	0.13	0.18	0.23	L	0.35	0.40	0.45
b1	0.12REF			E2	6.50	6.60	6.70
c	0.18	0.20	0.25	h	0.30	0.35	0.40
D	9.90	10.00	10.10	Nd	8.05BSC		
D2	6.50	6.60	6.70	Body Size	7.5*7.5		
e	0.35BSC						

21. Copyright